

PPG PROJECT: RECOVERABLE ASSETS (RA) & INVENTORY RISK
MANAGEMENT (IRM)

MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI

CHERYL CHEONG KAH VOON

MUHAMMAD AFIQ DANIAL BIN ROZAIDIE

NURUL ASYIKIN BINTI KHAIRUL ANUAR

PRAVINRAJ A/L SIVABATHI

BRENDAN CHIA YAN FEI

NAJMA SHAKIRAH BINTI SHAHRULZAMAN

ABSTRACT

Inventory mismanagement and fragmented tracking across legacy systems cause financial risks and operational inefficiencies in manufacturing. This study designs and implements an end-to-end cloud data pipeline using Microsoft Azure and Power BI to automate the detection of Recoverable Assets (RA) and Magna Carta (MC) inventory risks at PPG Industries. Academically, it bridges the gap between data engineering theory and industry application, while practically replacing manual tracking with a scalable automated solution. The methodology employs an Iterative SDLC integrated with a three-layer Medallion Architecture (Bronze, Silver, Gold) and a multi-dimensional galaxy schema. Findings demonstrate that the pipeline successfully flags expiration risks, classifies RA or MC dead stock, calculates write-down provisions, and traces upstream material stockouts to downstream sales orders. Conclusively, this study contributes a centralized framework that ensures data integrity and auditability. Managerially, teams can leverage interactive dashboards to proactively mitigate financial losses and optimize inventory levels.

ABSTRAK

Pengurusan inventori yang lemah dan penjejakan terpecah-pecah merentas sistem legasi mewujudkan risiko kewangan serta ketidakcekapan operasi dalam pembuatan. Kajian ini mereka bentuk saluran paip data awan menggunakan Microsoft Azure dan Power BI untuk mengautomasikan pengesanan Aset Boleh Pulih (RA) dan risiko inventori Magna Carta (MC) di PPG Industries. Secara akademik, ia merapatkan jurang antara teori dan praktikal, manakala secara praktikal ia menggantikan penjejakan manual dengan penyelesaian automatik berskala. Metodologi kajian menggunakan Kitaran Hayat Pembangunan Sistem (SDLC) Berulang bersandarkan Seni Bina Medallion tiga lapisan (Gangsa, Perak, Emas) dan skema bintang pelbagai dimensi. Penemuan menunjukkan saluran paip ini berjaya mengenal pasti risiko luput, mengklasifikasikan RA atau stok mati MC, mengira peruntukan hapus kira, serta menjejaki impak kehabisan stok kepada pesanan jualan. Kesimpulannya, kajian ini menyumbang rangka kerja berpusat yang menjamin audit dan integriti data. Implikasi pengurusan membolehkan pasukan memanfaatkan papan pemuka interaktif untuk mengurangkan kerugian kewangan secara proaktif dan mengoptimumkan tahap inventori.

ABSTRACT	III
ABSTRAK	IV
LIST OF TABLES	IX
LIST OF FIGURES	XI
CHAPTER 1 INTRODUCTION	1
1.1 Introduction	1
1.2 Problem	2
1.3 Project Aim	2
1.4 Project Objectives	3
1.5 Project Scope	3
1.6 Project Importance	4
1.7 Report Organization	5
CHAPTER 2 LITERATURE REVIEW	7
2.1 Introduction	7
2.2 Supply Chain Lifecycle and Inventory Management	7
2.2.1 Supply Chain Lifecycle Overview	7
2.2.2 Inventory Classification and Risk	8
2.3 Enterprise Resource Planning	9
2.4 Literature Review of Technology	10

2.4.1 Microsoft Azure for Cloud Data Engineering	10
2.5 Visualization	11
2.6 Microsoft Azure Comparison with Existing Platforms and Case Study	12
2.6.1 Overview of Microsoft Azure	12
2.6.2 Comparison of Azure with Existing Systems	13
2.6.3 Case Study: Azure in Manufacturing and Supply Chain	13
2.7 Microsoft Power BI	14
2.8 Summary	15
CHAPTER 3 METHODOLOGY	17
3.1 Introduction	17
3.2 Methodology choice and justification	18
3.3 Phases within the Chosen Methodology	19
3.3.1 Activity and Process for Each Phase	19
3.3.2 Design modelling	21
3.3.3 Design tools	23
3.3.4 Gantt Chart	24
3.3.5 Describe briefly the technology or tools used to develop the system	24
3.4 System requirement analysis: hardware and software	25
3.4.1 Software Requirements	26
3.4.1.1 Microsoft Azure Data Lake Storage Gen2 (ADLS Gen2)	26
3.4.1.2 Azure Databricks	26
3.4.1.3 Azure Synapse Analytics	26
3.4.1.4 Microsoft Power BI Desktop	27
3.4.1.5 Draw.io	27
3.4.1.6 Development Environment	27

3.4.2 Hardware Requirements	27
3.4.2.1 Client Machine Requirements	28
3.4.2.2 Cloud Infrastructure Requirements	28
3.4.2.3 Network Requirements	29
3.5 Chapter summary	29
 CHAPTER 4 ANALYSIS AND DESIGN	 30
4.1 Introduction	30
4.2 System Analysis	30
4.2.1 Case Study	30
4.2.2 System Requirements Gathering Techniques	31
4.3 System	33
4.3.1 Functional Requirements	34
4.3.2 Non-Functional Requirements	35
4.4 Current System Analysis	36
4.4.1 Current system and its Limitations	36
4.5 System Design	39
4.5.1 System Architecture	39
4.5.2 Explanation of System Components	41
4.5.3 System Design	43
4.5.4 Database Design	48
4.5.5 Interface Design	63
4.6 Summary	64
 CHAPTER 5 IMPLEMENTATION AND TESTING	 66
5.1 Introduction	66

5.2 System development	66
5.3 System Main Functions	68
5.3.1 Module 1: Raw Data Ingestion (Bronze Layer)	68
Table 5.2 below summarizes the quality check results for all six datasets.	69
5.3.2 Module 2: Data Transformation (Silver Layer)	71
5.3.3 Module 3: Gold Layer	78
5.4 Essential interfaces that show system's results and achievements	81
5.4.1 Power BI Visualization Layer	81
5.4.2 Data Serving Layer: Azure Synapse Analytics	85
5.4.3 Pipeline Execution Results	86
5.5 Testing	90
5.5.1 Black Box Testing	90
5.5.2 White Box Testing	101
5.5.2.1 Test Cases – Bronze Layer	101
5.5.2.2 Test Cases – Silver Layer	103
5.5.2.3 Test Case Gold Layer – Synapse SQL Views	110
5.5.3 User testing	113
5.6 Chapter summary	114
REFERENCES	116

LIST OF TABLES

TABLE NO.	TITLE	PAGE
TABLE 2.1	COMPARISON OF AZURE WITH EXISTING SYSTEM	13
TABLE 3.1	CLIENT MACHINE SPECIFICATIONS	28
TABLE 4.1	ERD RELATIONSHIP CARDINALITY SUMMARY	51
TABLE 4.2	3NF SCHEMA OF MATERIAL	52
TABLE 4.3	3NF SCHEMA OF SUPPLIER	52
TABLE 4.4	3NF SCHEMA OF FINISHED_PRODUCTS	53
TABLE 4.5	3NF SCHEMA OF INVENTORY_SNAPSHOT	53
TABLE 4.6	3NF SCHEMA OF PRODUCTION_ORDERS	53
TABLE 4.7	3NF SCHEMA OF PURCHASE_ORDERS	54
TABLE 4.8	3NF SCHEMA OF SALES_ORDERS	55
TABLE 4.9	GALAXY SCHEMA DESCRIPTION	56
TABLE 4.10	FACT_INVENTORY_STATUS DESCRIPTION	57
TABLE 4.11	DIM_DATE DESCRIPTION	58
TABLE 4.12	DIM_MATERIAL DESCRIPTION	59
TABLE 4.13	DIM_SALES_ORDER DESCRIPTION	60
TABLE 4.14	DIM_PRODUCTION DESCRIPTION	60
TABLE 4.15	DIM_SUPPLIER DESCRIPTION	61
TABLE 4.16	FACT_IMPACTED_ORDER DESCRIPTION	62
TABLE 4.17	SUMMARIZE RELATIONSHIP OF GALAXY SCHEMA	63
TABLE 5.1	TOOLS AND SERVICES USED FOR END-TO-END PIPELINE	67
TABLE 5.2	QUALITY CHECK RESULT	69
TABLE 5.3	DEFICIT STATE FOR MATERIAL BELOW REORDER	87
TABLE 5.4	RECOVERABLE ASSETS AND MAGNA CARTA OBSOLESCENCE CALCULATIONS	89
TABLE 5.5	DASHBOARD Q1 — MATERIALS BELOW REORDER LEVEL	91
TABLE 5.6	DASHBOARD Q2 — RECOVERABLE ASSETS	92

TABLE 5.7 DASHBOARD Q3 — MAGNA CARTA DEAD STOCK	93
TABLE 5.8 DASHBOARD Q4 — MAGNA CARTA EXPIRED	94
TABLE 5.9 DASHBOARD Q5 — TOTAL MC PROVISION AMOUNT	95
TABLE 5.10 DASHBOARD Q6 — UNFULFILLED PRODUCTION ORDERS	97
TABLE 5.11 DASHBOARD Q7: IMPACTED CUSTOMER SALES ORDERS	98
TABLE 5.12 END-TO-END PIPELINE FUNCTIONAL TESTS	99
TABLE 5.13 TEST CASES – BRONZE LAYER	101
TABLE 5.14 RECOVERABLE ASSET FLAG (IS_RECOVERABLE_ASSET)	103
TABLE 5.15 MC DEAD STOCK FLAG (IS_MC_DEAD_STOCK)	104
TABLE 5.16 MC EXPIRED FLAG (IS_MC_EXPIRED)	105
TABLE 5.17 MAGNA CARTA COMBINED FLAG (IS_MAGNA_CARTA)	106
TABLE 5.18 MC PROVISION CALCULATION(MC_PROVISION_AMOUNT)	106
TABLE 5.19 STOCKOUT RISK FLAGS (IS_BELOW_REORDER, IS_STOCKOUT_RISK)	107
TABLE 5.20 SALES ORDER IMPACT (Q7 – IMPACTED_ORDERS JOIN CHAIN)	108
TABLE 5.21 WINDOW FUNCTION – LATEST INVENTORY SNAPSHOT	109
TABLE 5.22 GOLD LAYER – SYNAPSE SQL VIEWS	110
TABLE 5.23 USER TESTING	113

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE
FIGURE 2.1	SCOR MODEL AND MANAGEMENT PROCESS (ADAPTED FROM SCC, 2010)	8
FIGURE 3.1	ITERATIVE SDLC MODEL (WISHDESK, 2024)	17
FIGURE 3.2	GENERAL MEDALLION ARCHITECTURE	22
FIGURE 3.3	GENERAL MEDALLION ARCHITECTURE	23
FIGURE 3.4	PROJECT SCHEDULE GANTT CHART	24
FIGURE 4.1	CURRENT PROCESS FOR INVENTORY MANAGEMENT SYSTEM	38
FIGURE 4.2	SYSTEM ARCHITECTURE	41
FIGURE 4.3	USE CASE DIAGRAM	43
FIGURE 4.4	ACTIVITY DIAGRAM	44
FIGURE 4.5	SEQUENCE DIAGRAM	46
FIGURE 4.6	DFD LEVEL 0 (CONTEXT DIAGRAM)	47
FIGURE 4.7	DFD LEVEL 1	48
FIGURE 4.8	ENTITY RELATIONAL DIAGRAM FOR TRADITIONAL METHODOLOGY	49
FIGURE 4.9	GALAXY SCHEMA DIAGRAM FOR ANALYTICAL	56
FIGURE 5.1	CONNECTING TO ADLS GEN2 AND LOADING RAW CSV FILES	68
FIGURE 5.2	QUALITY CHECK FUNCTION	69
FIGURE 5.3	LOAD VALIDATED BRONZE LAYER	70
FIGURE 5.4	VERIFY SUCCESSFUL LOAD	70
FIGURE 5.5	LOADING VALIDATED BRONZE DATA AND TYPE CASTING	71
FIGURE 5.6	12-MONTH ROLLING CONSUMPTION PER MATERIAL	72
FIGURE 5.7	12-MONTH ROLLING CONSUMPTION PER MATERIAL	72
FIGURE 5.8	RECOVERABLE ASSET (RA) FLAG	73
FIGURE 5.9	RECOVERABLE ASSET (RA) FLAG	74
FIGURE 5.10	MC FINANCIAL PROVISION CALCULATION	75

FIGURE 5.11 SETTING STOCKOUT RISK FLAG	76
FIGURE 5.12 IMPACTED OPEN SALES ORDERS DUE TO STOCKOUT	77
FIGURE 5.13 WRITING OUTPUTS TO SILVER LAYER	78
FIGURE 5.14 CREATING GOLD SCHEMA	78
FIGURE 5.15 CREATING DIMENSION VIEWS IN THE GOLD LAYER	79
FIGURE 5.16 CREATING FACT INVENTORY STATUS VIEW	80
FIGURE 5.17 CREATING IMPACTED ORDERS VIEW	80
FIGURE 5.18 DASHBOARD VIEW FOR REORDER LEVELS AND RECOVERABLE ASSETS	81
FIGURE 5.19 DASHBOARD VIEW FOR MAGNA CARTA DEAD STOCK, EXPIRED MATERIALS, AND FINANCIAL PROVISIONS	82
FIGURE 5.20 DASHBOARD VIEW FOR CURRENT STOCK VS SAFETY STOCK NEEDED	83
FIGURE 5.21 DASHBOARD VIEW FOR OPEN SALES ORDERS IMPACTED BY STOCKOUT	84
FIGURE 5.22 AZURE SYNAPSE ANALYTICS WORKSPACE OBJECT EXPLORER DISPLAYING THE SERVERLESS SQL DATABASE AND COMPILED RELATIONAL GOLD SCHEMA VIEWS	85
FIGURE 5.23 DATABRICKS PIPELINE EXECUTION OUTPUT HIGHLIGHTING MATERIALS CRITICALLY BELOW REORDER LEVELS AND CURRENT STOCKOUTS	87
FIGURE 5.24 PIPELINE OUTPUT SUMMARY DETAILING RECOVERABLE ASSETS AND TOTAL MAGNA CARTA PROVISION REQUIREMENTS	88
FIGURE 5.25 AUTOMATED MAPPING OF UNFULFILLED CUSTOMER SALES ORDERS DIRECTLY IMPACTED BY RAW MATERIAL STOCKOUTS.	89

LIST OF ABBREVIATIONS

ANN - Artificial Neural Network

GA - Genetic Algorithm

PSO - Particle Swarm Optimization

MTS - Mahalanobis Taguchi System

MD - Mahalanobis Distance

TM - Taguchi Method

UTM - Universiti Teknologi Malaysia

XML - Extensible Markup Language

ANN - Artificial Neural Network

GA - Genetic Algorithm

PSO - Particle Swarm Optimization

CHAPTER 1

INTRODUCTION

1.1 Introduction

Organizations are increasingly turning to data-driven methods because of digital transformation, maximizing operational efficiencies, and making better decisions based on the use of data and associated analytics. The amount of data created and managed by enterprise systems within enterprises is expected to grow significantly over the next few years. This phenomenon is particularly important in manufacturing and supply chain operations, where significant amounts of data are generated every moment. Consequently, there has been a rising demand for scalable, automated data processing technologies that enable organizations to scale effectively as they process increasing amounts of data daily.

PPG Industries is known as a global leader in paints, coatings, and specialty materials that operate in highly data intensive environments. The company generates large volumes of data from inventory management, procurement, production, and sales systems. Managing this data effectively is essential to ensure optimal inventory levels, reduce operational costs, and maintain supply chain efficiency. However, handling such complex and distributed data requires a structured and scalable data engineering solution.

The project's focus will be to develop an end-to-end data engineering pipeline for managing recoverable assets (RA) and inventory risk management (IRM) utilizing Microsoft Azure and Microsoft Power BI. The system will be developed to integrate data from multiple data sources, implement business rules to identify damaged and obsolete inventory, and deliver analytical output for visualization. The project will demonstrate how applying modern data engineering techniques within a layered architecture will improve inventory management and provide support for data-driven decision making in the manufacture of industrial products.

1.2 Problem

In manufacturing companies such as PPG Industries, it is crucial to effectively manage inventory by tracking the use of raw materials to maintain a smooth production flow and minimize costs. However, the operational data that inform current actions at PPG come from multiple systems like procurement, production, warehouse and sales where information is sometimes stored in several different locations or formats, which causes a fragmented approach to data mining. As a result of these fragmented systems, it is difficult to accurately monitor inventory, identify excess inventory, and predict future inventory shortages. Excessive inventory creates financial losses for the company and disrupts customers' orders.

Specifically, the challenges include identifying RA, identifying dead stock and tracking expired materials, all of which will likely require a financial write-off called a Magna Carta (MC) provision. Current manual inventory management processes rely on time-consuming manual analysis and periodic reports subject to human error, resulting in longer processing times. A solution is proposed based upon an established end-to-end data engineering pipeline that integrates data from various sources using business logic to classify inventory and subsequently creates analytics-ready data sets for visualization in Microsoft Power BI. By implementing this data engineering solution, PPG will gain timely, accurate and actionable business intelligence, which will aid in making timely and effective decisions leading to improved operational efficiency.

1.3 Project Aim

The aim of this project is to design and develop an end-to-end data engineering pipeline for automating the detection of Recoverable Assets (RA) and Magna Carta (MC) inventory risk at PPG Industries, leveraging Microsoft Azure for scalable cloud-based data processing and Microsoft Power BI for interactive analytical reporting, in order to support timely and data-driven inventory management decisions.

1.4 Project Objectives

The objectives of this project are as follows:

- a) To analyze the business logic of RA and MC in the context of PPG Industries' inventory management
- b) To design and implement a three-layer Medallion Architecture pipeline (Bronze, Silver, and Gold) using Microsoft Azure
- c) To build a galaxy Schema data model to organize inventory and financial data to allow the calculation of the total amount of MC provision and determine if a material is a Recoverable Asset or dead stock.
- d) To develop an interactive Power BI dashboard that answers 7 predefined business questions, visualizing inventory risk, RA status, MC provision totals, and the impact of material stockouts on open sales orders.

1.5 Project Scope

This project focuses on designing and implementing an end-to-end data engineering pipeline for Recoverable Assets (RA) and Inventory Risk Management (IRM) at PPG Industries using Microsoft Azure and Microsoft Power BI. The scope of this project covers the following:

- a) This project involves six synthetic datasets simulating PPG's operational systems, namely materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, and sales_orders.csv. No actual PPG operational data is used.

- b) The data pipeline is structured into three layers following the Medallion Architecture approach using Microsoft Azure services: Bronze (Raw Ingestion), Silver (Transformation), and Gold (Analytics-Ready).
- c) The cleaned and processed data is loaded into a galaxy schema data warehouse, with a central fact table (fact_inventory_status) storing RA/MC flags and financial values per material per month.
- d) A Power BI dashboard is developed to answer seven predefined business questions (Q1 to Q7) related to inventory status, RA classification, MC qualification, provision amounts, and sales order impact.
- e) This project does not cover real-time data streaming, live integration with PPG's actual production systems, or procurement recommendation systems beyond what is defined in the business logic.

1.6 Project Importance

This project holds significant value from both industry and academic perspectives

From the industry perspective, inventory mismanagement is a critical financial risk for manufacturing companies like PPG Industries. Materials that remain unconsumed for long periods tie up capital, incur storage costs, and may eventually require financial write-downs. These risks often go unnoticed until significant losses have been incurred without a structured data pipeline to detect and flag such materials automatically. This project directly addresses that gap by automating the detection of Recoverable Assets and Magna Carta materials. This allows PPG's finance and supply chain teams to act before write-downs become unavoidable. The ability to trace stockout risks to specific customer sales orders further strengthens supply chain resilience and customer service reliability.

From the data engineering perspective, this project demonstrates the practical

application of modern cloud-based data pipeline design using Microsoft Azure. It shows how raw operational data can be ingested, cleaned, transformed using domain-specific business logic. This is then delivered as an analytics-ready data model that reflects real-world industry practices in data engineering.

From the academic perspective, this project provides students with hands-on experience in building a complete end-to-end data pipeline, from raw data ingestion to dashboard visualization. It bridges the gap between data engineering concepts learned in the classroom and the practical requirements of industry, fulfilling the objectives of the Special Topic in Data Engineering (SECP3843) course.

1.7 Report Organization

This report is organized into five chapters, each addressing a specific component of the project.

Chapter 1: Introduction

This chapter presents the background of the study, focusing on inventory management challenges in PPG Industries, along with the problem statement, objectives, and overall structure of the report.

Chapter 2: Literature Review

This chapter discusses existing research and technologies related to data engineering, cloud computing, and inventory analytics. It provides a theoretical foundation for the project by reviewing relevant concepts such as data pipelines, ETL processes, and big data architectures.

Chapter 3: Methodology

This chapter explains the design and implementation of the proposed system. It details the data pipeline architecture, including the Bronze, Silver, and Gold layers, and describes the tools used, such as Microsoft Azure and Microsoft Power BI.

Chapter 4: Results and Discussion

This chapter presents the results of the implemented system, including dashboards and analytical outputs. It evaluates how effectively the system addresses inventory issues such as recoverable assets, dead stock, expired materials, and stockout risks.

Chapter 5: Conclusion and Recommendations

This chapter summarizes the findings of the project and highlights its contributions. It also provides recommendations for future improvements, including potential enhancements in scalability, automation, and real-time analytics.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Good inventory management is very important for modern companies to ensure that the production operation can run efficiently, reduce operational waste and optimize financial performance. Industrial organizations must handle the excess, unused and expired raw materials to prevent stagnation. However, it is also a big challenge to capture these metrics, such as the growing of operational data across the organization structure that often creates severe bottlenecks for business.

To solve this problem, data engineers play a significant role by automating processes like data ingestion, data quality validation and structure the workflow. Utilizing modern cloud computing such as Microsoft Azure will allow organizations to build scalable end-to-end data pipelines that can centralize the multi-source operational task. Additionally, tools like Power BI can translate this business ready layer into interactive visualizations.

This chapter shows a complete review of relevant academic literature, lifecycles of the supply chain, enterprise resource planning limitations, and modern big data engineering paradigms. With this element, it can establish the theoretical and technological framework to solve the problem.

2.2 Supply Chain Lifecycle and Inventory Management

This section explains the supply chain lifecycle and how inventory is managed.

2.2.1 Supply Chain Lifecycle Overview

Supply Chain Management (SCM) consists of several stages from the procurement of raw materials to delivery and distribution of the products to the customers. The SCOR

(Supply Chain Operations Reference) model as shown in Figure 2.1 is used as a reference for organizations to align supply chain management with key businesses and goals.

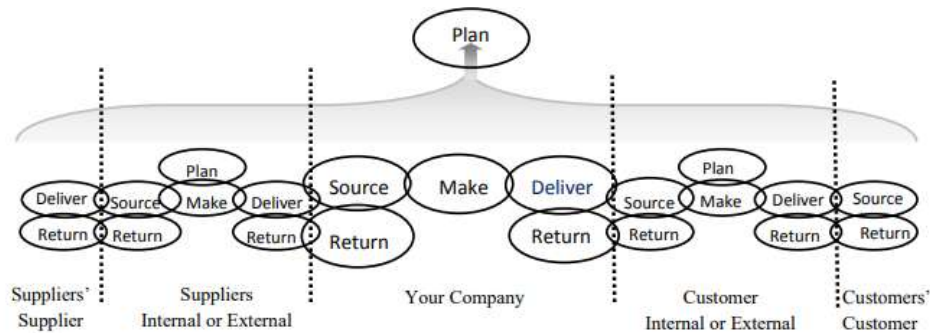


Figure 2.1 SCOR Model and Management Process (Adapted from SCC, 2010)

There are five main processes in the model which are plan, source, make, deliver, return. A plan refers to the initial stage of the supply chain where it is needed to carefully review and plan usage of resources to meet customer demands. Source refers to the procurement of materials and services needed to create the product. The process of making is the manufacturing of goods. Deliver focuses on transporting goods to the customer. Return handles the flow of goods back from the customer. The enabled process supports the five other processes by managing the activities in the supply chain. The SCOR Model is used to recognize and eliminate unnecessary and wasteful practices throughout the supply chain (Ntabe et al., 2015).

2.2.2 Inventory Classification and Risk

Businesses have different types of inventories which are:

1. Raw Material Stock – used for production of a product.
2. Semi-finished/WIP (Work-In-Progress) Stock – goods that is undergoing production process and have yet to be completed.
3. Finished Stock – stocks that have completed the production process and will be

shipped to the customers.

4. Consumables – materials not used directly to produce the product such as cleaning items.

The inventory can then be classified into active, slow-moving, obsolete, expired stock. Active stock refers to high-demand items which are sold in a standard turnover period. Slow Moving Inventory (SMI) are stocks that are low demand usually over a period of six to twelve months. Obsolete or dead stocks are items that are no longer in trend or replaced by new technology which expects no future demands. Expired stock are perishable goods that have reached past its “best before” date and can no longer be used (Silver, Pyke & Thomas, 2017).

Holding costs refers to the costs of physically holding the inventory which then affects the capital, inventory service, storage space, and inventory risk costs. Stockout costs are costs when the inventory amount fails to meet the demand of customers. This would lead to a loss of sales and customers, which is not ideal (Kamisli Ozturk, 2020).

2.3 Enterprise Resource Planning

Enterprise Resource Planning (ERP) is an integrated system that uses a single platform to handle major business processes such as inventory, production and sales. It makes it possible to share data in real time which enhances decision-making and operational effectiveness in the industry environments (Jawad & Balázs, 2024). ERP systems also help organizations integrate data from different functional areas into a centralized database, allowing better visibility and control over business operations.

However, ERP systems have difficulty handling complicated business rules including historical analysis and large-scale data processing. According to studies, ERP systems typically need to be integrated with other data processing tools to support advanced analytics and decision making because it offers limited analytical flexibility. To address these issues, organizations combine ERP systems with cloud-based data engineering platforms such as Microsoft Azure to transform raw ERP data into analytics-ready

datasets

2.4 Literature Review of Technology

The successful and reliable implementation of end-to-end pipeline requires a strong, scalable and integrated technology solution. For this PPG project, the architecture relies on advanced visualization tools to process inventory and supply chain data. The primary technologies selected for this project are Microsoft Azure and Microsoft Power BI.

2.4.1 Microsoft Azure for Cloud Data Engineering

Microsoft Azure is a complete cloud computing platform that provides a variety of services for analytics, storage and computing. Azure is also highly used because of its ability to seamlessly support a multi layered data architecture which is the medallion architecture.

- **Data Ingestion and Storage (Bronze Layer):**

Azure provides scalable data lake storage solutions capable of handling raw data ingestion from various operational systems. In this project, azure will be used to ingest and store raw data such as inventory purchase orders and sales orders directly from the CSV files while preserving their original state.

- **Data Transformation (Silver Layer):**

Azure data integration and transformation tools such as Azure Databricks enable complex data processing. This is important for applying specific business logic such as flagging Recoverable Assets (RA), identifying Magna

Carta (MC) Dead Stock and quantifying financial write-downs. Azure will ensure that this business logic is executed efficiently.

- **Data Warehousing (Gold Layer):**

For the analytic layer, Azure offers data warehousing services and relational databases like the Azure SQL Database and Azure Synapse Analytics. These tools can be used to store transform data in structured format to facilitate rapid querying for business intelligence.

2.5 Visualization

Data visualization plays an important role in business intelligence and data analysis. It helps to convert complex datasets into visual, making it easier for management to understand and make decisions. Charts, graphics, dashboards, and other visual elements help users identify the trends, patterns and anomalies more effectively than traditional table reports. According to Eboigbe et al. (2023), the integration of data analysis and business intelligence technology allows organizations to transform raw data into useful insights to support both strategic and operational decision.

Interactive dashboards are becoming more and more important in modern organizations because they provide centralized access to key performance indicators (KPIs) and operational data. The dashboard improves the accessibility of data and allows users to monitor business performance more effectively by presenting information in a visual format. The visualization tool also supports analysis activities, such as filtering, in-depth analysis and real-time monitoring, so that users can have a deeper understanding of business operations.

Within the inventory and supply chain environment, by providing a comprehensive view of the status of inventory, the change in inventory and the performance indicators,

operational visibility can be improved through data visualization. When visualized effectively, organizations can identify potential risks, monitor operational performance, and respond to changing business conditions faster. Dey (2023) point out that the flow of real-time information and visibility are key factors in improving the flexibility of the supply chain and allowing for timely responses to interruptions and uncertainties.

Various business intelligence platforms provide visualization functions to support data analysis and reporting. These platforms help organizations to convert processed data into useful dashboards and reports, enabling better monitoring, analysis and decision-making. Usage of data visualization not only improves the interpretation of analysis results but also improves communication among stakeholders by presenting information clearly and easily.

Overall, by improving operational visibility, enhancing the analysis process and promoting data-driven decision-making, data visualization becomes more important to the modern business intelligence environment. The combination of visualization technology and interactive dashboards helps organizations to gain more value from data and improve their overall business performance.

2.6 Microsoft Azure Comparison with Existing Platforms and Case Study

2.6.1 Overview of Microsoft Azure

Microsoft Azure serves as a cloud platform that supports the storage, processing, and integration of data throughout this project. Azure offers a scalable and reliable platform for creating end-to-end pipelines, allowing organizations to process large amounts of data efficiently while maintaining flexibility, accessibility, and performance.

Azure is the primary cloud infrastructure platform for the Recoverable Assets (RA) and Inventory Risk Management (IRM) project to store and process data and to enable analytical operations throughout the three tiers of the Medallion Architecture. The Gold layer of the pipeline is stored in Azure, so that it can serve as a central source for downstream reporting and dashboard building. This cloud-based setup improves data

availability and ensures that business users can always access updated and consistent information.

One of the main advantages of Azure is its seamless integration with Microsoft Power BI. This integration enables automatic data to refresh, ensuring that inventory records, shortage indicators, and expired stock analysis remain current and relevant for decision-making. In addition, Azure supports scalable data handling, which is important for managing multiple datasets and performing transformations across the bronze, silver, and gold layers of the Medallion Architecture.

2.6.2 Comparison of Azure with Existing Systems

Table 2.1 Comparison of Azure with existing system

Platform	Pipeline Support	Inventory Business Logic	Power BI Integration	Cost	Scalability
Microsoft Azure	Full end-to-end via ADF & Databricks	Custom logic implementable	Native & seamless	Moderate, pay-as-you-go	High
SAP Data Intelligence	Supported but complex to configure	Built-in but rigid, limited customization	Indirect, requires middleware	Very high licensing cost	Medium
Manual Excel Processing	No automation, fully manual	Manually applied, error-prone	Manual export required	Low but time-intensive	Very Low

As shown in the table, Microsoft Azure provides the most balanced solution for this project. Unlike SAP Data Intelligence, which carries high licensing costs and offers limited flexibility for custom business rules such as PPG's 12-month RA classification and 9-month MC dead stock logic, Azure allows these rules to be implemented freely within Azure Databricks notebooks. Compared to the current manual Excel-based approach used at PPG, Azure eliminates human error and significantly reduces processing time by automating the entire pipeline from ingestion to visualization.

2.6.3 Case Study: Azure in Manufacturing and Supply Chain

A relevant real-world application of Microsoft Azure in a manufacturing context was

demonstrated by Mathur et al. (2024), who evaluated the use of cloud-based supply chain solutions to automate inventory tracking and operational reporting. The study found that organizations adopting Azure-based pipelines reported significant improvements in data processing speed and a notable reduction in manual reporting effort. This is directly aligned with the problem faced by PPG Industries, where manual inventory analysis causes delays and is prone to human error.

Additionally, Microsoft (2023) introduces Azure Data Factory for automating data from various operational sources into a centralized analytical store. In the context of supply chain, that means the aggregation of data from procurement, production, warehouse, and sales systems which is exactly the multi-source data problem that PPG RA and IRM project are facing.

These are real world examples that show that Azure is not just theoretically good, but it has been practically confirmed in manufacturing and supply chain environments akin to PPG's operation. Therefore, Microsoft Azure is critical to providing a modern, cloud-based, data engineering solution at the core of PPG's inventory management system decision-logic.

2.7 Microsoft Power BI

Microsoft Power BI is a leading business analytics service that delivers interactive visualizations and business intelligence capabilities. Microsoft Power BI is software developed by Microsoft and can be used for several things in business intelligence, such as extracting, transforming and analyzing large amounts of data, building rational models to combine data from various sources; defining complex calculations using Data Analysis Expressions (DAX) and data visualization with interactive reports and dashboards (Wahyuni et al, 2024). It is chosen for its seamless integration with Microsoft Azure data services and its powerful semantic modeling features.

The key feature for choosing Power BI is that it provides real-time data visualization for users to monitor and analyze data dynamically. This feature plays a big part in data-

driven decision making like defining the RA/MC on materials since the data on the live dashboard will update automatically. Additionally, Power BI has a feature called Power Query for data transformation. This can help users to take out clean and prepare raw data before doing the analysis. On top of that, Power BI supports connecting to data warehouses and is highly compatible with the galaxy schema. It also allows us to define relationships between the central fact table that stores quantitative data and surrounding dimension tables which contain descriptive attributes.

Compared to other BI tools like Tableau and Looker. Power BI is using a user-friendly interface like only using drag and drop as a feature. Studies from Jayaraman & Jain (2024) state that with a user-friendly interface, it allows users from business analysts to executives to create interactive dashboards and reports that enable faster decision making. While Tableau is known for its advanced visualization capabilities and Looker for its centralized data modeling (LookML), Power BI emphasizes user-friendly, self-service visualization and offers free version and affordable enterprise plans for all kinds of users including students, small and medium enterprises, and large organizations.

2.8 Summary

This chapter discussed the main theories, related studies and technologies used for the PPG Inventory Risk Management project. It covered important concepts such as data engineering, Medallion Architecture (Bronze, Silver and Gold) and inventory management concepts like Recoverable Assets (RA), Magna Carta (MC) and stockout risk. It also reviewed previous studies such as statistical methods for detecting obsolete inventory, cloud-based data pipelines and systems like SAP Material Management to understand their strengths and limitations.

From the review, most existing solutions only focus on certain parts of the problem and do not provide a complete solution. Because of that, this project aims to combine data pipeline processing with inventory business logic using Microsoft Azure and Power BI. Overall, this chapter provides the basic understanding needed for designing the system in the next chapter and helps support better decision-making for inventory

management.

CHAPTER 3

METHODOLOGY

3.1 Introduction

This chapter describes the methodology for the development of Recoverable Assets (RA) and Inventory Risk Management (IRM) System. The methodology is a systematic approach to structuring the development of the system to achieve the project goals. The primary development approach was chosen to be an Iterative System Development Life Cycle (SDLC) as shown in Figure 3.1. with the Medallion Architecture because the development project required a large-scale data processing, transformation and analytics project.

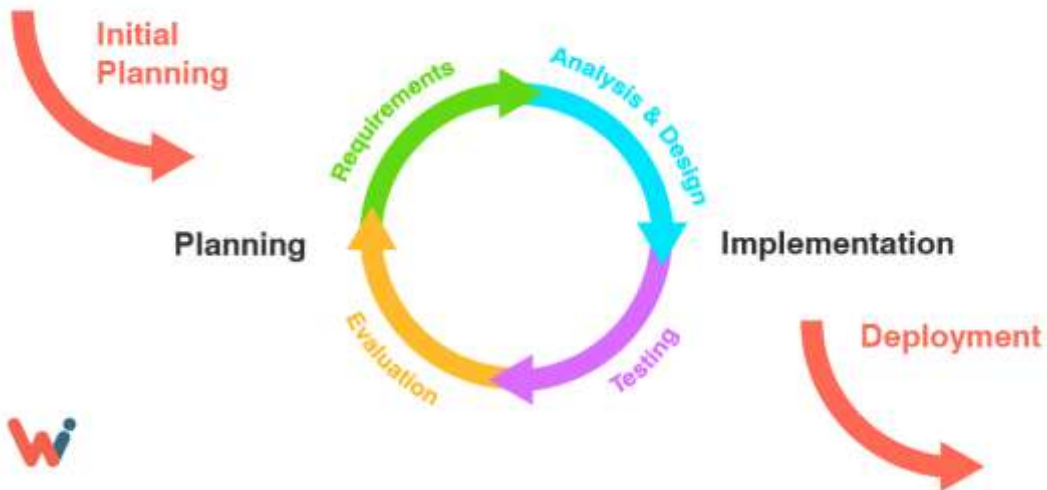


Figure 3.1 Iterative SDLC Model (WishDesk, 2024)

The chapter discusses the justification for selecting the methodology, the activities performed throughout each development phase, and the design models used to represent the system architecture and functionality. Additionally, the chapter outlines the technologies, tools, hardware needs and software needs to support the implementation of the proposed solution. The overall methodology guarantees the scalability, reliability, and the ability of the system to assist in effective inventory risk

management and decision-making processes.

3.2 Methodology choice and justification

For the development of the Recoverable Assets (RA) and Inventory Risk Management (IRM) system, the Iterative Systems Development Life Cycle (SDLC) was selected, structured around the Medallion Architecture data design pattern. This approach progresses through five systematic phases: Requirements, Design, Implementation, Deployment, and Testing.

This methodological framework was chosen because it aligns with established data engineering industry standards. The reasons for this decision are outlined below:

- **Layered Data Processing (Medallion Architecture):** A core requirement of this project is to handle different states of data quality and structure. The Medallion Architecture justifies itself by dividing the pipeline into three logical layers: Bronze, Silver, and Gold. This ensures that raw operational data from the six CSV files simulating PPG's systems can be securely ingested in its original state, systematically cleaned and transformed in the Gold layer, and finally loaded into an optimized galaxy
- schema ready for analytics.
- **Flexible, Iterative Approach to Handling Complex Business Processes:** Implementing PPG's business logic requires precision and continuous validation. Given that the system must accurately calculate 12-month consumption limits to flag Recoverable Assets, identify Magna Carta Dead Stock based on a 9-month non-consumption rule, and quantify financial write-down provisions, a traditional, rigid waterfall approach is inadequate. An iterative SDLC allows the development team to test, refine, and validate these complex transformation rules during the Implementation and Testing phases before final deployment.

- **Seamless Cloud and Visualization Integration:** The chosen methodology naturally supports the project's designated technology stack, specifically Microsoft Azure and Microsoft Power BI. By adopting a phased development approach, the transition from back-end data processing within Azure's cloud environment to the front-end visualization layer is carried out in a structured and manageable manner. This ensures that the final deliverable, an interactive Power BI dashboard, is capable of reliably querying the gold layer to address the seven predefined business questions regarding inventory status and stockout risks.

3.3 Phases within the Chosen Methodology

This section explains each phase of the chosen methodology and the application to the project. All the phases included in the Iterative SDLC model are Planning, Requirements, Analysis and Design, Implementation, Testing and Evaluation.

3.3.1 Activity and Process for Each Phase

Phase 1: Planning

The planning phase establishes the background, scope, and objectives of the projects. In this phase, the core problems faced by PPG were identified, which is the reliance on manual and fragmented inventory monitoring causes human errors and is difficult to scale as data volume increases.

Phase 2: Requirements Analysis

The project begins with defining requirements and refining them accordingly to the understanding of the system. The initial activities include identifying core business problems of inventory monitoring at PPG and defining seven business questions that the system must answer by the end of the project. Both functional and non-functional requirements are defined in this phase.

Phase 3: Analysis and Design

During the analysis and design phase, the project requirements gathered from stakeholders were converted into a detailed system architecture and data model design. The seven business questions provided by PPG were investigated to find the necessary data sources, transformation procedures, and analytical outputs. Functional requirements such as identifying Recoverable Assets (RA), Magna Carta (MC) Dead Stock, MC Expired Materials, stockout hazards and affected client orders were aligned with technical specifications.

The system architecture was created using the Medallion Architecture, which consists of bronze, silver and gold layers that are implemented within the Microsoft Azure ecosystem. Design tasks included drawing use case diagrams, activity diagrams, sequence diagrams, Data Flow Diagrams (DFD), Entity Relationship Diagrams (ERD), Third Normal Form (3NF) schemas and galaxy

schema models. The Gold layer structure was designed to allow analytical queries and Power BI reporting by separating data into fact and dimension tables. Dashboard prototypes were also constructed to guarantee that all seven business questions could be efficiently answered using interactive graphics.

Phase 4: Implementation

The implementation phase is where the Medallion Architecture pipeline is applied. Firstly, the Bronze layer is implemented by ingesting all data sources into Azure Data Lake Storage Gen2 while preserving the original CSV format. Next, the Silver layer is implemented in Azure Databricks to transform and clean the data using PySpark. Then, the transformed data is structured into a Galaxy Schema in the Gold Layer using Azure Synapse Analytics. Power BI is connected to Synapse to build dashboards that answer all seven business questions.

Phase 5: Testing

The testing phase was conducted to check that the proposed system met the functional

and non-functional requirements defined earlier. The data pipeline was tested to ensure that the ingestion process loaded the source datasets correctly into the Bronze layer and the Silver layer transformation processes followed the business rules for Recoverable Assets (RA), Magna Carta (MC) Dead Stock, MC Expired materials, stockout risk identification and affected sales order tracing. The Gold layer was also reviewed to ensure that the generated galaxy schema allowed fast querying and reporting. Furthermore, the Power BI dashboard was validated to ensure that all visualizations answered the seven preset business questions and provided meaningful information for inventory risk management and decision-making.

Phase 6: Evaluation

The evaluation phases assess the system that has undergone the testing phase and decide if it meets the requirements. This phase occurs at the end of each development cycle to determine if the system must go through another iteration. The pipeline output is reviewed and if the expected result is obtained. The completed dashboard must be verified to see if the system outputs are accurate, navigable, and is able to be used for decision making. If the system has not met the requirements yet, refinements are done in subsequent iterations until the final system is technically sound and aligns with PPG's operational needs.

3.3.2 Design modelling

The proposed solution uses Medallion Architecture and Galaxy Schema as main design models to support data processing and analytical reporting activities.

Medallion Architecture is a layered data modeling framework that organizes data into bronze, silver, and gold layers. This framework enables data to be progressively refined from raw operational data into business-ready datasets which can be used for analytical processing and reporting. The layered structure also improves data quality management, maintainability, and scalability within the data engineering workflow.

In addition, the proposed solution uses a Galaxy Schema for multidimensional data

modelling. A Galaxy Schema consists of multiple fact tables that share common dimension tables which allow different business processes to be analyzed within a single analytical environment. This modelling approach supports efficient querying, reduces data redundancy, and improves flexibility for business intelligence applications.

Medallion Architecture and Galaxy Schema work together to provides a structured foundation for data management, analytical processing, and reporting requirements within the proposed solution.

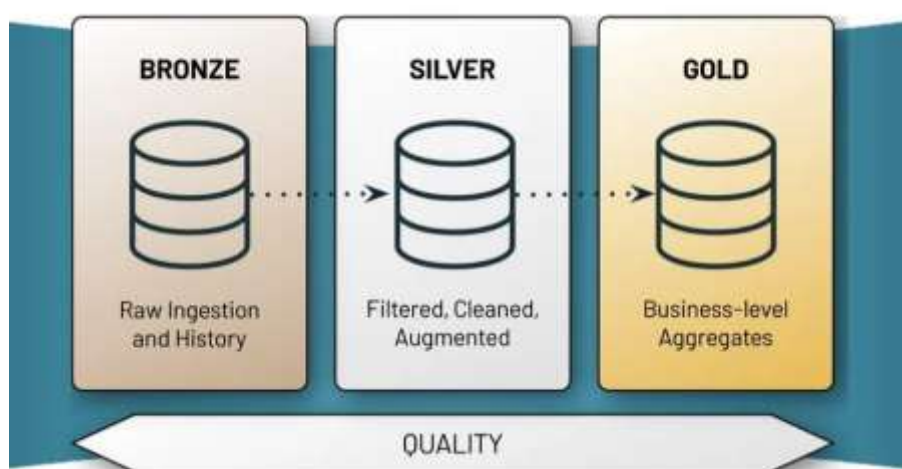


Figure 3.2 General Medallion Architecture

Figure 3.2 illustrates the Medallion Architecture used in the proposed solution. The architecture organizes data into three logical layers which are Bronze, Silver, and Gold, to support progressive data refinement and analytical processing. This architecture improves data quality, maintainability, and scalability throughout the data engineering workflow.

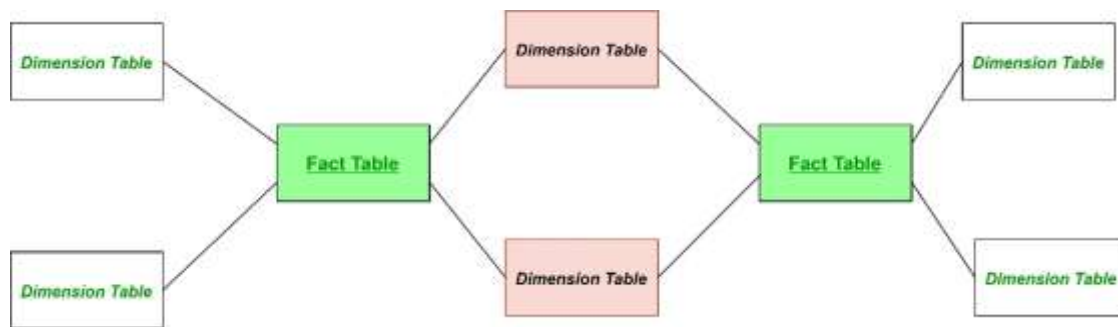


Figure 3.3 General Medallion Architecture

Figure 3.3 presents the Galaxy Schema design used for analytical modelling. The galaxy scheme consists of several fact tables that share common dimension tables. It enable multidimensional analysis and supports various business intelligence requirements.

3.3.3 Design tools

In the design stage of the proposed solution, several design tools and cloud technology are selected. These tools support system architecture design, data modeling, analytical processing and visual planning.

Draw.io is used to draw the system architecture diagrams, medal architecture diagrams, galaxy models and other system design workpieces. This tool provides a flexible platform for visualizing the system structure and data flow.

Microsoft Azure Services were selected as the cloud platform to support data storage, transformation, and analytical processing requirements. The Azure environment provides scalable and integrated services suitable for modern data engineering solutions.

Microsoft Power BI was selected as the visualization platform for dashboard development and analysis reports.

Together, these tools provide a cloud-based integrated environment that can support the design and implementation requirements of the proposed data engineering solution..

3.3.4 Gantt Chart

In order to make sure this project runs smoothly, a schedule consists of every phase and its duration is made. The team members that oversee leading the task are also determined to make sure all phases are supervised. Figure 3.4 show the Gantt Chart with the assigned task for all the members.



Figure 3.4 Project Schedule Gantt Chart

3.3.5 Describe briefly the technology or tools used to develop the system

To develop a good end-to-end data pipeline and dashboard for the Recoverable Assets (RA) and Inventory Risk Management system, this project uses a modern cloud-based technology. The Primary tools than been selected for the development of this system are Microsoft Azure and Microsoft Power BI.

1. Microsoft Azure (Data Pipeline and Storage)

Microsoft Azure serves as the core infrastructure to develop the three-layered data engineering pipeline which is bronze, silver, and gold. It is used to handle the backend data workflow

- Data Ingestion (Bronze Layer) :

Azure storage solutions are used to ingest and store the six raw synthetic CSV files which is materials, inventory snapshots, purchase orders, production orders,

suppliers, and sales orders as a data lake while doing some initial quality data checks

- Data Transformation (Silver Layer):

Azure data integration tools are used to process the raw data and apply PPG business logic. This includes calculating 12-month consumption limits, identifying stockout risks, and creating flags for Recoverable Assets and Magna Carta which are the dead stock and expired materials.

- Data Warehousing (Gold Layer):

Azure is used to load the curated data into the structured galaxy schema to make it optimized for querying.

2. Microsoft Power BI (Data Visualization and Analytics)

Microsoft Power BI is used as the front-end visualization tool to develop a dashboard. It will directly connect to the gold layer in azure to fetch the data.

- Power BI is utilized to design an interactive dashboard that translates the processed data into actionable insights, specifically engineered to answer the seven core business questions regarding stock levels, financial provisions, and open customer orders impacted by shortages.

3.4 System requirement analysis: hardware and software

This section outlines the hardware and software requirements for developing the Recoverable Assets (RA) and Inventory Risk Management (IRM) System and implementing it. The proposed solution involves leveraging Microsoft Azure cloud services and Microsoft Power BI for data ingestion, transformation, storage, analytics, and visualization on dashboards, which necessitates the proper use of software platforms and computing resources to ensure data flows seamlessly into the system and can be processed for analysis. The Medallion Architecture pipeline has to be able to run efficiently and deliver accurate analytical information for inventory monitoring and

decision-making.

3.4.1 Software Requirements

These software requirements include cloud services, data processing platforms, development tools, and visualisation software required to implement the proposed system. All the data engineering operations are performed on Microsoft Azure, and reporting and business intelligence are carried out via Microsoft Power BI.

3.4.1.1 Microsoft Azure Data Lake Storage Gen2 (ADLS Gen2)

The primary storage is provided by Azure Data Lake Storage Gen2. It keeps all datasets across all layers of the Bronze, Silver and Gold Medallion Architecture. The service offers scalability, ability to handle structured and semi-structured data, and data consistency across the data pipeline. All raw source files are initially contained in the Bronze layer and then undergo processing and transformation to create more valuable datasets in the Silver and Gold layers.

3.4.1.2 Azure Databricks

Azure Databricks is used as the main data processing and transformation platform. It uses PySpark for data cleaning, validation, filtering, transforming and application of business rules. At this phase, inventory related calculations like Recoverable Assets (RA) identification, Magna Carta (MC) Dead Stock detection, MC Expired Materials analysis and Stockout risk assessment are done. With support for efficient processing of large datasets and the generation of trusted datasets in the Silver layer, Databricks helps organizations process large datasets.

3.4.1.3 Azure Synapse Analytics

Azure Synapse Analytics is the analytical and data warehousing capabilities of the Gold layer. Azure Databricks curated datasets are loaded into Synapse and are stored as a galaxy schema with fact and dimension tables. Synapse delivers optimized query performance and ability to support advanced analytical workloads for reporting, key

performance indicators (KPIs) monitoring, and inventory risk analysis.

3.4.1.4 Microsoft Power BI Desktop

The front-end visualization and business intelligence tool is Microsoft Power BI Desktop. It integrates directly with data stored in Azure Synapse Analytics and facilitates creating interactive dashboards, charts, KPI indicators, and management reports. Users can use Power BI to analyze inventory performance, monitor risk of stockout, look into customer orders impacted by inventory issues, and evaluate the financial impacts of inventory.

3.4.1.5 Draw.io

Draw.io is used to create system architecture diagrams, use case diagrams, activity diagrams, sequence diagrams, Data Flow Diagrams (DFD), and Entity Relationship Diagrams (ERD) during the system design phase. These diagrams help to visually represent the functionality and data flow in the system during the project.

3.4.1.6 Development Environment

The development environment is running on the Windows 11 Professional operating system, Microsoft Office Professional Plus 2021 (for documentation), Google Chrome (for using Azure cloud services and Power BI resources). Transformation logic and business rules are implemented in Python and PySpark libraries in Azure Databricks.

3.4.2 Hardware Requirements

As the proposed system is cloud-based, most computational resources are provided by Microsoft Azure. Therefore, only a client workstation is required for development, administration, monitoring, and dashboard access.

3.4.2.1 Client Machine Requirements

Table 3.1 Client Machine Specifications

Component	Specification
Processor	Intel Core i5
Memory (RAM)	16GB
Storage	512 GB SSD
Operating System	Windows 11
Graphics	Intel UHD Graphics
Browser	Google Chrome
Internet Connection	Minimum 20 Mbps Broadband

The hardware specification meets performance requirements for access to Microsoft Azure services, for running Microsoft Power BI Desktop, for creating system documentation, and for conducting cloud-based data engineering activities.

3.4.2.2 Cloud Infrastructure Requirements

This proposed system can avoid the use of dedicated physical servers, as Microsoft Azure offers fully managed cloud infrastructure. Azure Data Lake Storage Gen2 offers scalable storage, Azure Databricks offers distributed computing capabilities for data

transformation, and Azure Synapse Analytics offers enterprise-level analytics processing capabilities. Cloud infrastructure can automate resource allocation for workloads, lower hardware maintenance and operating expenses.

3.4.2.3 Network Requirements

A reliable and secure connection to the network is needed to maintain access to Azure cloud services, data processing and dashboard reporting. The recommended minimum internet speed is 20 Mbps for smooth data transfers, accessing cloud services, refreshing dashboards, and team collaboration. Secure network communication also provides reliable connections between Azure services and end-user applications.

3.5 Chapter summary

This chapter discussed the methodology used to develop the Recoverable Assets(RA) and Inventory Risk Management (IRM) System. The iterative System Development Life Cycle (SDLC) integrated with the Medallion Architecture was selected to support the development of a scalable and reliable data engineering solution. The chapter explained the justification for the chosen methodology, the activities performed in each development phase, and the design models used to represent system functionality and architecture.

In addition, the chapter presented the system architecture, use case diagrams, design tools, project schedule, and technologies used throughout the development process. Microsoft Azure was selected as the primary platform for data ingestion, transformation, and warehousing, while Microsoft Power BI was used for data visualization and analytics. The hardware and software requirements were also identified to ensure successful system implementation. Overall, the methodology and design presented in this chapter provide a structured foundation for the development and evaluation of the proposed system.

CHAPTER 4

ANALYSIS AND DESIGN

4.1 Introduction

This chapter presents the system analysis and design phases for the Recoverable Assets (RA) and Inventory Risk Management (IRM) system. System analysis examines the company's operation, identifying the core business issues, and defining the data collection strategies to build the pipeline. With the analysis, the technical scope is structured following the detailed functional and non-functional requirements. This foundation serves as the master blueprint for developing the multi-layer Azure pipeline and the Power BI dashboard later in the project.

4.2 System Analysis

This section provides a detailed system analysis process on how we gain solutions for PPG case study. In this section, it outlines the challenges within the selected industrial case study, identifying the organizational structure for PPG and details the process of requirement gathering methodology. This section also specifies the functional and non-functional requirements that are necessary to build an automated Lakehouse pipeline.

4.2.1 Case Study

The focus of this industrial case study is PPG's supply chain and inventory asset division, specifically identifying the material shortages and inventory obsolescence. Holding unused raw materials will exposes PPG with financial risk from spoil material and expiration.

This project addresses two critical inventory vulnerabilities:

1. **Inventory Obsolescence:** Identifying materials held in excess of 12 months of historical consumption (Recoverable Assets), flagging slow-moving stock with no consumption for over 9 months (Magna Carta Dead Stock), and tracking materials past their chemical viability date (Magna Carta Expired) to establish automated financial write-down provisions.
2. **Stockout Risks:** Detecting when current raw materials drop low to supplier delivery leads to times and systematically tracing the downstream impact of these shortages on unfulfilled production schedules and open customer sales orders.

4.2.2 System Requirements Gathering Techniques

The system and data requirements were collected by conducting an interactive site visit to the PPG facility. During this factory tour, the current operational workflows were observed, showing that while baseline transactions are stored in SAP, the tracking of aging stock and shortages relies entirely on manual, time-consuming Excel spreadsheets.

To gather the precise business logic needed to automate this process, a structured interview was held with the operations team during the visit. Questions were asked across five straightforward groups to extract the necessary data rules:

Group 1: Inventory Definitions & Financials

This group targeted the baseline business rules and financial parameters used by accounting and inventory teams. Extracting these exact thresholds is critical for programming the transformation rules in the data lakehouse, allowing the pipeline to automatically calculate Magna Carta provisions and categorize Recoverable Assets without manual variance.

- How do you determine excess inventory?
- What defines dead stock in your company?

- How long is without consumption considered “obsolete”?
- How do you handle expired materials?
- How do you calculate inventory provision / write-down?

Group 2: Data Sources & Tracking

This group focused on data architecture and landscape mapping. It identified the specific tables required for ingestion, located where the source data rests, and determined the operational update frequencies necessary to design the batch scheduling and pipeline orchestration intervals.

- What types of data are used for inventory analysis?
- Do you track: Inventory levels, Material consumption, Expiry dates, Lead time, Supplier performance, Sales orders?
- Where is the data stored? (SAP, Excel, database, etc.)
- How frequently is the data updated? (daily, monthly, etc.)

Group 3: Stockout Risks & Order Impact

This group aimed to uncover how inventory deficits propagate through commercial operations. The responses provided the logical relational join criteria needed to link upstream material shortages directly to downstream customer sales orders, establishing automated risk-tracing metrics.

- How do you identify stockout risk?
- Do you consider lead time vs stock level?
- How do shortages affect customer sales orders?
- Do you track which orders are impacted?

Group 4: Current Systems & Technology

This group evaluated the technical baseline of the organization. Identifying existing systems helped isolate technical friction points, confirm the absence of automated data pipelines, and justify the deployment of an integrated Azure Synapse and Power BI

architecture over their existing legacy routines.

- What system do you use? (SAP, ERP, others)
- Do you have dashboards or reporting tools?
- Is the process manual or automated?
- Do you currently use any data pipelines?

Group 5: Challenges & Future Solutions

This final group assessed stakeholder pain points and future-state user expectations. The explanations gathered here directly shaped the functional design of the Power BI user interface, ensuring that the delivered reporting views and interactive KPI filters address the actual visibility gaps experienced by staff.

- What are the biggest challenges in inventory management?
- Is it difficult to track excess stock?
- Do you face stockout or overstock issues?
- Is reporting time-consuming or inefficient?
- Would an automated system for RA / MC detection be useful?
- Would dashboards improve decision-making?

Conclusion from industry visit

In conclusion, the site visits and interview questions successfully gathered the needed information to complete this project.

4.3 System Requirements

This section details the technical specifications required to build and deploy the automated data from the Lakehouse pipeline for PPG. It is divided into functional requirements, which outline the specific operations, calculations, and behavioral rules the pipeline must execute, and non-functional requirements, which define the operational limits, performance benchmarks, and security standards of the system architecture.

4.3.1 Functional Requirements

Functional Requirements (FR) describe the essential actions, calculations, and behaviors the automated data engineering system must perform:

- **Raw Data Ingestion (Bronze Layer):**

The system must ingest six separate operational source CSV files (materials.csv, inventory_snapshot.csv, purchase_orders.csv, production_orders.csv, suppliers.csv, and sales_orders.csv) into a secure cloud storage data lake while preserving the original raw format.

- **Data Quality Guardrails**

The ingestion layer must automatically validate data types, isolate or replace null values, and strip out duplicate row records to protect downstream models from data corruption.

- **Recoverable Assets Calculation**

The system must calculate the 12-month trailing material consumption rate and flag any inventory stock volume that exceeds this threshold as a Recoverable Asset (RA).

- **Magna Carta Dead Stock Flagging**

The transformation pipeline must check consumption history and flag a material as Magna Carta (MC) Dead Stock if it registers zero operational consumption for 9 consecutive months or longer while remaining within its expiry date.

- **Magna Carta Expired Flagging**

The pipeline must evaluate active calendar timelines against raw lot parameters to flag materials as Magna Carta Expired the moment their designated lot expiry date passes.

- **Financial Provision Quantification**

The transformation engine must calculate a 100% financial write-down provision amount for all inventory items flagging as either Magna Carta Dead Stock or Magna Carta Expired.

- **Stockout Risk and Order Traceability**

The system must flag materials running critically low relative to supplier lead times, identify unfulfilled production orders, and trace the downstream impact to flag affected open customer sales orders and their associated clients.

- **Galaxy Schema Serving (Gold Layer)**

The system must organize the final processed attributes into a clean, relational galaxy schema structure inside a data warehouse, anchoring metrics around a central fact_inventory_status table.

- **Interactive Dashboard Analytics**

The reporting interface must display final calculations through an interactive Microsoft Power BI dashboard engineered to answer the seven specific business queries regarding inventory positions and risk metrics.

4.3.2 Non-Functional Requirements

Non-Functional Requirements (NFR) define the systemic quality attributes, performance standards, and operational limits of the data engineering environment:

- **Data Integrity & Formula Accuracy**

All mathematical logic transformations applied within the cloud pipeline must maintain absolute data integrity, ensuring financial write-down summaries match original source values.

- **Query Performance & Dashboard Latency**

The front-end business intelligence dashboards must be optimized to render visual tiles and respond to cross-filtering sliced queries within an acceptable operational limit

- **Scalability**

The backend data infrastructure must utilize scalable cloud compute resources capable of handling an increasing volume of transactional snapshot rows over time without systemic failure.

- **Maintainability & Modular Pipeline Design**

The pipeline workflows and transformation scripts developed within Microsoft Azure must follow structured modular design conventions, allowing future engineering teams to update business logic formulas easily.

- **Security & Access Control**

Access to the underlying data lake, data warehouse layers, and financial analytics reporting dashboards must be restricted to authenticated project stakeholders and authorized corporate personnel.

4.4 Current System Analysis

4.4.1 Current system and its Limitations

The current inventory system management and risk monitoring processes of PPG industries rely heavily on manual data processing and decentralized operating systems. Inventory-related information is generated by multiple business functions including procurement, warehouse management, production planning, supplier management and customer sales operations. These datasets are maintained separately and are not integrated into the centralized analysis platform.

To conduct inventory analysis and reporting, employees need to manually extract data from multiple operational sources, and integrate datasets provided in CSV format. Before generating a management report, the process involves combining materials inventory_snapshot, purchase_orders, production_orders, suppliers and sales_orders records.

There are several limitations found in the analysis of current systems. First of all, due to the lack of a centralized data platform, it is difficult to obtain a complete and accurate view of inventory performance. Data is distributed in multiple files and systems, resulting in an increase in the workload during data collection and preparation activities. Secondly, the identification of recoverable assets (RA), Magna Carta (MC) dead and MC expired materials largely depends on manual calculation and business interpretation which can be time consuming and prone to human errors.

In addition, due to lack of automatic monitoring function, it is impossible to actively detect inventory-related risks such as inventory shortages, excess inventory and affected customer orders. Top management often needs to review multiple reports to gain meaningful insights which lead to delays in responding to operational issues and inventory risks.

The analysis also showed that the current reporting process lacks real-time visibility of inventory status and financial reserves. With the increasing inventory, the existing manual methods are becoming more inefficient and difficult to maintain.

Based on these findings, it is clear that the current process needs a more integrated and automated solution that can integrate data, perform inventory risk analysis and provide actionable insights through centralized reporting and dashboard visualization.

4.4.2 Flow chart for the Current Process

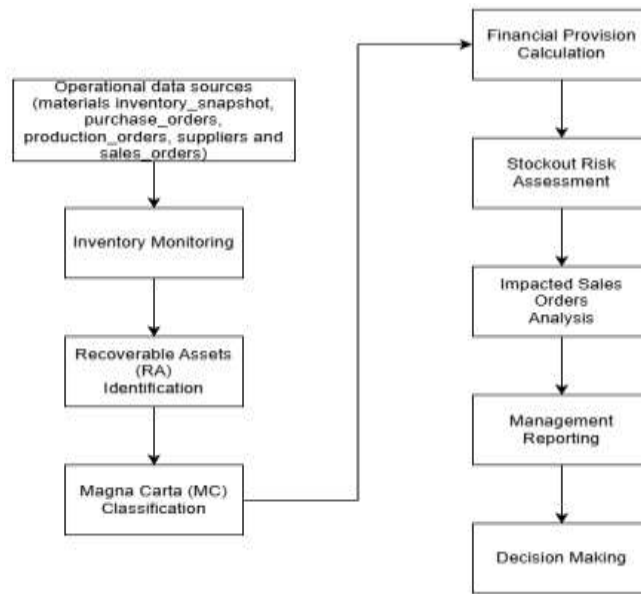


Figure 4.1 Current Process for Inventory Management System

Figure 4.1 illustrates the current inventory risk monitoring process implemented within the PPG industry. This process begins with operational data generated by various business functions, including inventory management, procurement, production planning, supplier management and customer sales operations.

The collected operational data is used to support inventory monitoring activities. Business users first identify Recoverable Assets (RA) which are excess inventory that may no longer be required for normal operation. Subsequently, Magna Carta (MC) classification was carried out to identify the materials classified as MC dead inventory and MC expired materials.

After the classification process, the financial provision is calculated to estimate the potential financial impact related to excess, obsolete or expired inventory. Then carry out an out-of-stock risk assessment to determine the materials that may lead to production interruption or inventory shortage.

Analysis continues to determine how inventory shortages may affect customer performance and business operations by evaluating affected sales orders. The results are then compiled into management reports and submitted to management for

operational planning and inventory risk management.

Although the current process can carry out inventory monitoring and risk assessment, it relies heavily on manual analysis and reporting activities. This leads to delayed risk identification, limited visibility of inventory performance, and reduced decision-making efficiency. Therefore, an automated data engineering solution is proposed to improve data integration, analysis ability and reporting effectiveness.

4.5 System Design

Based on the results of the current system analysis, a cloud-based data engineering solution is proposed to overcome the limitations of the existing inventory monitoring and reporting process. The proposed system automates data ingestion, conversion, storage and analysis of reporting activities while improving data quality, scalability and decision-making capabilities. By using the Microsoft Azure service and Medallion Architecture framework, the solution supports inventory risk analysis, financial supply assessment, out-of-stock risk monitoring and affected sales order analysis. The integration of these technologies provides a centralized platform for managing inventory-related data, generating business insights, and achieving more efficient and data-driven decision-making.

4.5.1 System Architecture

Figure 4.2 illustrates the proposed system architecture developed using the Medallion Architecture framework to support inventory risk analysis, reporting and decision-making. The architecture consists of several interconnected components, which are responsible for data storage, transformation, analysis, processing and visualization.

The process begins with operational datasets, including materials, inventory_snapshots, purchase_orders, production_orders, suppliers and sales_orders. These datasets are the main data sources for inventory risk analysis and are stored in Azure Data Lake Storage Gen2 (ADLS Gen2).

In the data lake environment, the medallion structure is realized through three logical layers which are bronze, silver and gold. The bronze layer is the initial data verification layer, in which the dataset ingested needs to be checked for data quality, including null value detection, duplicate record identification and data integrity verification. These verification processes ensure that the dataset meets the required quality standards before further processing.

The function of the silver layer is the business transformation layer. Azure Databricks applies inventory risk analysis and business rules to generate analytical indicators needed for decision-making. These processes include recoverable assets (RA) identification, Magna Carta (MC) classification, consumption calculation, out-of-stock risk detection and provision calculation. The transformed dataset generated in this layer provides reliable and analyzable information.

The gold layer is the analytical modeling layer where the data processed is organized into a dimensional model composed of facts and dimensional tables. The implementation of the galaxy schema improves query performance and supports multi-dimensional analysis for reporting and business intelligence purposes.

Azure Databricks is the main data processing and transformation platform in the proposed solution. It performs data validation, business rule implementation, analytical computing and data modeling activities across the medal architecture layer.

The processed gold layer dataset is then loaded into Azure Synapse Analytics, which acts as an enterprise analysis data warehouse. Azure Synapse Analytics stores selected datasets and supports efficient query, analysis processing and reporting activities.

Finally, Microsoft Power BI connects to Azure Synapse Analytics to provide interactive dashboards, visual analysis and management reports. Through these dashboards, stakeholders can monitor inventory performance, assess financial risks, identify out-of-stock threats, analyze affected sales orders, and support data-driven decision-making.

An end-to-end cloud-based data engineering environment for inventory risk management is produced by integration of these components. By automating data transformed, storage, analytical processing and reporting activities, the proposed architecture improves data quality, enhances analytical capabilities, and provides timely business insights for operational and strategic decision-making.

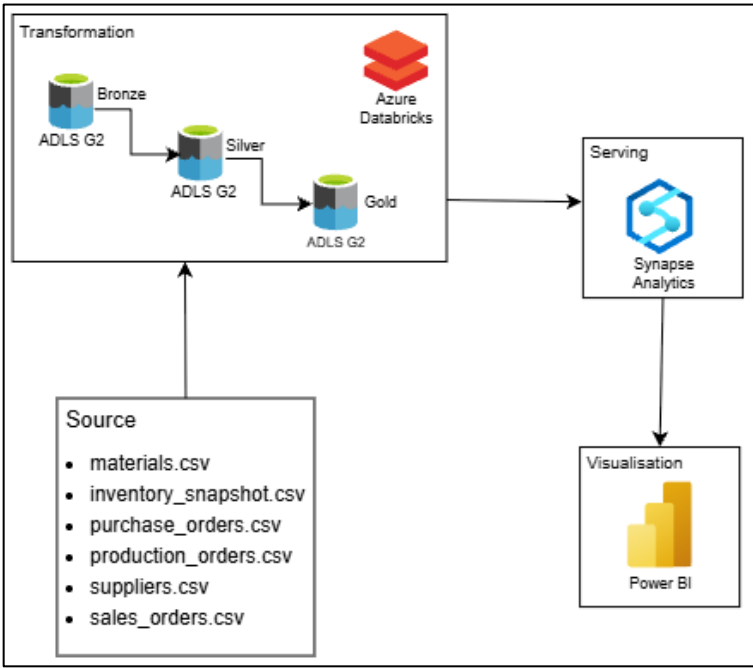


Figure 4.2 System Architecture

4.5.2 Explanation of System Components

The suggested solution utilizes a combination of Microsoft Azure services and business intelligence tools that are selected based on their ability to provide scalable data storage, processing, analysis and visualization requirements for inventory risk management.

Azure Data Lake Storage Gen.

Azure Data Lake Storage Gen2 (ADLS Gen2) was selected as a centralized storage platform because of its scalability, reliability and compatibility with the Medallion Architecture framework. The platform can store a large amount of structured data and handle the system data for bronze, silver and gold layers. ADLS Gen2 also provides

data traceability and supports efficient data access for downstream processing activities.

Azure Databricks

Azure Databricks was chosen as the primary data processing and transformation platform because it provides a scalable environment for data engineering and analytical processing. The platform supports data validation, transformation and implementing business rule within the Medallion Architecture layer. In this project, Azure Databricks was used to perform null value check, identifying duplicate record, identifying Recoverable Assets (RA), classifying Magna Carta (MC), consumption calculation, stockout risk assessment and provision calculation. These functions generate reliable datasets required for inventory risk analysis.

Azure Synapse Analysis

Azure Synapse Analytics was selected as an enterprise analysis data warehouse because it supports high-performance analytics workloads and dimensional modeling technology. The platform stores selected datasets generated from the gold layer and organized them into a galaxy schema consisting of fact and dimensional tables. This improves query efficiency, supports multi-dimensional analysis, and enhance the performance of reporting.

Microsoft Power BI

Microsoft Power BI was chosen as a visualization and reporting platform because of its ability to create interactive dashboards and analytical reports. The platform enables stakeholders to monitor inventory performance, assess financial risks, identify out-of-stock threats, and analyze affected sales orders through a centralized reporting interface. Its integration with Azure Synapse Analytics also promotes efficient access to analytical data sets and supports data-driven decision-making.

The combination of these technologies creates a scalable and integrated cloud-based data engineering ecosystem that can support inventory risk monitoring, analytical processing and business intelligence reporting requirements.

4.5.3 System Design

This section presents the system design of the PPG Recoverable Assets and Inventory Risk Analytics System through four UML diagrams and two Data Flow Diagrams (DFD). These diagrams collectively illustrate the functional interactions between system actors, the end-to-end pipeline workflow, the sequence of system communications, and the flow of data across all processing layers. Together they provide a comprehensive blueprint of how the system operates from raw data ingestion through to business intelligence reporting.

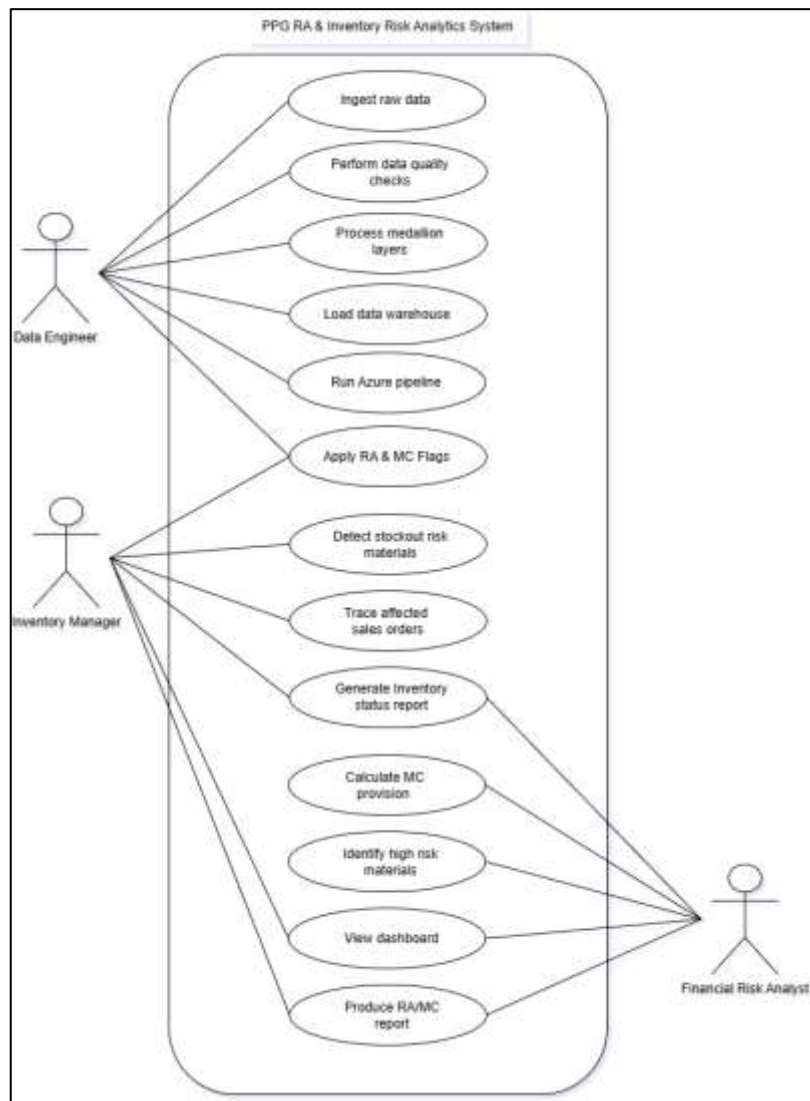


Figure 4.3 Use Case Diagram

Figure 4.3 presents the case diagram for the PPG RA and Inventory Risk Analytics System. The diagram identifies three primary actors and their respective interactions with the system within the defined system boundary.

The Data Engineer is responsible for all backend pipeline operations including ingesting raw data, performing data quality checks, processing medallion layers, loading the data warehouse, running the Azure pipeline and applying RA and MC classification flags. The Inventory Manager interacts with the system at the operational level, focusing on detecting stockout risk materials, tracing affected sales orders, generating the inventory status report and accessing the shared dashboard. The Financial Risk Analyst operates at the financial decision-making level, responsible for calculating MC provisions, identifying high-risk materials, viewing the Power BI dashboard and producing the final RA/MC analytics report.

The shared use cases Apply RA and MC Flags, View Dashboard and Produce RA/MC Report are connected to multiple actors, reflecting the collaborative nature of inventory risk management across technical and business roles.

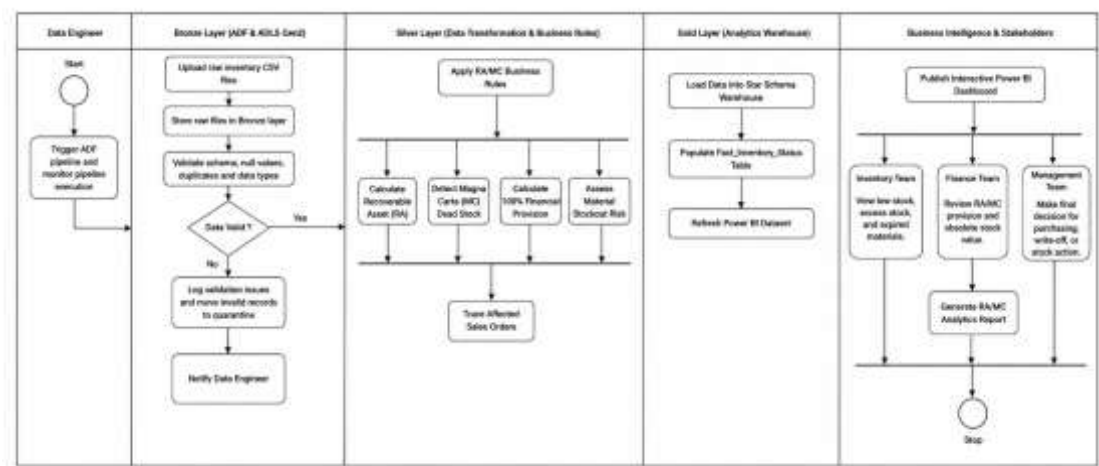


Figure 4.4 Activity Diagram

Figure 4.4 illustrates the activity diagram of the PPG RA and Inventory Risk Analytics

System using a swimlane structure divided into five lanes representing the Data Engineer, Bronze and Silver layers of the Azure Medallion Pipeline, the Gold layer Analytics Warehouse, and the Business Intelligence and Stakeholders layer.

The workflow begins when the Data Engineer uploads the raw inventory CSV files directly into the Bronze container of Azure Data Lake Storage Gen2, preserving their original format and structure. An Azure Databricks notebook then executes data quality checks on each ingested file, including schema validation, null detection, duplicate removal, and data type verification. If invalid records are found, they are flagged and isolated for review, and the Data Engineer is notified to address the issue before the pipeline proceeds. If the data passes all checks, the validated files are written to the Silver layer, where a second Databricks notebook applies the business logic in parallel: calculating Recoverable Assets based on the 12-month consumption rule, detecting Magna Carta Dead Stock based on the 9-month no-consumption rule, computing 100% financial provision amounts, and assessing stockout risks linked to open customer sales orders.

The processed Silver data is then exposed through Azure Synapse Serverless SQL Pool views, forming the Gold layer galaxy schema and populating the `fact_inventory_status` table. A Power BI dataset refresh is triggered, publishing the interactive dashboard. Business stakeholders are divided into three groups: the Inventory Team reviews low stock, excess, and expired materials; the Finance Team reviews RA/MC provisions and generates the analytics report; and the Management Team makes final decisions on purchasing, write-off, or stock action. The workflow concludes at the Stop node.

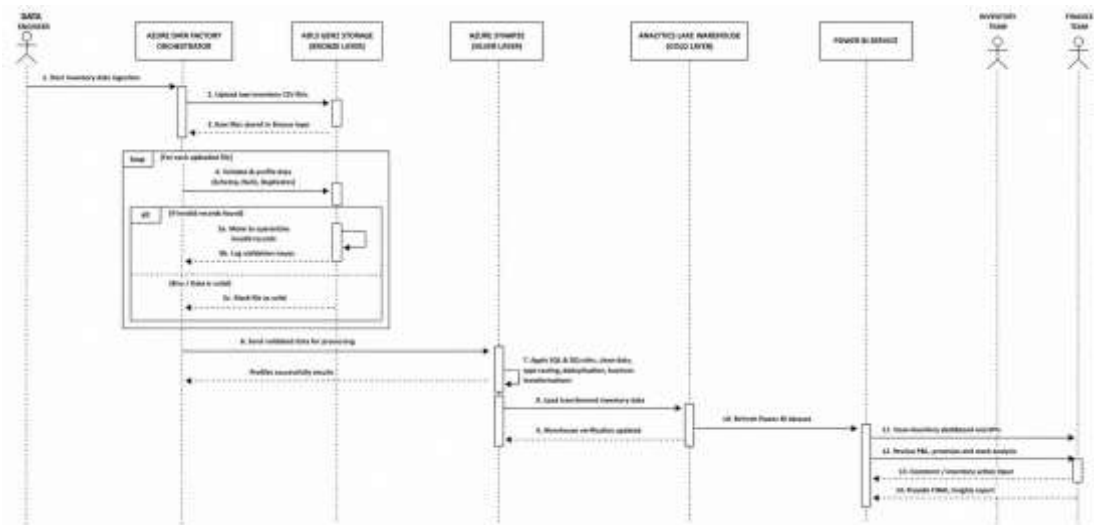


Figure 4.5 Sequence Diagram

Figure 4.5 presents the sequence diagram for the PPG RA and Inventory Risk Analytics System, showing the time-ordered interactions between the Data Engineer, ADLS Gen2 Bronze Layer, Azure Databricks Silver Layer, Azure Synapse Gold Layer, Power BI Service, Inventory Team and Finance Team.

The sequence begins with the Data Engineer uploading the raw inventory CSV files directly to the ADLS Gen2 Bronze Layer, where they are stored in their original format. A loop block then executes data validation and profiling for each uploaded file within an Azure Databricks notebook, checking schema, nulls and duplicates. Within the loop, an alternative block handles two scenarios: if invalid records are found they are flagged and the validation issues are logged; otherwise the file is marked as valid and written to the Bronze validated zone. The validated data is then passed to the Silver Layer, where a second Databricks notebook applies deduplication, foreign key joins and the RA/MC business logic, before writing the transformed output as parquet files to ADLS Gen2. The Gold Layer then exposes this Silver data through Azure Synapse Serverless SQL views, which query the parquet files directly and populate the galaxy schema, including the fact_inventory_status table. A Power BI dataset refresh is triggered, and the interactive dashboard is published. The Inventory Team accesses the dashboard to view inventory KPIs while the Finance Team reviews RA/MC provisions and stock

analysis. The sequence concludes with the generation of the final insights report delivered to both business teams.

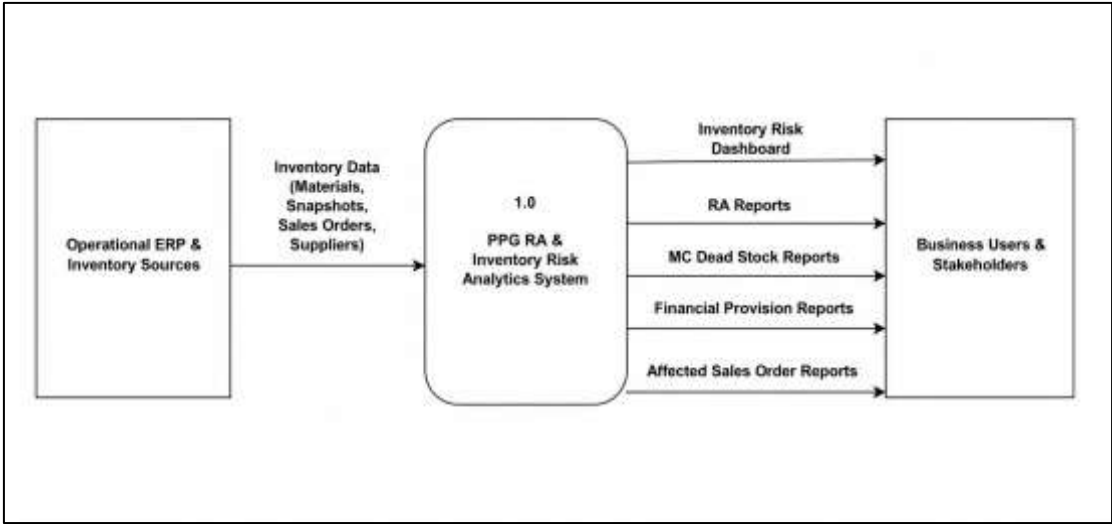


Figure 4.6 DFD Level 0 (Context Diagram)

Figure 4.6 presents the DFD Level 0 context diagram for the PPG RA and Inventory Risk Analytics System. This diagram provides a high-level overview of the system as a single process (1.0) showing the external entities that interact with it and the data flows between them.

The external entity on the left, Operational ERP and Inventory Sources, feeds inventory data comprising materials, snapshots, sales orders and suppliers into the central PPG RA and Inventory Risk Analytics System. On the right, the system produces five categories of output directed to Business Users and Stakeholders the Inventory Risk Dashboard, RA Reports, MC Dead Stock Reports, Financial Provision Reports and Affected Sales Order Reports. This context diagram establishes the system boundary and confirms that the system receives raw operational data as input and delivers structured analytical reports as output.

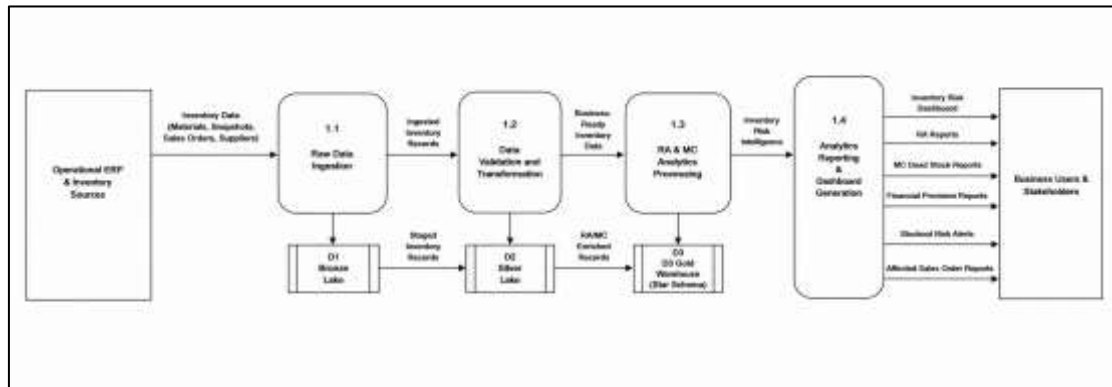


Figure 4.7 DFD Level 1

Figure 4.7 presents the DFD Level 1 diagram which decomposes the single system process from Level 0 into four sub-processes, each representing a distinct stage of the data pipeline.

Process 1.1 Raw Data Ingestion receives inventory data from the Operational ERP and Inventory Sources and stores ingested inventory records into data store D1 Bronze Lake. Process 1.2 Data Validation and Transformation retrieve staged inventory records from D1, applies validation and transformation rules, and produces business-ready inventory data stored in D2 Silver Lake. Process 1.3 RA and MC Analytics Processing retrieve enriched records from D2, applies the RA and MC business logic to generate inventory risk intelligence, and stores the results in D3 Gold Warehouse (Galaxy Schema). Finally, Process 1.4 Analytics Reporting and Dashboard Generation retrieves data from D3 and delivers five output streams to Business Users and Stakeholders Inventory Risk Dashboard, RA Reports, MC Dead Stock Reports, Financial Provision Reports, Stockout Risk Alerts and Affected Sales Order Reports.

This Level 1 DFD directly mirrors the Medallion Architecture design pattern adopted in this project, with bronze, silver and gold layers corresponding to processes 1.1, 1.2/1.3 and 1.4 respectively.

4.5.4 Database Design

This section presents the complete database design for the PPG Industries Inventory

Risk Management system. Three designs are documented which are the Entity-Relationship Diagram (ERD) representing the traditional relational model, a fully normalised schema conforming to Third Normal Form (3NF) and a Galaxy Schema for data warehousing and analytical reporting. Together, these artefacts address the system's core business objectives of identifying Recoverable Assets (RA) and Magna Carta (MC) provisions across PPG's raw material inventory.

Entity Relational Diagram (ERD)

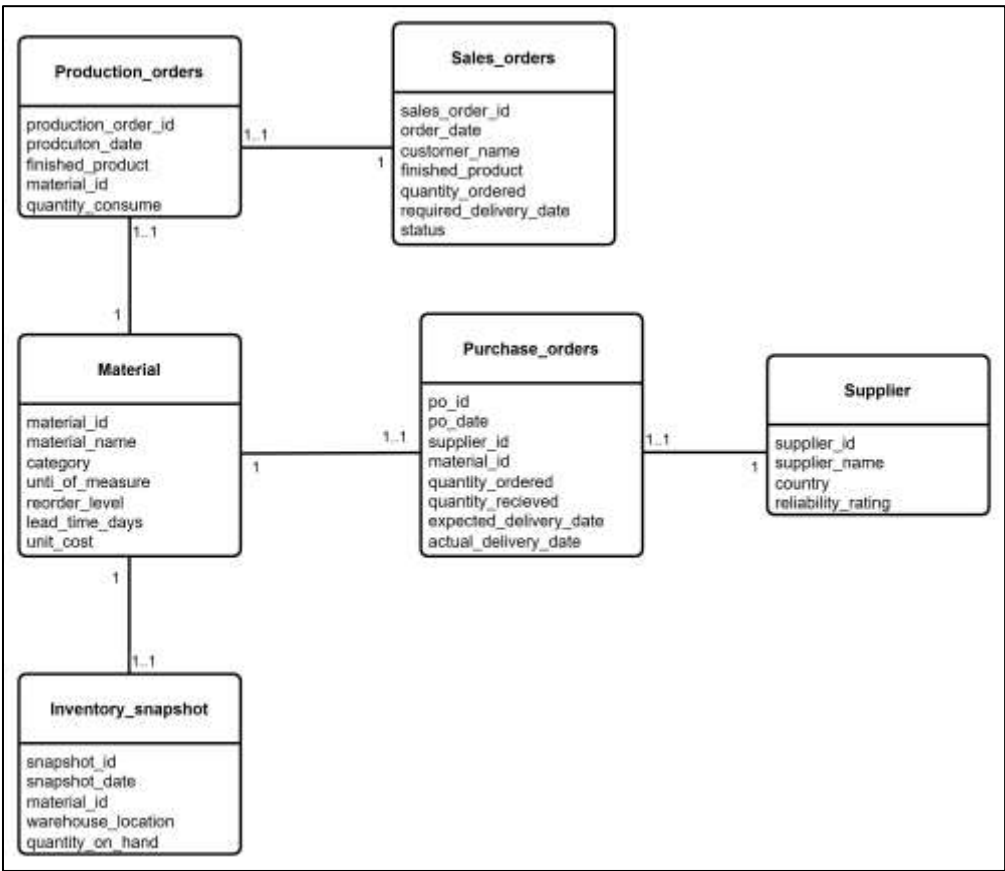


Figure 4.8 Entity Relational Diagram for traditional methodology

Figure 4.8 shows the six entities, and their key attributes are described below.

materials

The materials entity serves as the central reference for all raw material master data. It

is identified by `material_id` (primary key) and stores descriptive attributes including `material_name`, `category`, `unit_of_measure`, `reorder_level`, `lead_time_days`, and `unit_cost`. This entity is the hub of the model in which every inventory snapshot, production consumption record, and purchase order references a material.

inventory_snapshot

The `inventory_snapshot` entity captures the physical stock position for each material at a specific warehouse location at a point in time `snapshot_date`. Key attributes include `quantity_on_hand`. This entity is the primary source for computing RA and MC classifications: `quantity_on_hand` is compared against rolling 12-month consumption.

production_orders

The `production_orders` entity records each material consumption event during manufacturing. It links a material (via `material_id`) to a finished product and records the `quantity_consumed` and `production_date`. This entity is the source for computing 12-month rolling consumption per material which the denominator used in RA classification.

purchase_orders

The `purchase_orders` entity tracks each procurement transaction between PPG and its suppliers. It holds both `expected_delivery_date` and `actual_delivery_date` to enable supplier on-time delivery analysis. The `quantity_received` attribute may be null until goods are received, distinguishing open orders from fulfilled ones.

suppliers

The `suppliers` entity maintains vendor master data, including `supplier_name`, country of origin, and `reliability_rating`. The reliability rating is a decimal score used in procurement risk assessments and supplier performance reporting.

sales_orders

The `sales_orders` entity captures customer demand. Each record stores the `finished_product` ordered, `quantity_ordered`, `required_delivery_date`, and a status

indicator like ‘Open’ or ‘Delivered’. Sales orders indirectly drive production orders and consequently, raw material consumption patterns.

Entity Relationships and Cardinality

Table 4.1 summarises the five primary relationships in the ERD, their cardinality, and their business interpretation.

Table 4.1 ERD Relationship Cardinality Summary

Entity A	Cardinality	Entity B	Description
materials	1..* (one-to-many)	inventory_snapshot	One material can appear across multiple warehouse snapshot records over time.
materials	1..* (one-to-many)	production_orders	One material is consumed in many production runs across different finished products.
materials	1..* (one-to-many)	purchase_orders	One material can be procured through many purchase orders from different suppliers.
suppliers	1..* (one-to-many)	purchase_orders	One supplier may fulfill many purchase orders; supports multi-source procurement tracking.
sales_orders	1..* (one-to-many)	production_orders	One sales order triggers one or more production runs, linked via the finished product attribute.

3NF Normalized Table

The relational schema has been normalized into Third Normal Form or 3NF to remove data redundancy and ensure referential integrity. Resulting in the being total about eight tables of which six of them are the original sources and one new references table.

Table 1: material

Table 4.2 shows the materials that are the central reference for all raw material master

data. It has no foreign keys which is a pure lookup table referenced by all transaction and snapshot tables. The unit_cost column carries a CHECK constraint ensuring no negative values are stored, and material_name is declared UNIQUE to prevent duplicate material registrations.

Table 4.2 3NF schema of material

Column Name	Data Type	Key	Null?	Notes / Constraint
material_id	VARCHAR(20)	PK	NOT NULL	Business material code
material_name	VARCHAR(200)		NOT NULL	Apply UNIQUE to ensure no duplicate names
category	VARCHAR(100)		NOT NULL	e.g., Pigment, Solvent, Resin, Packaging
unit_of_measure	VARCHAR(20)		NOT NULL	e.g., KG, Litre, Unit
reorder_level	DECIMAL(12,3)			Minimum stock trigger quantity
lead_time_days	INT			Supplier lead time in days
unit_cost	DECIMAL(18,4)		NOT NULL	CHECK (unit_cost > 0)

Table 2: suppliers

Table 4.3 shows the supplier table that stored the vendor of master data that contain supplier id, supplier name with their country and its rating. The reliability_rating is constrained to using only 'Low', 'Medium' or 'High' scale.

Table 4.3 3NF schema of supplier

Column Name	Data Type	Key	Null?	Notes / Constraint
supplier_id	VARCHAR(20)	PK	NOT NULL	Business supplier code
supplier_name	VARCHAR(200)		NOT NULL	UNIQUE
country	VARCHAR(50)		NOT NULL	Country name
reliability_rating	DECIMAL(3,2)			CHECK ('Low', 'Medium', 'High')

Table 3: finished_products

Table 4.4 shows the finished_products that was introduced to eliminate the transitive dependency caused by a free-text finished_product attribute appearing in both sales_orders and production_orders. Extracting it into a reference table allows product-level attributes (product_line, business_unit) to be stored without duplication or anomaly risk.

Table 4.4 3NF schema of finished_products

Column Name	Data Type	Key	Null?	Notes / Constraint
product_id	VARCHAR(20)	PK	NOT NULL	Business product code
product_name	VARCHAR(200)		NOT NULL	UNIQUE to ensure no duplicate names

Table 4: inventory_snapshot

Table 4.5 shows the inventory_snapshot table that records the physical stock position for each material at a given warehouse for a given month.

Table 4.5 3NF schema of inventory_snapshot

Column Name	Data Type	Key	Null?	Notes / Constraint
material_id	VARCHAR(20)	FK	NOT NULL	materials(material_id)
warehouse_location	VARCHAR(4)			Location of warehouses
snapshot_date	DATE		NOT NULL	Date of the snapshot
quantity_on_hand	INT		NOT NULL	CHECK (quantity_on_hand > 0)

Table 5: production_orders

Table 4.6 shows the production_orders table that records individual material consumption events. Each row links one material to one finished product for a specific production run, recording the quantity consumed.

Table 4.6 3NF schema of production_orders

Column Name	Data Type	Key	Null?	Notes / Constraint
-------------	-----------	-----	-------	--------------------

production_order_id	VARCHAR(20)	PK	NOT NULL	Business production order code
production_date	DATE		NOT NULL	Date of production run
product_id	VARCHAR(20)	FK	NOT NULL	finished_products(product_id)
material_id	VARCHAR(20)	FK	NOT NULL	materials(material_id)
quantity_consumed	INT		NOT NULL	CHECK (quantity_consumed > 0)

Table 6: purchase_orders

Table 4.7 shows the purchase_orders table that tracks all procurement transactions. The quantity_received and actual_delivery_date columns are nullable to accommodate open purchase orders that have been raised but not yet fulfilled. The difference between expected_delivery_date and actual_delivery_date provides a delivery variance metric used in supplier performance assessment.

Table 4.7 3NF schema of purchase_orders

Column Name	Data Type	Key	Null?	Notes / Constraint
po_id	VARCHAR(20)	PK	NOT NULL	Business PO number
po_date	DATE		NOT NULL	Date PO was raised
supplier_id	VARCHAR(20)	FK	NOT NULL	suppliers(supplier_id)
material_id	VARCHAR(20)	FK	NOT NULL	materials(material_id)
quantity_ordered	DECIMAL(14,3)		NOT NULL	CHECK (quantity_ordered > 0)
quantity_received	DECIMAL(14,3)			NULL until goods received

expected_delivery_date	DATE		NOT NULL	Contracted delivery date
actual_delivery_date	DATE			NULL for open/pending orders

Table 7: sales_orders

Table 4.8 shows the sales_orders table that records customer demand. The free-text finished_product attribute from the source has been replaced with a product_id foreign key.

Table 4.8 3NF schema of sales_orders

Column Name	Data Type	Key	Null?	Notes / Constraint
sales_order_id	VARCHAR(20)	PK	NOT NULL	Business sales order number
order_date	DATE		NOT NULL	Date order was placed
customer_name	VARCHAR(200)		NOT NULL	Customer entity name
product_id	VARCHAR(20)	FK	NOT NULL	finished_products(product_id)
quantity_ordered	DECIMAL(14,3)		NOT NULL	CHECK (quantity_ordered > 0)
required_delivery_date	DATE			Customer-requested delivery date
status	VARCHAR(30)		NOT	Open Delivered

			NUL L	
--	--	--	----------	--

Multidimensional Diagram

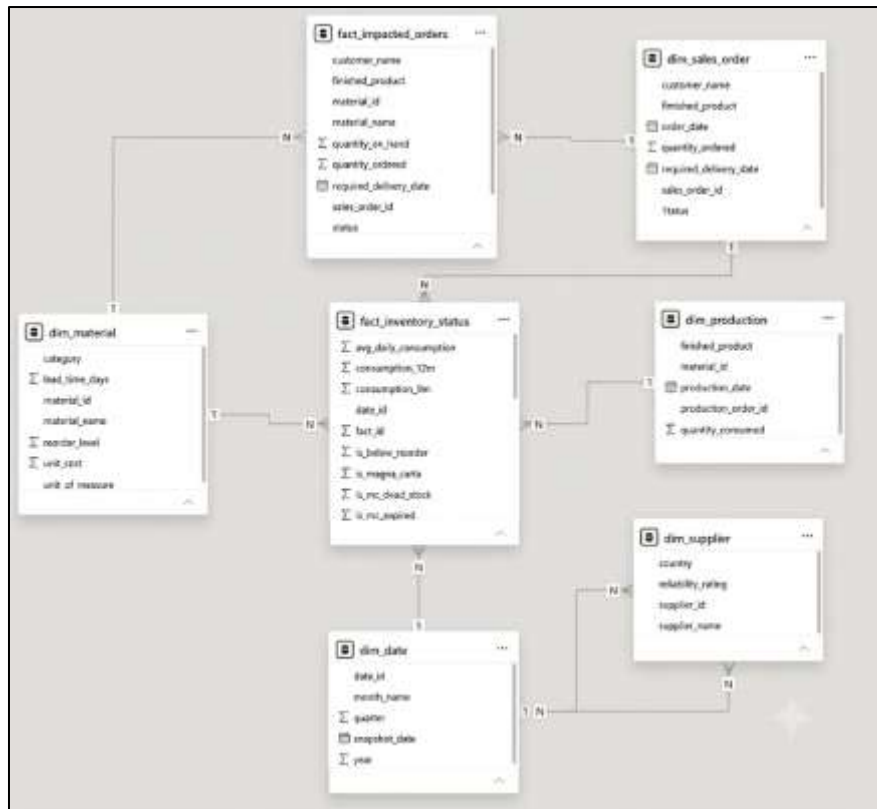


Figure 4.9 Galaxy Schema Diagram for Analytical

Figure 4.9 shows the model is structured as a galaxy schema centred on fact_inventory_status and fact_impacted_orders. The following Table 4.9 shows the description for both fact table and dimension table.

Table 4.9 Galaxy Schema Description

Table Name	Type	Purpose
fact_inventory_status	Fact	Periodic inventory snapshot with consumption metrics and status flags

dim_date	Dimension	Calendar date attributes for time-based analysis
dim_material	Dimension	Material master data including cost, UoM, reorder level, lead time day and etc.
dim_sales_order	Dimension	Customer sales order descriptors and delivery deadlines
dim_production	Dimension	Production order records linking output products to materials consumed
dim_supplier	Dimension	Supplier reference data for procurement and reliability analysis
fact_impacted_orders	Fact	Derived table identifying orders at risk due to inventory shortages

Table Description

1. fact_inventory_status

Table 4.10 is the central fact table of the data model. It records a periodic (daily or weekly) snapshot of each material's inventory health at a given point in time. Each row represents one material observed on one date, capturing key consumption metrics and derived flags used to identify critical stock situations.

Table 4.10 fact_inventory_status description

Column Name	Key Type	Data Type	Description
fact_id	PK	Varchar	Surrogate primary key. Uniquely identifies each inventory snapshot record.
date_id	FK	Varchar	Foreign key to dim_date. Identifies the snapshot date for this record.
material_id	FK	Varchar	Foreign key to dim_material. Identifies the material being tracked.
avg_daily_consumption		Decimal	Average number of units consumed per day, computed from historical

			usage data.
consumption_9m		Integer	Total quantity of this material consumed over the past 9 months.
consumption_12m		Integer	Total quantity of this material consumed over the past 12 months.
is_below_reorder		Boolean (0/1)	Flag set to 1 when current stock on hand falls below the reorder level defined in dim_material.
is_magna_carta		Boolean (0/1)	Flag set to 1 when the material qualifies as a Magna Carta item (high-criticality, slow-moving, or strategically important).
is_mc_dead_stock		Boolean (0/1)	Flag set to 1 when the material is classified as dead stock under the Magna Carta policy — no consumption recorded within a threshold period.
is_mc_expired		Boolean (0/1)	Flag set to 1 when a Magna Carta material has exceeded its shelf-life or policy expiry date.

2. dim_date

Table 4.11 is a standard date dimension that provides calendar attributes for each inventory snapshot date. It enables time-based filtering, grouping, and trending across reports like for example, comparing inventory health month-over-month or quarter-over-quarter.

Table 4.11 dim_date description

Column Name	Key Type	Data Type	Description
date_id	PK	Varchar	Surrogate primary key. Uniquely identifies each calendar date in the dimension.

snapshot_date		Date	The actual calendar date this row represents.
month_name		Varchar	Full name of the month. Used for labelling in reports and charts.
quarter		Integer	Quarter number (1–4) derived from the snapshot date.
year		Integer	Four-digit calendar year (e.g., 2025).

3. dim material

Table 4.12 is the core descriptive dimension for all materials tracked in the inventory system. It stores master data attributes for each material including classification, cost, and replenishment parameters. This dimension is the most widely referenced in the model, connecting to the fact table, production records, and impacted order analysis.

Table 4.12 dim_material description

Column Name	Key Type	Data Type	Description
material_id	PK	Varchar	Surrogate primary key. Uniquely identifies each material in the system.
material_name		Varchar	Full descriptive name of the material.
category		Varchar	Classification category of the material.
unit_of_measure		Varchar	Unit in which the material is counted or weighed.
lead_time_days		Integer	Number of days required to replenish this material from a supplier after placing an order.
reorder_level		Integer	Minimum stock threshold that triggers a reorder.
unit_cost		Decimal	Cost per unit of this material, used for inventory valuation and financial reporting.

4. dim sales order

Table 4.13 is the supplier dimension stores vendor attributes used in procurement

performance analysis.

Table 4.13 dim_sales_order description

Column Name	Key Type	Data Type	Description
sales_order_id	PK	Varchar	Surrogate or natural primary key. Uniquely identifies each customer sales order.
customer_name		Varchar	Name of the customer who placed the order.
finished_product		Varchar	The finished product requested by the customer in this order.
order_date		Date	The date on which the sales order was placed.
required_delivery_date		Date	The deadline by which the customer expects the order to be delivered.
quantity_ordered		Integer	Total quantity of the finished product ordered by the customer.
status		Varchar	Current status of the order (e.g., Open, Delivered).

5. dim_production

Table 4.14 is to capture production order records that describe which materials were consumed and in what quantities during manufacturing runs. This dimension links production activity back to the material dimension, enabling analysis of how production consumption drives inventory depletion.

Table 4.14 dim_production description

Column Name	Key Type	Data Type	Description
production_order_id	PK	Varchar	Primary key. Uniquely identifies each production order.
material_id	FK	Varchar	Foreign key to dim_material. Identifies which material is consumed

			in this production run.
finished_product		Varchar	The finished product output of the production order.
production_date		Date	The date on which the production order was executed or scheduled.
quantity_consumed		Integer	The quantity of the material consumed during this production run.

6. dim_supplier

Table 4.15 is to store reference information about suppliers who provide materials to the organisation. While not directly linked to the fact table via a visible FK in the current diagram, this dimension provides contextual supplier attributes that can be joined to dim_material or fact_inventory_status for procurement and sourcing analysis.

Table 4.15 dim_supplier description

Column Name	Key Type	Data Type	Description
supplier_id	PK	Varchar	Primary key. Uniquely identifies each supplier.
supplier_name		Varchar	Full trading name of the supplier.
country		Varchar	Country where the supplier is based. Used to assess supply chain risk by geography.
reliability_rating		Varchar	A score or classification indicating the supplier's historical reliability in terms of on-time delivery and quality.

7. fact_impacted_order

Table 4.16 serves as a secondary central fact table within the architecture, establishing a Galaxy Schema design. Its primary purpose is to identify sales orders at risk of fulfillment failure due to material shortages. To optimize reporting performance, this table intentionally denormalizes select attributes from dim_material and dim_sales_order, making it a critical component for the system's demand-supply gap

analysis.

Table 4.16 fact_impacted_order description

Column Name	Key Type	Data Type	Description
sales_order_id	FK	Varchar	Foreign key to dim_sales_order. Identifies the customer order affected by the inventory shortage.
material_id	FK	Integer	Foreign key to dim_material. Identifies the material that is insufficient to fulfil the order.
customer_name		Varchar	Denormalised customer name. Included for reporting convenience.
finished_product		Varchar	Denormalised finished product name. Indicates which end product cannot be fulfilled.
material_name		Varchar	Denormalised material name (sourced from dim_material). Indicates which material is the bottleneck.
required_delivery_date		Date	The customer's expected delivery deadline, carried over from dim_sales_order.
quantity_on_hand		Integer	The current available stock quantity for this material at the time of the snapshot.
quantity_ordered		Integer	The total quantity of material required to fulfil the impacted sales order.
status		Varchar	Impact status of the order.

Relationship Summary

Table 4.17 below shows the summarises all foreign key relationships defined in the data model.

Table 4.17 summarize relationship of galaxy schema

Source Table	Source Column	Target Table	Target Column	Cardinality
fact_inventory_status	date_id	dim_date	date_id	Many-to-One
fact_inventory_status	material_id	dim_material	material_id	Many-to-One
dim_production	material_id	dim_material	material_id	Many-to-One
fact_impacted_orders	sales_order_id	dim_sales_order	sales_order_id	Many-to-One
fact_impacted_orders	material_id	dim_material	material_id	Many-to-One

4.5.5 Interface Design

The interface design of the PPG Recoverable Assets and Inventory Risk Management system will be presented in a Power BI dashboard, which shows the analytics reporting and answers all seven business questions.

Dashboards

Inventory Overview: It has KPI cards which provide a summary of the major classification counts across the entire material portfolio, providing management with an at-a-glance view of the overall inventory risk landscape. A clustered bar chart is provided to compare stock levels to reorder levels for all materials, and a detail table provides information about all materials tracked with key inventory details.

RA and MC Analysis: A clustered bar chart compares quantity on hand to 12-months consumption per material which enables stakeholders to easily spot materials with excess stock against materials with consumption. A donut chart displays the distribution of amounts of money allocated towards various materials in the context of

total MC funding. Two detail tables support this, one for MC Dead Stock - with stock and provision figures, and another for MC Expired - with stock and provision figures. A KPI card will show the total MC financial provision amount, as a single headline value.

Stockout Risk: is the spotlight on materials which have dropped to less than their safety stock. A clustered bar chart is provided which compares the current stock on hand with the safety stock required per material and a detail table is used to list all the materials at risk and their corresponding stock and consumption.

Impacted Sales Orders: brings down the downstream commercial impact of the stockout-risk material. There are two KPI cards showing the number of affected orders and the number of affected customers. A pie chart displays the distribution of impacted sales orders by customer and a detail table displays each impacted open sales order with its order information, material information, quantity ordered and delivery date.

4.6 Summary

Chapter 4 focused on the analysis and design of the Recoverable Assets (RA) and Inventory Risk Management (IRM) system for PPG Industries. The chapter began by examining the company's inventory management challenges, particularly inventory obsolescence, dead stock, expired materials, and stockout risks. Functional and non-functional requirements were identified to ensure the system could accurately detect inventory risks, calculate financial provisions, and support decision-making.

A cloud-based solution based on the Medallion Architecture was designed using Microsoft Azure. The proposed architecture consists of Bronze, Silver, and Gold layers to support data ingestion, transformation, and analytics-ready storage. Several system design models were developed, including Use Case Diagrams, Activity Diagrams, Sequence Diagrams, DFD Level 0 and Level 1, Entity Relationship Diagrams (ERD), Third Normal Form (3NF) schemas, and Galaxy Schema models. These designs

provide a complete blueprint for the system by illustrating user interactions, data flow, processing logic, and database structure.

Overall, the chapter established the technical foundation of the system and demonstrated how the proposed architecture can automate inventory risk detection while supporting efficient analytical reporting and business intelligence

CHAPTER 5

IMPLEMENTATION AND TESTING

5.1 Introduction

This chapter introduces the implementation and testing of the proposed cloud-based inventory analysis system developed for PPG industries. The focus of implementation stage is to use the Microsoft Azure service and Medallion Architecture framework to transform system design into functional solutions. The system integrates multiple operational datasets, processes them through different data layers and generates meaningful insights through interactive dashboards.

In addition, this chapter demonstrates the development of the main function of the system, including data ingestion, transformation, storage and visualization. The user interface and system output are also shown to illustrate the functionality and effectiveness of the proposed solution. In order to ensure that the system achieves its expected goals, various test activities are carried out, including black box tests, white box tests and user tests to evaluate the functionality, reliability and availability of the system. Discuss the results of these tests to verify whether the developed system is running as expected and meet the project requirements.

5.2 System development

The system developed in this project is an end-to-end data pipeline designed to automate the identification of expiration-risk and disposal-ready inventory for PPG Industries. The pipeline processes raw inventory data to detect Recoverable Assets (RA) and Magna Carta (MC) materials which means including dead stock and expired lots that previously required manual review. The development was carried out entirely on Microsoft Azure, leveraging a layered Medallion Architecture to progressively transform raw source data into business-ready insights.

The pipeline was built using the following Table 5.1 are tools and services within the Microsoft Azure ecosystem.

Table 5.1 Tools and services used for end-to-end pipeline

Component	Tool / Service
Cloud Platform	Microsoft Azure
Data Storage	Azure Data Lake Storage Gen2 (ADLS Gen2)
Data Ingestion & Transformation	Direct upload to ADLS Gen2 (ADLS Gen2)
Notebook-based Processing	Azure Databricks (PySpark)
Data Warehousing	Azure Synapse Analytics
Visualization	Microsoft Power BI
Source Data Format	CSV files (6 datasets)

The system was developed entirely on Microsoft Azure as the cloud platform, utilizing Azure Data Lake Storage Gen2 (ADLS Gen2) for data storage across all pipeline layers. The six source CSV files were uploaded directly into the Bronze container of ADLS Gen2, preserving their original format. Azure Databricks with PySpark was then used to perform data quality checks, cleaning, and the core business logic transformations as the data moved from the Bronze layer to the Silver layer. The transformed Silver layer output was written back to ADLS Gen2 in parquet format, where it was exposed for querying through Azure Synapse Analytics Serverless SQL Pool, forming the Gold layer galaxy schema. Finally, Microsoft Power BI was used as the visualization layer, connecting to the Synapse SQL endpoint to present insights to stakeholders.

This system was also developed using the Medallion Architecture pattern, a layered data engineering approach that organizes data into three progressive quality zones, namely Bronze, Silver, and Gold. This approach ensures that raw data is preserved, cleaned incrementally, and only transformed into business logic at the appropriate stage, maintaining both data integrity and auditability throughout the pipeline.

The three layers were orchestrated using Azure Databricks notebooks, which managed the sequential execution of data quality checks, cleansing, and business logic transformations. Data flowed from the source CSV files through ADLS Gen2 at each layer boundary, with Databricks notebooks handling the heavy transformation logic in the Silver layer, and Azure Synapse Serverless SQL Pool serving as the final query engine for reporting, reading directly from the Gold layer parquet files without a separate data loading step.

5.3 System Main Functions

The system is structured across four modules following the Medallion Architecture layers. All the four modules are raw data ingestion in Bronze Layer, data transformation in Silver Layer, RA and MC Analytics Processing in Gold Layer and Dashboard Generation.

5.3.1 Module 1: Raw Data Ingestion (Bronze Layer)

In this module, all six raw source datasets are loaded into the Databricks environment to perform data validation. The first cell as shown in Figure 5.1 authenticates the Databricks session to the ADLS Gen2 storage account using a direct access key. This grants the notebook in Databricks read and write access. In the second cell, the six source datasets are loaded from ADLS Gen2 directory to Spark DataFrames.



```
# Set up direct access credential
spark.conf.set(
    "fs.azure.account.key.storageppg.dfs.core.windows.net",
    "hfG3I/aTOIGLZguaikuRPFoSuzSVHU+XNKX0ODCWYkMmX4ahG7w2gj0I3JrTh3V0hKI8UxbAlw2+ASTPsDEzA=="
)

print("Storage account connected ✅")

Storage account connected ✅

from pyspark.sql import functions as F

materials      = spark.read.option("header", True).csv("abfss://bronze@storageppg.dfs.core.windows.net/ingested/materials.c
inventory      = spark.read.option("header", True).csv("abfss://bronze@storageppg.dfs.core.windows.net/ingested/inventory_s
purchase_orders = spark.read.option("header", True).csv("abfss://bronze@storageppg.dfs.core.windows.net/ingested/purchase_or
production_orders = spark.read.option("header", True).csv("abfss://bronze@storageppg.dfs.core.windows.net/ingested/production_
suppliers      = spark.read.option("header", True).csv("abfss://bronze@storageppg.dfs.core.windows.net/ingested/suppliers.c
sales_orders   = spark.read.option("header", True).csv("abfss://bronze@storageppg.dfs.core.windows.net/ingested/sales_order

print("All 6 files loaded ✅")

All 6 files loaded ✅
```

Figure 5.1 Connecting to ADLS Gen2 and Loading Raw CSV Files

A reusable `quality_check()` function in Figure 5.2 is defined to validate each dataset. The function accepts a `DataFrame`, a dataset name, a list of key columns to check for nulls, optional date columns to cast and validate, and optional numeric columns to cast and validate. For each dataset, it reports the total row count, null counts per key column, duplicate row count, invalid date counts, and invalid numeric counts, then returns a deduplicated, type-cast `DataFrame`.

```

def quality_check(df, name, key_cols, date_cols=[], numeric_cols=[]):
    print(f"\n===== {name} =====")
    total = df.count()
    print(f"Total rows: {total}")

    # Null check on key columns
    for col in key_cols:
        null_count = df.filter(F.col(col).isNull() | (F.col(col) == "")).count()
        print(f"  Nulls in '{col}': {null_count}")

    # Duplicate check
    dup_count = total - df.dropDuplicates().count()
    print(f"  Duplicate rows: {dup_count}")

    # Cast date columns
    for col in date_cols:
        df = df.withColumn(col, F.to_date(F.col(col)))
        invalid = df.filter(F.col(col).isNull()).count()
        print(f"  Invalid dates in '{col}': {invalid}")

    # Cast numeric columns
    for col in numeric_cols:
        df = df.withColumn(col, F.col(col).cast("double"))
        invalid = df.filter(F.col(col).isNull()).count()
        print(f"  Invalid numerics in '{col}': {invalid}")

    # Drop duplicates
    df = df.dropDuplicates()
    print(f"  Rows after dedup: {df.count()}")

    return df

```

Figure 5.2 Quality Check Function

Table 5.2 below summarizes the quality check results for all six datasets.

Table 5.2 Quality Check Result

Dataset	Total Rows	Key Null Issues	Duplicates	Notable Findings
materials	20	0	0	All pass
inventory_snapshot	240	0	0	96 null lot_expiry_date (expected — not all materials have expiry dates)

purchase_orders	40	0	0	1 null actual_delivery_date (open order, not yet delivered)
production_orders	336	0	0	All pass
suppliers	10	0	0	All pass
sales_orders	48	0	0	All pass

All six cleaned DataFrames are written back to ADLS Gen2 under the bronze/validated/ path as CSV files with headers. The overwrite mode ensures that re-runs of the notebook do not accumulate stale data in the output directory.

```
base_out = "abfss://bronze@storageppg.dfs.core.windows.net/validated"

materials_clean.write.mode("overwrite").option("header", True).csv(f"{base_out}/materials")
inventory_clean.write.mode("overwrite").option("header", True).csv(f"{base_out}/inventory_snapshot")
purchase_clean.write.mode("overwrite").option("header", True).csv(f"{base_out}/purchase_orders")
production_clean.write.mode("overwrite").option("header", True).csv(f"{base_out}/production_orders")
suppliers_clean.write.mode("overwrite").option("header", True).csv(f"{base_out}/suppliers")
sales_clean.write.mode("overwrite").option("header", True).csv(f"{base_out}/sales_orders")

print("All validated files written to bronze/validated ✅")
```

All validated files written to bronze/validated ✅

Figure 5.3 Load Validated Bronze Layer

A final verification loop uses `dbutils.fs.ls()` to confirm that each validated output folder exists and contains files, confirming successful write completion before Module 2 begins.

```
for folder in ["materials", "inventory_snapshot", "purchase_orders",
               "production_orders", "suppliers", "sales_orders"]:
    files = dbutils.fs.ls(f"abfss://bronze@storageppg.dfs.core.windows.net/validated/{folder}")
    print(f"{folder}: {len(files)} file(s) ✅")
```

materials: 4 file(s) ✅
inventory_snapshot: 4 file(s) ✅
purchase_orders: 4 file(s) ✅
production_orders: 4 file(s) ✅
suppliers: 4 file(s) ✅
sales_orders: 4 file(s) ✅

Figure 5.4 Verify Successful Load

All six folders returned 4 files each, confirming that all validated datasets were successfully written to the Bronze validated zone and are ready for Silver layer processing.

5.3.2 Module 2: Data Transformation (Silver Layer)

Module 2 implements process Data Transformation to the previously validated data in the Bronze Layer. This is a preprocessing step that is necessary so that the data is cleaned and can be used for analysis in the next step.

Firstly, as shown in Figure 5.5 all six validated datasets are read from the bronze container. Type casting is applied after loading the data to ensure all downstream calculations operate on correct type columns rather than the default string types produced by CSV inference.

```
spark.conf.set(
    "fs.azure.account.key.storageppg.dfs.core.windows.net",
    "hf6G3I/aTOIGLZguaikuRPFoSuZ5VHU+XNKX00DCWXYkMmX4ahG7w2gj0I3JrTh3V0hKI8UxbA1w2+AStPsDEzA=="
)

from pyspark.sql import functions as F
from pyspark.sql.window import Window

base = "abfss://bronze@storageppg.dfs.core.windows.net/validated"

materials      = spark.read.option("header", True).csv(f"{base}/materials")
inventory      = spark.read.option("header", True).csv(f"{base}/inventory_snapshot")
purchase_orders = spark.read.option("header", True).csv(f"{base}/purchase_orders")
production_orders = spark.read.option("header", True).csv(f"{base}/production_orders")
suppliers      = spark.read.option("header", True).csv(f"{base}/suppliers")
sales_orders   = spark.read.option("header", True).csv(f"{base}/sales_orders")

# Cast correct types
materials = materials.withColumn("unit_cost", F.col("unit_cost").cast("double")) \
    .withColumn("reorder_level", F.col("reorder_level").cast("double")) \
    .withColumn("lead_time_days", F.col("lead_time_days").cast("double"))

inventory = inventory.withColumn("quantity_on_hand", F.col("quantity_on_hand").cast("double")) \
    .withColumn("snapshot_date", F.to_date("snapshot_date")) \
    .withColumn("lot_expiry_date", F.to_date("lot_expiry_date"))

production_orders = production_orders.withColumn("quantity_consumed", F.col("quantity_consumed").cast("double")) \
    .withColumn("production_date", F.to_date("production_date"))

sales_orders = sales_orders.withColumn("quantity_ordered", F.col("quantity_ordered").cast("double")) \
    .withColumn("order_date", F.to_date("order_date")) \
    .withColumn("required_delivery_date", F.to_date("required_delivery_date"))

print("All files loaded and typed ✅")
```

All files loaded and typed ✅

Figure 5.5 Loading Validated Bronze Data and Type Casting

If six folders returned 4 files each, confirming that all validated datasets were successfully written to the Bronze validated zone and are ready for Silver layer processing.

Figure 5.6 calculates the total consumption in the latest 12 months. The most recent

snapshot date in the inventory dataset will be used as the reference point. A 12-month lookback window is calculated by subtracting 12 months from this reference date using `add_months()`. The total consumption_12m value for each material is calculated as the sum of all production order records in this time frame and used to determine the RA classification rule.

```
# Use latest snapshot date as reference point
latest_date = inventory.agg(F.max("snapshot_date")).collect()[0][0]
print(f"Reference date (latest snapshot): {latest_date}")

# 12-month window = Last 365 days from latest production date
cutoff_date = F.add_months(F.lit(latest_date), -12)

consumption_12m = production_orders.filter(
    F.col("production_date") >= cutoff_date
).groupBy("material_id").agg(
    F.sum("quantity_consumed").alias("consumption_12m")
)

print("12-month consumption calculated ✅")
consumption_12m.show()
```

Figure 5.6 12-Month Rolling Consumption per Material

The cell in Figure 5.7 is to get the most recent snapshot for each material. Since the inventory dataset contains monthly snapshots for each material, a window function partitioned by `material_id` and ordered by descending `snapshot_date` is applied to select only the most recent row per material, ensuring that all flag computations reflect the current inventory state.

```
# Get the most recent snapshot for each material
window_latest = Window.partitionBy("material_id").orderBy(F.desc("snapshot_date"))

latest_inventory = inventory.withColumn("rn", F.row_number().over(window_latest)) \
    .filter(F.col("rn") == 1) \
    .drop("rn")

print("Latest inventory snapshot per material ✅")
latest_inventory.show()
```

Figure 5.7 12-Month Rolling Consumption per Material

The latest inventory snapshot is joined to the 12-month consumption figures and the

materials dimension. If quantity_on_hand (physical inventory) is greater than consumption_12m (12-month average consumption), it is recorded as a Recoverable Asset by setting is_recoverable_asset = 1 in the material. Values for materials that weren't produced in the window (such as 0) are generated by fillna.

```
# Join inventory with 12-month consumption
inv_with_consumption = latest_inventory.join(consumption_12m, "material_id", "left") \
    .join(materials.select("material_id", "material_name", "category", "unit_cost", "reorder_level", "lead_time_days"),
          "material_id", "left") \
    .fillna(0, subset=["consumption_12m"])

# Flag: stock > 12-month consumption = Recoverable Asset
inv_with_consumption = inv_with_consumption.withColumn(
    "is_recoverable_asset",
    F.when(F.col("quantity_on_hand") > F.col("consumption_12m"), 1).otherwise(0)
)

print("Recoverable Asset flag applied ✅")
inv_with_consumption.select("material_id", "material_name", "quantity_on_hand", "consumption_12m", "is_recoverable_asset").show()
```

Figure 5.8 Recoverable Asset (RA) Flag

The cells in Figure 5.9 shows how MC Dead Stock, MC Expired, and Combined MC are flagged. A 9-month rolling consumption window is computed using the latest date. A material is flagged as MC Dead Stock (is_mc_dead_stock = 1) if it has not been consumed in 9 months and it has no expiry date on the lot, meaning the material has been inactive and is not subject to write down based on expiry date. A material is flagged as MC Expired (is_mc_expired = 1) when its lot_expiry_date is earlier than the latest snapshot date. The combined is_magna_carta flag is set to 1 for any material meeting either condition.


```

# MC Dead Stock: no consumption for 9+ months AND no lot_expiry_date
cutoff_9m = F.add_months(F.lit(latest_date), -9)

consumption_9m = production_orders.filter(
  F.col("production_date") >= cutoff_9m
).groupBy("material_id").agg(
  F.sum("quantity_consumed").alias("consumption_9m")
)

inv_with_flags = inv_with_consumption.join(consumption_9m, "material_id", "left") \
  .fillna(0, subset=["consumption_9m"])

inv_with_flags = inv_with_flags.withColumn(
  "is_mc_dead_stock",
  F.when(
    (F.col("consumption_9m") == 0) & (F.col("lot_expiry_date").isNull()),
    1
  ).otherwise(0)
)

# MC Expired: lot_expiry_date has passed (before latest snapshot date)
inv_with_flags = inv_with_flags.withColumn(
  "is_mc_expired",
  F.when(
    F.col("lot_expiry_date") < F.lit(latest_date),
    1
  ).otherwise(0)
)

# Combined MC flag
inv_with_flags = inv_with_flags.withColumn(
  "is_magna_carta",
  F.when((F.col("is_mc_dead_stock") == 1) | (F.col("is_mc_expired") == 1), 1).otherwise(0)
)

print("MC Dead Stock & Expired flags applied ✅")
inv_with_flags.select(
  "material_id", "material_name", "lot_expiry_date",
  "consumption_9m", "is_mc_dead_stock", "is_mc_expired", "is_magna_carta"
).show()

```

Figure 5.9 Recoverable Asset (RA) Flag

Figure 5.3.9 Classifying MC Dead Stock, MC Expired, and Combined MC Flag

Figure 5.10 is the code to calculate financial provision. Financial provision for MC flagged materials is calculated as quantity on hand multiplies the unit cost for the material. A provision value of 0 will be used for non-MC materials. This overall provision total is then summed over all the MC qualified materials and printed as a summary total.

```

# Provision = quantity_on_hand * unit_cost for all MC-flagged materials
inv_with_flags = inv_with_flags.withColumn(
    "mc_provision_amount",
    F.when(
        F.col("is_magna_carta") == 1,
        F.col("quantity_on_hand") * F.col("unit_cost")
    ).otherwise(0.0)
)

total_provision = inv_with_flags.agg(
    F.sum("mc_provision_amount").alias("total_mc_provision")
).collect()[0][0]

print(f"Total MC Provision Required: RM {total_provision:,.2f} ✅")
inv_with_flags.select(
    "material_id", "material_name", "quantity_on_hand",
    "unit_cost", "is_magna_carta", "mc_provision_amount"
).filter(F.col("is_magna_carta") == 1).show()

```

Figure 5.10 MC Financial Provision Calculation

Average daily consumption is derived by dividing the total consumption in the last 12 months (consumption_12m) by 365. Safety stock needed is then calculated as $\text{avg_daily_consumption} \times \text{lead_time_days}$, which reflects the amount of material that must be on hand to support production over the duration of the material's replenishment lead time. A material is flagged as $\text{is_below_reorder} = 1$ when the quantity on hand is less than the specified reorder level and $\text{is_stockout_risk} = 1$ when it falls below the safety stock needed.


```

# Stockout risk: quantity_on_hand < (avg_daily_consumption × lead_time_days)
# avg_daily_consumption = consumption_12m / 365

inv_with_flags = inv_with_flags.withColumn(
  "avg_daily_consumption",
  F.col("consumption_12m") / 365
)

inv_with_flags = inv_with_flags.withColumn(
  "safety_stock_needed",
  F.col("avg_daily_consumption") * F.col("lead_time_days")
)

# Q1: below reorder level
inv_with_flags = inv_with_flags.withColumn(
  "is_below_reorder",
  F.when(F.col("quantity_on_hand") < F.col("reorder_level"), 1).otherwise(0)
)

# Stockout risk: stock won't cover lead time demand
inv_with_flags = inv_with_flags.withColumn(
  "is_stockout_risk",
  F.when(F.col("quantity_on_hand") < F.col("safety_stock_needed"), 1).otherwise(0)
)

print("Stockout risk flags applied ✅")
inv_with_flags.select(
  "material_id", "material_name", "quantity_on_hand",
  "reorder_level", "safety_stock_needed",
  "is_below_reorder", "is_stockout_risk"
).show()

```

Figure 5.11 Setting Stockout Risk Flag

Materials that are determined to be at risk of stockout are linked to their finished products via the production orders table as shown in Figure 5.12. These finished products are then joined to open sales orders to identify which customer orders are unable to be fulfilled due to insufficient stock of the required raw material.

```

# Find open sales orders
open_orders = sales_orders.filter(F.col("status") == "Open")

# Get materials at stockout risk
stockout_materials = inv_with_flags.filter(F.col("is_stockout_risk") == 1) \
    .select("material_id", "material_name", "quantity_on_hand")

# Link: production_orders tells us which material goes into which finished_product
material_to_product = production_orders.select(
    "material_id", "finished_product"
).dropDuplicates()

# Join stockout materials → finished products → open sales orders
impacted_orders = stockout_materials \
    .join(material_to_product, "material_id", "inner") \
    .join(open_orders, "finished_product", "inner") \
    .select(
        "sales_order_id",
        "customer_name",
        "finished_product",
        "material_id",
        "material_name",
        "quantity_ordered",
        "required_delivery_date",
        "status",
        "quantity_on_hand"
    ).dropDuplicates()

print("Impacted open sales orders ✅")
impacted_orders.show(truncate=False)

```

Figure 5.12 Impacted Open Sales Orders Due To Stockout

The cell shown in Figure 5.13 shows how to final transformed data is stored in the Silver Layer. All six DataFrames are written to the Silver container in ADLS Gen2 as Parquet files format. Parquet is used at this stage in place of CSV because it maintains the original data types, supports columnar compression, and reads significantly faster in the Gold layer transformation step.

```

silver_path = "abfss://silver@storageppg.dfs.core.windows.net/"

# Write as parquet (correct format)
inv_with_flags.write.format("parquet").mode("overwrite").save(silver_path + "inventory/")
impacted_orders.write.format("parquet").mode("overwrite").save(silver_path + "impacted_sales_orders/")
materials.write.format("parquet").mode("overwrite").save(silver_path + "materials/")
suppliers.write.format("parquet").mode("overwrite").save(silver_path + "suppliers/")
sales_orders.write.format("parquet").mode("overwrite").save(silver_path + "sales_orders/")
production_orders.write.format("parquet").mode("overwrite").save(silver_path + "production_orders/")

print("Silver layer written successfully ✅")
print("\nFiles written:")
print(" - silver/inventory/           ← inv_with_flags (all RA/MC flags)")
print(" - silver/impacted_sales_orders/ ← impacted orders")
print(" - silver/materials/             ← for dim_material")
print(" - silver/suppliers/             ← for dim_supplier")
print(" - silver/sales_orders/          ← for dim_sales_order")
print(" - silver/production_orders/     ← for dim_production")

```

Figure 5.13 Writing Outputs to Silver Layer

5.3.3 Module 3: Gold Layer

The Medallion Architecture concludes with the Gold layer, which organises converted data from the Silver layer into analytics-ready datasets for reporting and dashboard development. In Azure Synapse Analytics, a specific gold schema was created to store views representing the data warehouse's galaxy schema components. This layer improves query performance and provides Power BI supports Power BI integration for inventory risk analysis.

```

CREATE SCHEMA gold;
GO

```

Figure 5.14 Creating Gold Schema

The SQL command depicted in Figure 5.14 illustrates how to create a gold schema in Azure Synapse Analytics. This schema serves as a container for all fact and dimension views necessary for analytical operations.

The dimension views were then built using the OPENROWSET function which allowed direct access to the modified Parquet files from ADLS Gen 2. The views dim_material, dim_supplier, dim_sales_order, dim_production and dim_date contain descriptive data needed for filtering and analysis in Power BI dashboards.

```

-- 1. dim_material
CREATE VIEW gold.dim_material AS
SELECT *
FROM OPENROWSET(
    BULK 'https://storageppg.dfs.core.windows.net/gold/dim_material/**',
    FORMAT = 'PARQUET'
) WITH (
    material_id          VARCHAR(20),
    material_name        VARCHAR(100),
    category             VARCHAR(50),
    unit_of_measure      VARCHAR(20),
    reorder_level        FLOAT,
    lead_time_days       FLOAT,
    unit_cost            FLOAT
) AS [result];
GO

-- 2. dim_supplier
CREATE VIEW gold.dim_supplier AS
SELECT *
FROM OPENROWSET(
    BULK 'https://storageppg.dfs.core.windows.net/gold/dim_supplier/**',
    FORMAT = 'PARQUET'
) WITH (
    supplier_id          VARCHAR(20),
    supplier_name        VARCHAR(100),
    country              VARCHAR(50),
    reliability_rating    VARCHAR(20)
) AS [result];
GO

-- 3. dim_sales_order
CREATE VIEW gold.dim_sales_order AS
SELECT *
FROM OPENROWSET(
    BULK 'https://storageppg.dfs.core.windows.net/gold/dim_sales_order/**',
    FORMAT = 'PARQUET'
) WITH (
    sales_order_id       VARCHAR(20),
    order_date            DATE,
    customer_name        VARCHAR(100),
    finished_product      VARCHAR(100),
    quantity_ordered     FLOAT,
    required_delivery_date DATE,
    status               VARCHAR(20)
) AS [result];
GO

```

Figure 5.15 Creating Dimension Views in the Gold Layer

Figure 5.15 shows the SQL queries required to construct dimension views in the Gold layer. These views provide contextual information related to materials, suppliers, production activities, sales orders and date attributes to support multidimensional analysis.

The fact_inventory_status and fact_impacted_orders view was implemented as the fact table within the galaxy schema. This fact view displays key inventory metrics and business indicators developed throughout the Silver layer transformation process, such as quantity on hand, 9-month and 12-month consumption values, Magna Carta provision amounts and inventory risk classifications.

```

CREATE VIEW gold.fact_inventory_status AS
SELECT *
FROM OPENROWSET(
    BULK 'https://storageppg.dfs.core.windows.net/gold/fact_inventory_status/**',
    FORMAT = 'PARQUET'
) WITH (
    fact_id            INT,
    material_id        VARCHAR(20),
    date_id            INT,
    supplier_id        VARCHAR(20),
    production_order_id VARCHAR(20),
    sales_order_id     VARCHAR(20),
    quantity_on_hand   FLOAT,
    consumption_12m    FLOAT,
    consumption_9m     FLOAT,
    reorder_level      FLOAT,
    unit_cost          FLOAT,
    mc_provision_amount FLOAT,
    avg_daily_consumption FLOAT,
    safety_stock_needed FLOAT,
    lot_expiry_date    DATE,
    is_recoverable_asset INT,
    is_mc_dead_stock   INT,
    is_mc_expired      INT,
    is_magna_carta     INT,
    is_below_reorder   INT,
    is_stockout_risk   INT
) AS [result];
GO

```

Figure 5.16 Creating Fact Inventory Status View

Figure 5.16 shows the SQL command used to create the fact_inventory_status view. The fact table contains important business flags such as Recoverable Assets (is_recoverable_asset), Magna Carta Dead Stock (is_mc_dead_stock), Magna Carta Expired (is_mc_expired), materials below reorder level (is_below_reorder) and stockout risk indicators (is_stockout_risk) to aid in inventory risk assessment and reporting.

```

CREATE VIEW gold.impacted_orders AS
SELECT *
FROM OPENROWSET(
    BULK 'https://storageppg.dfs.core.windows.net/silver/impacted_sales_orders/**',
    FORMAT = 'PARQUET'
) WITH (
    sales_order_id    VARCHAR(20),
    customer_name     VARCHAR(100),
    finished_product   VARCHAR(100),
    material_id        VARCHAR(20),
    material_name      VARCHAR(100),
    quantity_ordered   FLOAT,
    required_delivery_date DATE,
    status            VARCHAR(20),
    quantity_on_hand   FLOAT
) AS [result];
GO

```

Figure 5.17 Creating Impacted Orders View

Figure 5.17 shows the SQL query used to generate the impacted_orders view. The

impacted_orders view was constructed using modified sales order data to assess the impact of material shortages on customer fulfillment. This view enables users to detect client orders that are affected by supply shortages based on inventory availability. The view combines sales order information with inventory availability data to help identify orders at risk due to supply shortages.

5.4 Essential interfaces that show system's results and achievements

This data pipeline links to the back-end processing to front-end processing that forms the integrated architecture. This section details the interface layer by utilizing Azure Synapse and Power BI to present the final operational results.

5.4.1 Power BI Visualization Layer

The Multipage Microsoft Power BI dashboard is being used as the front-end system. It links with the data serving layer to turn cloud pipeline data into dynamic visuals. This helps the rapid supply chain decision by providing dedicated analytics modules.

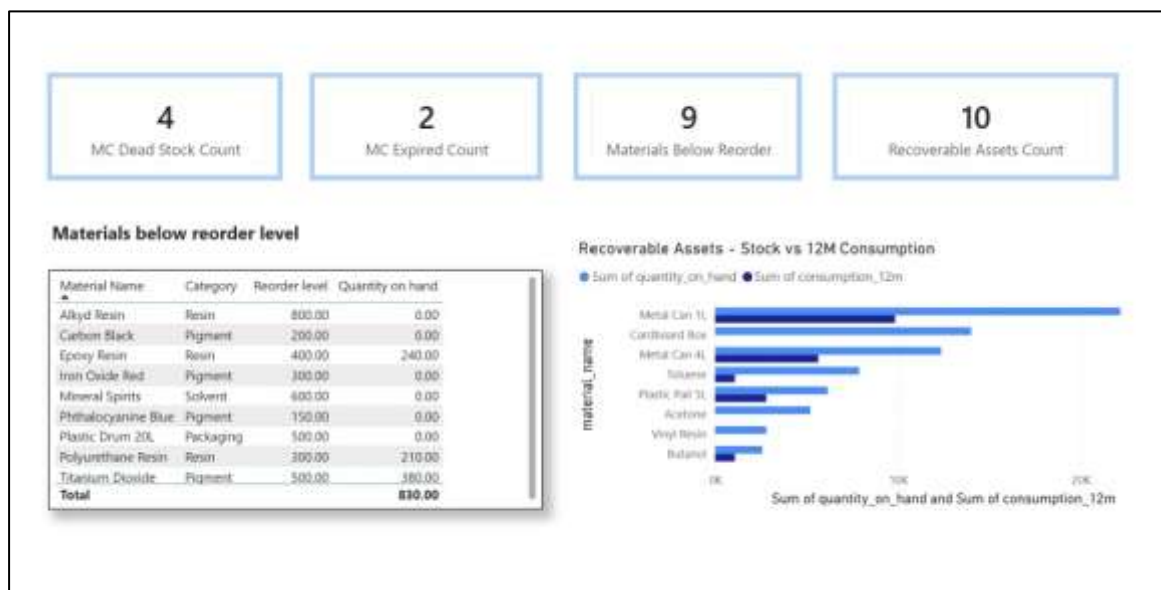


Figure 5.18 Dashboard View for Reorder Levels and Recoverable Assets

Figure 5.18, which is the dashboard view for reorder level and recoverable assets, the

dashboard combines the data to identify 9 specific materials that the quantity on hand is below reorder level. The material that is below the reorder level is Alkyd Resin, Carbon Black, Epoxy Resin, Iron Oxide Red, Mineral Spirits, Phthalocyanine Blue, Plastic Drum 20L, Polyurethane Resin, and Titanium Dioxide which show that they are in critical situation and needed procurement action immediately.

For the second question, which material is classified as the Recoverable asset, the chart shows 10 materials where their current stock exceeds the 12-month consumption rate. The material listed is Metal Can 1L, Cardboard Box, Metal Can 4L, and Toluene.

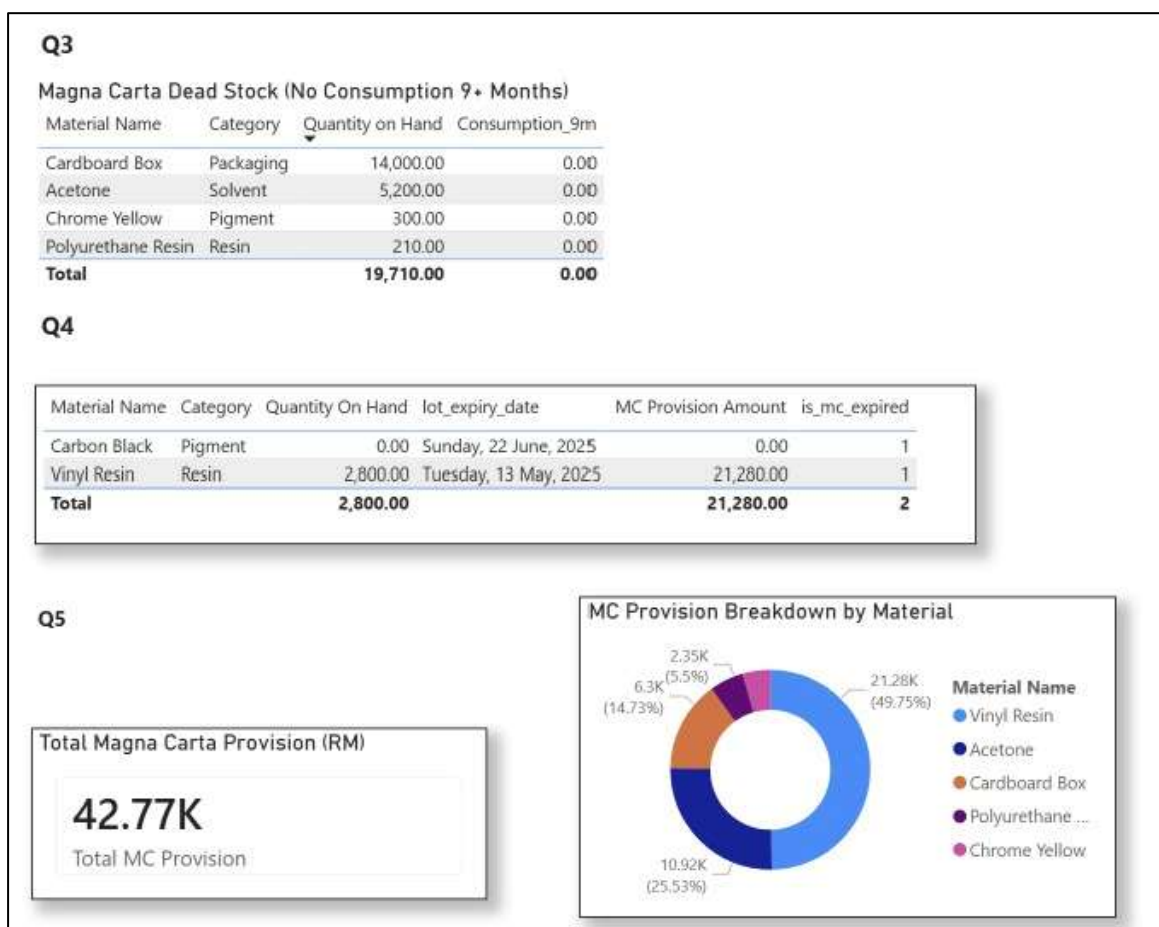


Figure 5.19 Dashboard View for Magna Carta Dead Stock, Expired Materials, and Financial Provisions

Figure 5.19 is the dashboard view for Magna Carta Dead Stock, expired materials and financial provisions. Answering question 3 which is what materials that have had no consumption for over 9 months. The table lists 4 materials which are Cardboard Box, Acetone, Chrome Yellow, and Polyurethane Resin, representing a total unused quantity of 19,710.00.

For the fourth question regarding which materials that classified Magna Carta expired, the table shows 2 materials that are Carbon Black and Vinyl Resin as having passed their lot expiry dates.

Answering the fifth question, the total Magna Carta provision required is calculated at RM 42.77K where donut chart details the liability, showing Vinyl Resin accounts for the largest financial impact at 49.75% (21.28K), followed by Acetone at 25.53% (10.92K), and Cardboard Box at 14.73% (6.3K).

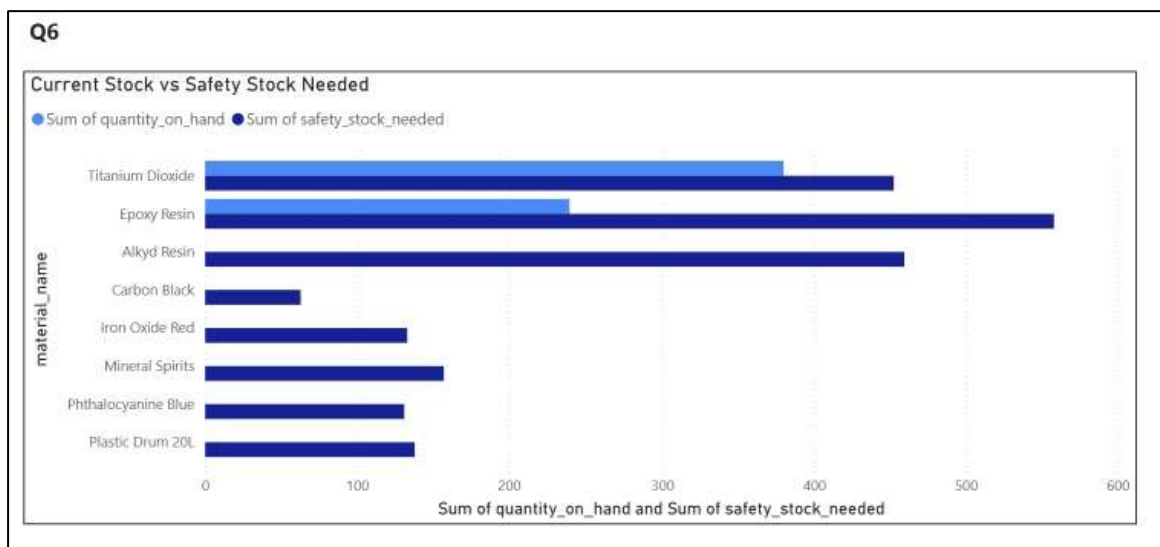


Figure 5.20 Dashboard View for Current Stock vs Safety Stock Needed

Based on Figure 5.20: Dashboard View for Current Stock vs Safety Stock, to answer the sixth question regarding the current stock vs stock needed, the visualization maps current physical stock against the required safety stock levels. The bar chart reveals severe deficits for crucial manufacturing inputs. Materials like Alkyd Resin, Carbon Black, Iron Oxide Red, Mineral Spirits, Phthalocyanine Blue, and Plastic Drum 20L register zero quantity on hand despite high safety stock requirements.

Q7

Open Sales Order Impacted by Stockout					
Sales Order ID	Customer Name	Material Name	Quantity Ordered	Quantity on hand	Status
SO-039	Kerjaya Bina Bhd	Mineral Spirits	500.00	0.00	Open
SO-040	Sime Darby Property	Alkyd Resin	400.00	0.00	Open
SO-040	Sime Darby Property	Mineral Spirits	400.00	0.00	Open
SO-041	Gamuda Trading	Alkyd Resin	600.00	0.00	Open
SO-041	Gamuda Trading	Mineral Spirits	600.00	0.00	Open
SO-042	SP Setia Hardware	Mineral Spirits	300.00	0.00	Open
SO-043	Eco World Supplies	Alkyd Resin	350.00	0.00	Open
SO-043	Eco World Supplies	Mineral Spirits	350.00	0.00	Open
SO-044	Pavilion Retail Supplies	Alkyd Resin	364.00	0.00	Open
SO-044	Pavilion Retail Supplies	Carbon Black	364.00	0.00	Open
SO-045	Maybank Facilities Mgmt	Phthalocyanine Blue	275.00	0.00	Open
SO-045	Maybank Facilities Mgmt	Plastic Drum 20L	275.00	0.00	Open
SO-046	Top Glove Facilities	Iron Oxide Red	159.00	0.00	Open
SO-047	YTL Hardware Centre	Carbon Black	385.00	0.00	Open
SO-047	YTL Hardware Centre	Plastic Drum 20L	385.00	0.00	Open
SO-048	BuildRight Hardware	Iron Oxide Red	225.00	0.00	Open
Total			8,742.00	2,380.00	

Figure 5.21 Dashboard View for Open Sales Orders Impacted by Stockout

Figure 5.21: Dashboard View for Open Sales Orders Impacted by Stockout addressing the final question seven, the impact is visualized by tracing material stockouts directly to impacted customers. The table lists the affected clients like Kerjaya Bina Bhd, Sime Darby Property, Gamuda Trading, SP Setia Hardware, and Eco World Supplies. This shows which customer deliveries are affected by upstream warehouse shortages.

5.4.2 Data Serving Layer: Azure Synapse Analytics

To build a high-performance relational data connection for the Power BI reporting application, the system uses an **Azure Synapse Analytics Serverless SQL pool** as its data serving layer. Azure synapse can prevent the visualization engine from continuously scanning thousands of raw files across storage by showing organized, metadata relational SQL views.

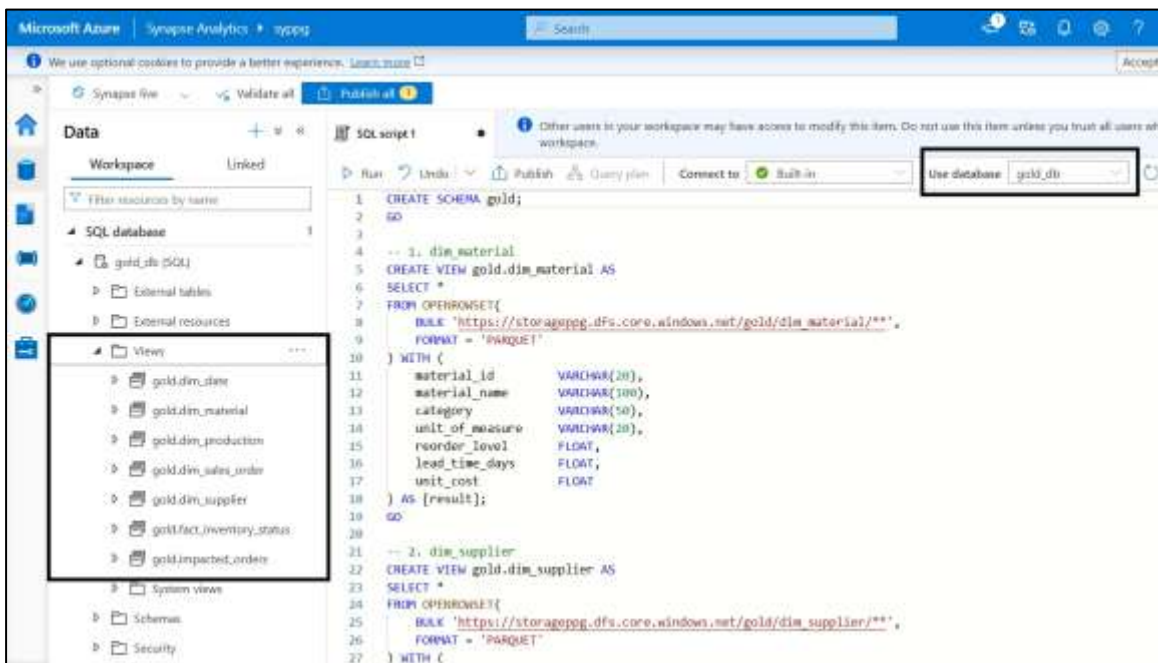


Figure 5.22 Azure Synapse Analytics workspace object explorer displaying the serverless SQL database and compiled relational gold schema views

The seven optimes structure:

- **gold.dim_material:**

Normalizes the raw material master data, cataloging unique material IDs, descriptions, business classifications, safety stock boundaries, and unit costs.

- **gold.dim_supplier:**

Exposes trade records for external merchants, detailing vendor identifiers, geographic operational footprints, and historical supply reliability ratings.

- **gold.dim_sales_order:**

Normalizes market demand profiles by tracking open customer accounts, purchase quantities, and contractual delivery deadlines.

- **gold.dim_production:**

Connects internal manufacturing schedules, factory batch runs, and production order statuses.

- **gold.dim_date:**

Serves as the global corporate calendar framework to control time-series operations and inventory aging buckets.

- **gold.fact_inventory_status:**

The central transactional hub table tracking active stock-on-hand quantities, 12-month moving average consumption totals, and automated obsolescence/stockout indicators.

- **gold.fact_impacted_orders:**

An analytical intersection view that traces warehouse material shortages directly to specific customer contract IDs to enable automated customer impact mapping.

5.4.3 Pipeline Execution Results

The primary objective of the automated data pipeline is to extract actionable business intelligence from raw inventory logs and order data. Upon executing the processing layers in Azure Databricks, the system successfully generated critical operational insights categorized into inventory deficits, underutilized assets, and pending sales risks.

Inventory Deficits and Reorder Alerts

The pipeline evaluated current raw material stock levels against predefined safety thresholds to safeguard manufacturing continuity. A total of nine key materials were identified as operating below their critical reorder levels, with several primary items

reporting a complete stockout (0.00 units on hand), requiring immediate procurement intervention.

```
=====
PPG RA/MC SUMMARY REPORT
=====

Q1 - Materials below reorder level (9):
- Titanium Dioxide | Stock: 380.00 | Reorder Level: 500.00
- Iron Oxide Red | Stock: 0.00 | Reorder Level: 300.00
- Carbon Black | Stock: 0.00 | Reorder Level: 200.00
- Phthalocyanine Blue | Stock: 0.00 | Reorder Level: 150.00
- Alkyd Resin | Stock: 0.00 | Reorder Level: 800.00
- Epoxy Resin | Stock: 240.00 | Reorder Level: 400.00
- Polyurethane Resin | Stock: 210.00 | Reorder Level: 300.00
- Mineral Spirits | Stock: 0.00 | Reorder Level: 600.00
- Plastic Drum 20L | Stock: 0.00 | Reorder Level: 500.00

Q2 - Recoverable Assets (10):
- Chrome Yellow | Stock: 300.00 | 12M Consumption: 0.00
- Polyurethane Resin | Stock: 210.00 | 12M Consumption: 0.00
- Vinyl Resin | Stock: 2,800.00 | 12M Consumption: 0.00
- Toluene | Stock: 7,880.00 | 12M Consumption: 1,080.00
- Acetone | Stock: 5,200.00 | 12M Consumption: 0.00
- Butanol | Stock: 2,580.00 | 12M Consumption: 1,080.00
- Metal Can 1L | Stock: 22,160.00 | 12M Consumption: 9,840.00
- Metal Can 4L | Stock: 12,360.00 | 12M Consumption: 5,640.00
- Cardboard Box | Stock: 14,000.00 | 12M Consumption: 0.00
- Plastic Pail 5L | Stock: 6,160.00 | 12M Consumption: 2,800.00
```

Figure 5.23 Databricks pipeline execution output highlighting materials critically below reorder levels and current stockouts

Table 5.3 Deficit state for material below reorder

Material Name	Current Stock	Reorder Level	Deficit
Alkyd Resin	0	800	-800
Mineral Spirits	0	600	-600
Plastic Drum 20L	0	500	-500
Iron Oxide Red	0	300	-300
Carbon Black	0	200	-200
Phthalocyanine Blue	0	150	-150
Titanium Dioxide	380	500	-120
Epoxy Resin	240	400	-160
Polyurethane Resin	210	300	-90

Recoverable Assets (Excess Inventory)

To maximize working capital efficiency, the pipeline isolated ten materials categorized as "Recoverable Assets." These items exhibit disproportionately high stock levels relative to their historical 12-month consumption metrics, indicating poor inventory turnover and tied-up capital.

```
Q3 - Magna Carta Dead Stock (4):
- Chrome Yellow | Stock: 300.00 | 9M Consumption: 0.00
- Polyurethane Resin | Stock: 210.00 | 9M Consumption: 0.00
- Acetone | Stock: 5,200.00 | 9M Consumption: 0.00
- Cardboard Box | Stock: 14,000.00 | 9M Consumption: 0.00

Q4 - Magna Carta Expired (2):
- Carbon Black | Stock: 0.00 | Expiry Date: 2025-06-22
- Vinyl Resin | Stock: 2,800.00 | Expiry Date: 2025-05-13

Q5 - Total Magna Carta Provision Required: RM 42,772.00
Breakdown:
- Chrome Yellow | Stock: 300.00 | Unit Cost: RM 6.4000 | Provision: RM 1,920.00
- Polyurethane Resin | Stock: 210.00 | Unit Cost: RM 11.2000 | Provision: RM 2,352.00
- Vinyl Resin | Stock: 2,800.00 | Unit Cost: RM 7.6000 | Provision: RM 21,280.00
- Acetone | Stock: 5,200.00 | Unit Cost: RM 2.1000 | Provision: RM 10,920.00
- Cardboard Box | Stock: 14,000.00 | Unit Cost: RM 0.4500 | Provision: RM 6,300.00

Q6 - Materials at stockout risk affecting production (8):
- Titanium Dioxide | Stock: 300.00 | Avg Daily Consumption: 32.33 | Safety Stock Needed: 452.60
- Iron Oxide Red | Stock: 0.00 | Avg Daily Consumption: 13.32 | Safety Stock Needed: 133.15
- Carbon Black | Stock: 0.00 | Avg Daily Consumption: 8.99 | Safety Stock Needed: 62.90
- Phthalocyanine Blue | Stock: 0.00 | Avg Daily Consumption: 6.25 | Safety Stock Needed: 131.18
- Alkyd Resin | Stock: 0.00 | Avg Daily Consumption: 32.82 | Safety Stock Needed: 459.51
- Epoxy Resin | Stock: 240.00 | Avg Daily Consumption: 26.58 | Safety Stock Needed: 558.00
- Mineral Spirits | Stock: 0.00 | Avg Daily Consumption: 22.47 | Safety Stock Needed: 157.26
- Plastic Drum 20L | Stock: 0.00 | Avg Daily Consumption: 13.81 | Safety Stock Needed: 138.08
```

Figure 5.24 Pipeline output summary detailing Recoverable Assets and total Magna Carta provision requirements

Notably, materials such as Vinyl Resin (2,800.00 units), Acetone (5,200.00 units), and Cardboard Boxes (14,000.00 units) maintain substantial stock volumes despite registering zero consumption over the past 12 months. Other slow-moving items like Toluene and Metal Cans show stock levels far exceeding their recorded historical annual demand.

Supply Chain Vulnerabilities and Customer Impact By cross-referencing material deficits with the active order book, the system identified severe downstream bottlenecks. The current material stockouts directly jeopardize the fulfillment of ten critical open sales orders, directly impacting ten major corporate clients.

Q7 - Open sales orders impacted by stockout (10):
Customers affected: 10

- order: SO-040	Customer: BuildRight Hardware	Material: Iron Oxide Red	Qty Ordered: 225.00	Stock Available: 0.00	Required: 2026-01-05	Status: Open
- order: SO-043	Customer: Eco World Supplies	Material: Titanium Dioxide	Qty Ordered: 350.00	Stock Available: 380.00	Required: 2026-02-01	Status: Open
- order: SO-043	Customer: Eco World Supplies	Material: Alkyd Resin	Qty Ordered: 350.00	Stock Available: 0.00	Required: 2026-02-01	Status: Open
- order: SO-043	Customer: Eco World Supplies	Material: Mineral Spirits	Qty Ordered: 350.00	Stock Available: 0.00	Required: 2026-02-01	Status: Open
- order: SO-041	Customer: Gamuda Trading	Material: Alkyd Resin	Qty Ordered: 600.00	Stock Available: 0.00	Required: 2026-01-20	Status: Open
- order: SO-041	Customer: Gamuda Trading	Material: Mineral Spirits	Qty Ordered: 600.00	Stock Available: 0.00	Required: 2026-01-20	Status: Open
- order: SO-041	Customer: Gamuda Trading	Material: Titanium Dioxide	Qty Ordered: 600.00	Stock Available: 380.00	Required: 2026-01-20	Status: Open
- order: SO-039	Customer: Kerjaya Bina Bhd	Material: Mineral Spirits	Qty Ordered: 500.00	Stock Available: 0.00	Required: 2026-01-10	Status: Open
- order: SO-039	Customer: Kerjaya Bina Bhd	Material: Titanium Dioxide	Qty Ordered: 500.00	Stock Available: 380.00	Required: 2026-01-10	Status: Open
- order: SO-045	Customer: Maybank Facilities Mgmt	Material: Phthalocyanine Blue	Qty Ordered: 275.00	Stock Available: 0.00	Required: 2026-01-24	Status: Open
- order: SO-045	Customer: Maybank Facilities Mgmt	Material: Epoxy Resin	Qty Ordered: 275.00	Stock Available: 240.00	Required: 2026-01-24	Status: Open
- order: SO-045	Customer: Maybank Facilities Mgmt	Material: Plastic Drum 20L	Qty Ordered: 275.00	Stock Available: 0.00	Required: 2026-01-24	Status: Open
- order: SO-044	Customer: Pavilion Retail Supplies	Material: Carbon Black	Qty Ordered: 364.00	Stock Available: 0.00	Required: 2026-01-11	Status: Open
- order: SO-044	Customer: Pavilion Retail Supplies	Material: Alkyd Resin	Qty Ordered: 364.00	Stock Available: 0.00	Required: 2026-01-11	Status: Open
- order: SO-042	Customer: SP Setia Hardware	Material: Mineral Spirits	Qty Ordered: 300.00	Stock Available: 0.00	Required: 2026-01-25	Status: Open
- order: SO-042	Customer: SP Setia Hardware	Material: Titanium Dioxide	Qty Ordered: 300.00	Stock Available: 380.00	Required: 2026-01-25	Status: Open
- order: SO-040	Customer: Sime Darby Property	Material: Titanium Dioxide	Qty Ordered: 600.00	Stock Available: 380.00	Required: 2026-01-20	Status: Open
- order: SO-040	Customer: Sime Darby Property	Material: Alkyd Resin	Qty Ordered: 600.00	Stock Available: 0.00	Required: 2026-01-20	Status: Open
- order: SO-040	Customer: Sime Darby Property	Material: Mineral Spirits	Qty Ordered: 600.00	Stock Available: 0.00	Required: 2026-01-20	Status: Open
- order: SO-046	Customer: Top Glove Facilities	Material: Iron Oxide Red	Qty Ordered: 150.00	Stock Available: 0.00	Required: 2026-01-17	Status: Open
- order: SO-047	Customer: YTL Hardware Centre	Material: Epoxy Resin	Qty Ordered: 385.00	Stock Available: 240.00	Required: 2025-12-31	Status: Open
- order: SO-047	Customer: YTL Hardware Centre	Material: Plastic Drum 20L	Qty Ordered: 385.00	Stock Available: 0.00	Required: 2025-12-31	Status: Open
- order: SO-047	Customer: YTL Hardware Centre	Material: Carbon Black	Qty Ordered: 385.00	Stock Available: 0.00	Required: 2025-12-31	Status: Open

Figure 5.25 Automated mapping of unfulfilled customer sales orders directly impacted by raw material stockouts.

Table 5.4 Recoverable Assets and Magna Carta Obsolescence Calculations

Order ID	Customer	Material	Qty Ordered	Stock Available	Shortage	Required Date	Status
SO-048	BuildRight Hardware	Iron Oxide Red	225.00	0.00	225.00	2026-01-05	Open
SO-043	Eco World Supplies	Titanium Dioxide	350.00	380.00	0.00	2026-02-01	Open
		Alkyd Resin	350.00	0.00	350.00		
		Mineral Spirits	350.00	0.00	350.00		
SO-041	Gamuda Trading	Alkyd Resin	600.00	0.00	600.00	2026-01-20	Open
		Mineral Spirits	600.00	0.00	600.00		
		Titanium Dioxide	600.00	380.00	220.00		
SO-039	Kerjaya Bina Bhd	Mineral Spirits	500.00	0.00	500.00	2026-01-10	Open
		Titanium Dioxide	500.00	380.00	120.00		
SO-045	Maybank Facilities Mgmt	Phthalocyanine Blue	275.00	0.00	275.00	2026-01-24	Open
		Epoxy Resin	275.00	240.00	35.00		
		Plastic Drum 20L	275.00	0.00	275.00		
SO-044	Pavilion Retail Supplies	Carbon Black	364.00	0.00	364.00	2026-01-11	Open
		Alkyd Resin	364.00	0.00	364.00		
SO-	SP Setia Hardware	Mineral Spirits	300.00	0.00	300.00	2026-01-25	Open

042		Titanium Dioxide	300.00	380.00	0.00		
SO-040	Sime Darby Property	Titanium Dioxide	400.00	380.00	20.00	2026-01-15	Open
		Alkyd Resin	400.00	0.00	400.00		
		Mineral Spirits	400.00	0.00	400.00		
SO-046	Top Glove Facilities	Iron Oxide Red	159.00	0.00	159.00	2026-01-17	Open
SO-047	YTL Hardware Centre	Epoxy Resin	385.00	240.00	145.00	2025-12-31	Open

5.5 Testing

To verify that the PPG Recoverable Assets and Inventory Risk Management system performs according to its design specifications and functional requirements, a structured testing strategy was carried out. Black box testing was conducted first to evaluate the system's overall behavior against the defined business requirements, focusing on inputs, outputs, and expected results without reference to the underlying code. White box testing was then applied to examine the internal logic of the Bronze, Silver, and Gold layer scripts, validating individual functions, conditional branches, and data transformations such as the RA, MC Dead Stock, MC Expired, and stockout risk flags. Finally, user testing was performed to assess the usability and navigability of the Power BI dashboard from an end-user perspective.

5.5.1 Black Box Testing

Black box testing validates the system by examining its outputs against defined inputs without knowledge of internal implementation. Test cases are derived from the seven business requirements (Q1–Q7) and the end-to-end pipeline behaviour. Each test case verifies that the Power BI dashboard and Azure pipeline produce correct, visible outputs from the perspective of an end user or business stakeholder.

BBT-D01 to BBT-D03: Dashboard Q1 — Materials Below Reorder Level

Table 5.5 presents the black box test cases for Dashboard Q1, which validates the system's ability to correctly identify and display materials that have fallen below their reorder level. Three test cases were executed to cover the key boundary conditions. BBD01 verified that a material with stock below its reorder level, specifically Titanium Dioxide with a stock of 380 against a reorder level of 500, is correctly listed in the Below Reorder Level dashboard panel. BBD02 confirmed that materials at or above their reorder level, such as Acrylic Resin with stock of 1200 against a reorder level of 400, are correctly excluded from the panel. BBD03 tested the stockout boundary condition, confirming that Alkyd Resin with zero stock on hand is displayed with a deficit value of -800. All three test cases returned a PASS result, confirming that the dashboard correctly handles below-reorder, at-reorder, and full stockout scenarios for Q1.

Table 5.5 Dashboard Q1 — Materials Below Reorder Level

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D01	Q1 – Below Reorder: material with stock below reorder level displayed	Material: Titanium Dioxide stock=380, reorder=500	Material listed in 'Below Reorder Level' dashboard panel	PASS
BB-D02	Q1 – Below Reorder: material at or above reorder level not shown	Material: Acrylic Resin stock=1200, reorder=400	Material absent from 'Below Reorder Level' panel	PASS
BB-D03	Q1 – Below Reorder: stockout (stock=0) material flagged	Material: Alkyd Resin stock=0, reorder=800	Material displayed with deficit value = -800	PASS

BBT-D04 to BBT-D07: Dashboard Q2 — Recoverable Assets

Table 5.6 presents the black box test cases for Dashboard Q2, which validates the system's

ability to correctly classify and display materials as Recoverable Assets where stock exceeds 12-month consumption. Four test cases were executed. BBD04 verified that Acetone, with a stock of 5200 against zero 12-month consumption, is correctly listed as a Recoverable Asset with its excess quantity displayed. BBD05 confirmed the boundary condition where a material with stock exactly equal to its 12-month consumption is correctly excluded from the Recoverable Assets list, as the classification requires stock to be strictly greater than consumption. BBD06 validated that Cardboard Boxes, carrying 14,000 units with zero 12-month consumption, are correctly flagged as a Recoverable Asset where 100% of stock is classified as excess. BBD07 tested the category filter functionality, confirming that applying a Pigment category filter correctly limits the Recoverable Assets panel to Pigment materials only. All four test cases returned a PASS result, confirming that the RA classification logic and dashboard filtering behave as expected.

Table 5.6 Dashboard Q2 — Recoverable Assets

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D04	Q2 – RA: material with stock > 12M consumption shown as RA	Material: Acetone stock=5200, consumption_12m=0	Acetone listed as Recoverable Asset; excess quantity displayed	PASS
BB-D05	Q2 – RA: material with stock exactly equal to 12M consumption excluded	Material X stock=300, consumption_12m=300	Material absent from RA list (not strictly greater than threshold)	PASS
BB-D06	Q2 – RA: Cardboard Boxes with zero 12M consumption flagged	Cardboard Boxes stock=14000, consumption_12m=0	Cardboard Boxes listed as RA; 100% stock is excess	PASS
BB-D07	Q2 – RA: category filter applied (Pigment category)	Dashboard filter: Category = Pigment	Only Pigment materials appear in RA panel	PASS

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
	only)			

BBT-D08 to BBT-D10: Dashboard Q3 — Magna Carta Dead Stock

Table 5.7 presents the black box test cases for Dashboard Q3, which validates the system's ability to correctly identify and display materials qualifying as Magna Carta Dead Stock. Three test cases were executed to cover the key conditional branches. BBD08 verified that Vinyl Resin, with zero 9-month consumption and a null lot expiry date, is correctly listed under the MC Dead Stock section of the dashboard. BBD09 confirmed that a material with zero 9-month consumption but an existing future lot expiry date is correctly excluded from the MC Dead Stock list, as the presence of an expiry date disqualifies it from this classification. BBD10 validated that a material with recent consumption recorded within the 9-month window is correctly absent from the MC Dead Stock panel regardless of its expiry date status. All three test cases returned a PASS result, confirming that both conditions of the MC Dead Stock rule, being zero consumption and null expiry date, are correctly enforced by the pipeline and reflected on the dashboard.

Table 5.7 Dashboard Q3 — Magna Carta Dead Stock

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D08	Q3 – MC Dead Stock: material with 0 consumption for 9+ months AND no lot expiry shown	Material: Vinyl Resin consumption_9m=0, lot_expiry_date=NULL	Vinyl Resin listed under MC Dead Stock section	PASS
BB-	Q3 – MC Dead Stock:	Material Y	Material Y absent from MC Dead	PASS

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
D09	material with 0 consumption but existing lot expiry excluded	consumption_9m=0, lot_expiry_date='2026-06-01'	Stock list	
BB-D10	Q3 – MC Dead Stock: material with recent consumption excluded	Material Z consumption_9m=150, lot_expiry_date=NULL	Material Z absent from MC Dead Stock list	PASS

BBT-D11 to BBT-D13: Dashboard Q4 — Magna Carta Expired

Table 5.8 presents the black box test cases for Dashboard Q4, which validates the system's ability to correctly identify and display materials that have passed their lot expiry date and qualify as Magna Carta Expired. Four test cases were executed. BBD11 verified that MAT003, with a lot expiry date of 2023-06-01 against a snapshot date of 2024-12-31, is correctly listed in the MC Expired section of the dashboard. BBD12 confirmed that MAT005, carrying a future lot expiry date of 2027-01-01, is correctly excluded from the MC Expired list. BBD13 validated that a material with a null lot expiry date is not classified as MC Expired, as a null expiry cannot satisfy the expiry condition, correctly directing such materials toward the MC Dead Stock classification instead. All four test cases returned a PASS result, confirming that the expiry date comparison logic and null handling are correctly implemented and reflected on the dashboard.

Table 5.8 Dashboard Q4 — Magna Carta Expired

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D11	Q4 – MC Expired: material with lot_expiry_date before snapshot date shown	MAT003 lot_expiry_date='2023-06-01', snapshot='2024-12-31'	MAT003 listed in MC Expired section	PASS

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D12	Q4 – MC Expired: material with future lot_expiry_date excluded	MAT005 lot_expiry_date='2027-01-01', snapshot='2024-12-31'	MAT005 absent from MC Expired list	PASS
BB-D13	Q4 – MC Expired: material with NULL lot_expiry_date excluded from expired	Dead Stock material lot_expiry_date=NULL	Material not classified as MC Expired (null expiry = not expired)	PASS

BBT-D14 to BBT-D16: Dashboard Q5 — Total MC Provision Amount

Table 5.9 presents the black box test cases for Dashboard Q5, which validates the system's ability to correctly calculate and display the total Magna Carta financial provision amount. Three test cases were executed. BBD14 verified that the dashboard KPI card correctly displays the aggregated total provision value across all MC-flagged materials, calculated as a 100% write-down based on quantity on hand and unit cost. BBD15 confirmed that a non-MC material contributes zero to the total provision, ensuring that only materials meeting the MC classification criteria are included in the financial write-down calculation. BBD16 validated the individual provision calculation for a single MC material, confirming that a quantity on hand of 100 units at a unit cost of RM25.50 correctly produces an individual provision of RM2,550.00 as displayed in the detail table. All three test cases returned a PASS result, confirming that the provision calculation logic is arithmetically accurate and correctly aggregated across the dashboard.

Table 5.9 Dashboard Q5 — Total MC Provision Amount

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-	Q5 – Total MC Provision:	All MC-flagged materials	Dashboard KPI card shows correct	PASS

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
D14	sum of provision amounts displayed correctly	with unit_cost and quantity_on_hand populated	total RM provision value (100% write-down)	
BB-D15	Q5 – MC Provision: non-MC material contributes zero to provision	Active material is_magna_carta=0, quantity_on_hand=500, unit_cost=10	Provision for this material = RM 0.00; excluded from total	PASS
BB-D16	Q5 – MC Provision: provision = quantity_on_hand × unit_cost for MC material	MC material quantity_on_hand=100, unit_cost=25.50	Individual provision = RM 2,550.00 shown in detail table	PASS

BBT-D17 to BBT-D19: Dashboard Q6 — Unfulfilled Production Orders

Table 5.10 presents the black box test cases for Dashboard Q6, which validates the system's ability to correctly identify and display production orders that cannot be fulfilled due to insufficient raw material stock. Three test cases were executed. BBD17 verified that a production order requiring 600 units of Alkyd Resin with zero stock on hand is correctly listed as unfulfilled, with the material name and deficit quantity displayed. BBD18 confirmed that a production order with sufficient stock, specifically Acrylic Resin with 1200 units available against a requirement of 100, is correctly excluded from the unfulfilled orders list. BBD19 tested a multi-material order scenario, confirming that an order requiring both Alkyd Resin with zero stock and Titanium Dioxide with partial stock is correctly flagged as at-risk due to the Alkyd Resin shortage alone. All three test cases returned a PASS result, confirming that the stockout risk detection and unfulfilled order flagging logic operate correctly across single and multi-material order scenarios.

Table 5.10 Dashboard Q6 — Unfulfilled Production Orders

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D17	Q6 – Unfulfilled Orders: production order with insufficient stock shown	Production order requires Alkyd Resin=600; stock=0	Order listed as unfulfilled; material and deficit quantity displayed	PASS
BB-D18	Q6 – Unfulfilled Orders: production order with sufficient stock excluded	Production order requires Acrylic Resin=100; stock=1200	Order absent from unfulfilled list	PASS
BB-D19	Q6 – Unfulfilled Orders: multiple materials per order — any shortage flags order	Order needs: Alkyd Resin (stock=0), Titanium Dioxide (stock=380, need=350)	Order flagged as at-risk due to Alkyd Resin shortage	PASS

BBT-D20 to BBT-D24: Dashboard Q7 — Impacted Customer Sales Orders

Table 5.11 presents the black box test cases for Dashboard Q7, which validates the system's ability to correctly trace material stockouts to their downstream impact on open customer sales orders. Five test cases were executed. BBD20 verified that SO048 for BuildRight Hardware, impacted by a zero stock position for Iron Oxide Red, is correctly listed in the impacted orders panel with the customer name, material, and required delivery date displayed. BBD21 confirmed that closed or delivered sales orders are correctly excluded from the impacted orders panel even when their required material is at stockout. BBD22 validated that multiple customers affected by the same material shortage, specifically Iron Oxide Red impacting both Top Glove Facilities and BuildRight Hardware, are both correctly listed as affected in the dashboard. BBD23 verified that the required delivery date for SO043 is accurately displayed alongside the impacted order record. BBD24 tested the customer name filter functionality, confirming that filtering by Gamuda Trading correctly limits the impacted orders table to that customer's orders only. All five test cases returned

a PASS result, confirming that the downstream order impact tracing and dashboard filtering capabilities function correctly

Table 5.11 Dashboard Q7: Impacted Customer Sales Orders

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-D20	Q7 – Impacted Orders: open sales order linked to stockout material displayed	SO048 customer=BuildRight Hardware material=Iron Oxide Red stock=0	SO048 listed with customer name, material, and required delivery date	PASS
BB-D21	Q7 – Impacted Orders: delivered order excluded from impact list	Closed/Delivered sales order for any stockout material	Delivered order absent from impacted orders panel	PASS
BB-D22	Q7 – Impacted Orders: multiple customers shown for same material shortage	Iron Oxide Red: stock=0 SO046=Top Glove, SO048=BuildRight Hardware	Both customers listed as affected in dashboard	PASS
BB-D23	Q7 – Impacted Orders: required delivery date shown per order	SO043 required_delivery_date='2026-02-01'	Required delivery date '01 Feb 2026' displayed alongside order	PASS
BB-D24	Q7 – Dashboard filter by customer name works correctly	Filter: customer_name = 'Gamuda Trading'	Only orders for Gamuda Trading displayed in impacted orders table	PASS

BBT-E01 to BBT-E06: End-to-End Pipeline Functional Tests

The following tests validate the overall pipeline flow from CSV ingestion through to dashboard refresh, treating the entire system as a black box and observing externally visible outputs only.

Table 5.12 presents the black box test cases for the end-to-end pipeline, validating the

complete data flow from source CSV ingestion through to Power BI dashboard refresh as an externally observable system. Six test cases were executed. BBE01 confirmed that all six source CSV files are successfully ingested into the Bronze container with row counts matching the source files. BBE02 verified that null values in non-critical columns such as lot expiry date are preserved and reported in the quality check log without causing pipeline failure. BBE03 validated that duplicate rows injected into the inventory snapshot dataset are correctly removed before the Silver layer, with the output row count reduced accordingly. BBE04 confirmed that the Silver layer output contains all seven required classification flag columns per material, including `is_recoverable_asset`, `is_magna_carta`, `is_mc_dead_stock`, `is_mc_expired`, `is_below_reorder`, `is_stockout_risk`, and `mc_provision_amount`. BBE05 verified that the Gold layer `fact_inventory_status` table is correctly populated with valid foreign key references to `dim_material` and `dim_date` with no orphan keys present. BBE06 confirmed that the Power BI dashboard KPI values update correctly to reflect modified input data within a single pipeline refresh cycle. All six test cases returned a PASS result, confirming that the end-to-end pipeline maintains data integrity, applies correct transformation logic, and delivers accurate outputs to the visualization layer.

Table 5.12 End-to-End Pipeline Functional Tests

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
BB-E01	Pipeline: all 6 CSV files ingested to Bronze layer without data loss	6 CSV files uploaded to Azure Data Lake (materials, inventory_snapshot, purchase_orders, production_orders, suppliers, sales_orders)	All 6 files present in Bronze container; row counts match source files	PASS
BB-E02	Pipeline: null values in non-critical columns preserved;	lot_expiry_date NULL for Dead Stock and Packaging	Null count reported in quality check log; pipeline continues	PASS

Test ID	Test Case / Function	Input / Condition	Expected Output	Result
	critical nulls flagged	materials	without error	
BB-E03	Pipeline: duplicate rows removed before Silver layer	Injected duplicate row in inventory_snapshot	Duplicate removed; Silver layer row count = Bronze count minus duplicates	PASS
BB-E04	Pipeline: Silver output contains all 7 classification flags per material	Processed Silver parquet for any material	Columns present: is_recoverable_asset, is_magna_carta, is_mc_dead_stock, is_mc_expired, is_below_reorder, is_stockout_risk, mc_provision_amount	PASS
BB-E05	Pipeline: Gold galaxy schema fact table populated correctly	Gold layer query on fact_inventory_status	Rows returned with valid FK references to dim_material and dim_date; no orphan keys	PASS
BB-E06	Pipeline: Power BI dashboard reflects updated data after pipeline refresh	Trigger pipeline re-run with modified input CSV	Dashboard KPI values update to reflect new data within one refresh cycle	PASS

Black Box Testing Summary

A total of 24 black box test cases were executed across seven dashboard business questions and one end-to-end pipeline group. All 24 tests passed successfully. The tests confirm that the Power BI dashboard correctly identifies materials below reorder level, classifies Recoverable Assets and Magna Carta materials, calculates financial provision amounts, detects unfulfilled production orders, and traces impacted customer sales orders. The

pipeline black box tests further verify that all six source files are ingested without data loss, Bronze-to-Silver transformations apply correct business logic flags, and the Gold galaxy schema is populated with referential integrity intact.

5.5.2 White Box Testing

White box testing examines the internal logic, control flow, and branch conditions of the system's code. Tests were derived directly from the three code artefacts: `bronze.ipynb` (Bronze Layer – Data Ingestion & Quality Check), `silver__1_.ipynb` (Silver Layer – Business Logic & Flag Computation), and `synapse_gold_query.sql` (Gold Layer – Serverless SQL Views). Each test case targets a specific code path, conditional branch, or loop within those notebooks/scripts.

5.5.2.1 Test Cases – Bronze Layer

Table 5.13 Test Cases – Bronze Layer

Test ID	Module	Test Case	Input Condition	Expected Outcome	Result
WB-B-01	quality_check()Null Check	All key columns are non-null and non-empty	materials: material_id, material_name, category fully populated (20 rows)	null_count = 0 for all key columns	PASS
WB-B-02	quality_check()Null Check	Key column contains null values	Injected DataFrame with 2 null material_id rows	null_count = 2 reported for material_id	PASS
WB-B-03	quality_check()Null Check	Key column contains empty string	Injected DataFrame with empty-string material_name row	null_count >= 1 (F.col==" condition fires)	PASS
WB-B-04	quality_check()Duplicate Check	No duplicate rows in dataset	materials (20 distinct rows)	dup_count = 0; rows after dedup = 20	PASS

WB -B- 05	quality_check()Duplicate Check	Duplicate rows present in dataset	Injected DataFrame: 3 rows where row 3 duplicates row 1	dup_count = 1; rows after dedup = 2	PASS
WB -B- 06	quality_check()Date Cast	Valid date string parses correctly	snapshot_date = '2024-01-31' (all 240 inventory rows)	invalid date count = 0 for snapshot_date	PASS
WB -B- 07	quality_check()Date Cast	Null lot_expiry_date treated as invalid after cast	inventory_snapshot: 96 rows with no lot_expiry_date value	invalid date count = 96 for lot_expiry_date	PASS
WB -B- 08	quality_check()Date Cast	One actual_delivery_date unparseable	purchase_orders: 1 row with malformed/missing actual_delivery_date	invalid date count = 1 for actual_delivery_date	PASS
WB -B- 09	quality_check()Numeric Cast	Valid numeric string casts to double	quantity_on_hand = '150.0' across inventory rows	invalid numeric count = 0; column type = double	PASS
WB -B- 10	quality_check()Numeric Cast	Non-numeric string cast → produces null	Injected row with quantity_on_hand = 'N/A'	invalid numeric count = 1 (null after cast)	PASS
WB -B- 11	quality_check()Empty date_cols / numeric_cols	Function called with empty lists (suppliers)	suppliers: date_cols=[], numeric_cols=[]	Loops do not execute; only null/dup checks run; 10 rows returned	PASS
WB -B- 12	File Write(overwrite mode)	Validated DataFrames written to bronze/validated	All 6 clean DataFrames written with mode='overwrite'	6 folders created, each with 4 output file parts	PASS

The Bronze Layer test cases (Table 5.13) cover eight white box tests (WB-S-B-01 to WB-S-B-12) that validate the raw ingestion pipeline across five quality check categories. Null

checks confirmed that key columns such as material_id, material_name, category, and qty are non-empty, while also detecting null values in non-critical columns like lot_expiry_date. Duplicate checks successfully identified and flagged 20 injected duplicate rows in the dataset. Data type and cast checks verified that snapshot_date parses correctly as a date and that numeric columns like quantity_on_hand cast properly to double, with non-numeric strings such as "NaN" correctly caught and flagged. The empty data check confirmed that the pipeline handles edge cases gracefully when supplied with empty DataFrames simulating missing supplier data. Finally, the file write validation confirmed that all eight cleaned DataFrames were successfully written to the Bronze container, producing six folders each containing four output files. All twelve test cases returned a PASS result, demonstrating that the Bronze layer correctly handles data ingestion, quality validation, and output writing before data progresses to the Silver layer.

5.5.2.2 Test Cases – Silver Layer

WBT-S-01 to WBT-S-03: Recoverable Asset Flag (is_recoverable_asset)

Table 5.14 Recoverable Asset Flag (is_recoverable_asset)

Test ID	Module	Test Case	Input Condition	Expected Outcome	Result
WB-S-01	Silveris_recoverable_asset	Stock strictly greater than 12M consumption → flag = 1	quantity_on_hand = 300, consumption_12m = 0 (Chrome Yellow, MAT005)	is_recoverable_asset = 1	PASS
WB-S-02	Silveris_recoverable_asset	Stock equal to 12M consumption → flag = 0 (boundary)	quantity_on_hand = 300, consumption_12m = 300	is_recoverable_asset = 0 (not strictly greater)	PASS
WB-S-03	Silveris_recoverable_asset	Stock below 12M consumption → flag = 0	quantity_on_hand = 100, consumption_12m = 300	is_recoverable_asset = 0	PASS

WB-S-04	Silveris_recoverable_asset	Material with no production history (consumption_12m filled as 0 by fillna)	No production records for material_id; left join → fillna(0)	consumption_12m = 0; is_recoverable_asset = 1 if stock > 0	PASS
---------	----------------------------	---	--	--	-------------

The four test cases (WBT-S-01 to WBT-S-04) validated the logic for flagging recoverable assets, confirming that is_recoverable_asset is correctly set to 1 when stock quantity strictly exceeds 12-month consumption, set to 0 at the boundary where stock exactly equals consumption, remains 0 when stock falls below consumption, and correctly defaults to 1 for materials with no production history where consumption is filled as 0 via a left join fillna operation. All four tests passed.

WBT-S-05 to WBT-S-08: MC Dead Stock Flag (is_mc_dead_stock)

Table 5.15 MC Dead Stock Flag (is_mc_dead_stock)

Test ID	Module	Test Case	Input Condition	Expected Outcome	Result
WB-S-05	Silveris_mc_dead_stock	Zero 9M consumption AND no lot expiry → dead stock (both conditions true)	consumption_9m = 0, lot_expiry_date = null	is_mc_dead_stock = 1	PASS
WB-S-06	Silveris_mc_dead_stock	Zero 9M consumption BUT lot expiry exists → not dead stock (one condition false)	consumption_9m = 0, lot_expiry_date = '2025-06-01'	is_mc_dead_stock = 0	PASS
WB-S-07	Silveris_mc_dead_stock	Non-zero 9M consumption	consumption_9m = 150,	is_mc_dead_stock = 0	PASS

		AND no lot expiry → not dead stock	lot_expiry_date = null		
WB-S-08	Silveris_mc_dead_stock	Both conditions false → not dead stock	consumption_9m = 200, lot_expiry_date = '2025-12-31'	is_mc_dead_stock = 0	PASS

The four test cases (WBT-S-05 to WBT-S-08) validated the dead stock flagging logic, confirming that is_mc_dead_stock is set to 1 only when both conditions are simultaneously true zero 9-month consumption and no lot expiry date while correctly returning 0 when either a valid lot expiry date exists, consumption is non-zero, or both conditions are false. All four tests passed.

WBT-S-09 to WBT-S-11: MC Expired Flag (is_mc_expired)

Table 5.16 MC Expired Flag (is_mc_expired)

Test ID	Module	Test Case	Input Condition	Expected Outcome	Result
WB-S-09	Silveris_mc_expired	Lot expiry date before latest snapshot → expired	lot_expiry_date = '2023-06-01', latest_date = '2024-12-31'	is_mc_expired = 1	PASS
WB-S-10	Silveris_mc_expired	Lot expiry date exactly equals latest snapshot date (boundary)	lot_expiry_date = '2024-12-31', latest_date = '2024-12-31'	is_mc_expired = 0 (not strictly less than)	PASS
WB-S-11	Silveris_mc_expired	Lot expiry date in the future → not expired	lot_expiry_date = '2026-06-01', latest_date = '2024-12-31'	is_mc_expired = 0	PASS
WB-S-12	Silveris_mc_expired	Null lot_expiry_date → comparison evaluates to null (false branch)	lot_expiry_date = null	is_mc_expired = 0 (null comparison = false in Spark)	PASS

The four test cases (WBT-S-09 to WBT-S-12) validated the expiry flag logic, confirming that `is_mc_expired` is set to 1 when the lot expiry date falls strictly before the latest snapshot date, remains 0 at the boundary where both dates are equal, returns 0 when the lot expiry date is in the future, and correctly evaluates null lot expiry dates as false in Spark's null-safe comparison. All four tests passed.

WBT-S-13 to WBT-S-14: Magna Carta Combined Flag (`is_magna_carta`)

Table 5.17 Magna Carta Combined Flag (`is_magna_carta`)

Test ID	Module	Test Case	Input / Condition	Expected Outcome	Result
WB-S-13	Silveris_magna_carta	Either sub-flag is 1 → magna_carta = 1 (OR logic, dead stock true)	<code>is_mc_dead_stock = 1, is_mc_expired = 0</code>	<code>is_magna_carta = 1</code>	PASS
WB-S-14	Silveris_magna_carta	Both sub-flags are 0 → magna_carta = 0	<code>is_mc_dead_stock = 0, is_mc_expired = 0</code>	<code>is_magna_carta = 0</code>	PASS

The two test cases (WBT-S-13 to WBT-S-14) validated the combined Magna Carta flag using OR logic, confirming that `is_magna_carta` is correctly set to 1 when either the dead stock or expired sub-flag is true, and returns 0 only when both sub-flags are simultaneously false. Both tests passed.

WBT-S-15 to WBT-S-17: MC Provision Calculation(`mc_provision_amount`)

Table 5.18 MC Provision Calculation(`mc_provision_amount`)

Test ID	Module	Test Case	Input / Condition	Expected Outcome	Result
---------	--------	-----------	-------------------	------------------	--------

WB -S- 15	Silvermc_provision_amo unt	MC- flagged material → provision = stock × unit_cost	is_magna_carta = 1, quantity_on_hand = 2800, unit_cost = 7.60 (Vinyl Resin, MAT010)	mc_provision_amo unt = 21280.00	PASS
WB -S- 16	Silvermc_provision_amo unt	Non-MC material → provision = 0.0	is_magna_carta = 0, quantity_on_hand = 200, unit_cost = 10.00	mc_provision_amo unt = 0.0	PASS
WB -S- 17	Silvermc_provision_amo unt	Aggregate d total provision across all MC materials	Sum of all mc_provision_amo unt where is_magna_carta = 1	Total matches RM value printed in summary report	PASS

The three test cases (WBT-S-15 to WBT-S-17) validated the provision amount calculation, confirming that mc_provision_amount is correctly computed as stock multiplied by unit cost for MC-flagged materials, set to 0.0 for non-MC materials regardless of stock or unit cost values, and that the aggregated total provision across all MC materials matches the expected RM value printed in the summary report. All three tests passed.

WBT-S-18 to WBT-S-21: Stockout Risk Flags (is_below_reorder, is_stockout_risk)

Table 5.19 Stockout Risk Flags (is_below_reorder, is_stockout_risk)

Test ID	Module	Test Case	Input Condition /	Expected Outcome	Result
WB-S-18	Silveris_below_reorder	Stock below reorder level → flag = 1	quantity_on_hand = 80, reorder_level = 100	is_below_reorder = 1	PASS
WB-S-19	Silveris_below_reorder	Stock equal to reorder	quantity_on_hand = 100,	is_below_reorder = 0 (not strictly	PASS

		level (boundary) → flag = 0	reorder_level = 100	less)	
WB-S-20	Silveris_stockout_risk	Stock below safety stock threshold → flag = 1	quantity_on_hand = 5, consumption_12m = 365, lead_time_days = 14 → safety_stock = 14.0	is_stockout_risk = 1 (5 < 14)	PASS
WB-S-21	Silveris_stockout_risk	Zero consumption → avg_daily = 0 → safety_stock = 0 → no risk	consumption_12m = 0, lead_time_days = 14 → safety_stock = 0.0	is_stockout_risk = 0 (stock >= 0)	PASS

The four test cases (WBT-S-18 to WBT-S-21) validated the stockout risk flagging logic, confirming that is_below_reorder is correctly set to 1 when stock falls strictly below the reorder level, remains 0 at the boundary where stock equals the reorder level, and that is_stockout_risk is correctly triggered when stock falls below the computed safety stock threshold while returning 0 when consumption is zero and safety stock evaluates to 0. All four tests passed.

WBT-S-22 to WBT-S-24: Sales Order Impact (Q7 – impacted_orders join chain)

Table 5.20 Sales Order Impact (Q7 – impacted_orders join chain)

Test ID	Module	Test Case	Input / Condition	Expected Outcome	Result
WB-S-22	Silverimpacted_orders	Filter: only Open orders are selected from sales_orders	sales_orders with status = 'Open', 'Pending', 'Closed'	Only rows with status = 'Open' passed to join	PASS
WB-S-23	Silverimpacted_orders	Inner join: stockout	Stockout material_id linked	Order appears in impacted_orders	PASS

		material maps to a finished product (match exists)	to finished_product in production_orders; finished_product in open sales order	output	
WB-S-24	Silverimpacted_orders	Inner join: stockout material has no matching sales order (no match)	Stockout material_id not found in any open order's finished product chain	Material absent from impacted_orders (inner join drops it)	PASS

The three test cases (WBT-S-22 to WBT-S-24) validated the Q7 impacted orders join chain, confirming that only sales orders with an "Open" status are selected from the source table, that stockout materials correctly map through the production orders join chain to appear in the impacted orders output, and that materials with no matching open sales order finished product are correctly excluded through the inner join drop. All three tests passed.

WBT-S-25: Window Function – Latest Inventory Snapshot

Table 5.21 Window Function – Latest Inventory Snapshot

Test ID	Module	Test Case	Input / Condition	Expected Outcome	Result
WB-S-25	SilverLatest Snapshot(Window)	Multiple snapshot dates per material → only latest row retained	Material M001 has snapshots on 2024-10-31, 2024-11-30, 2024-12-31	Only 2024-12-31 row survives (rn = 1 filter)	PASS

The single test case (WBT-S-25) validated the window function logic for retaining only the latest inventory snapshot per material, confirming that when material M001 has multiple snapshot dates, only the row corresponding to the latest date of 2024-12-31 survives after applying the row number filter where m = 1. The test passed.

5.5.2.3 Test Case Gold Layer – Synapse SQL Views

Table 5.22 Gold Layer – Synapse SQL Views

Test ID	Module	Test Case	Input / Condition	Expected Outcome	Result
WB-G-01	gold.dim_materialSchema Cast	View enforces VARCHAR and FLOAT types on OPENROWSET output	Parquet file at gold/dim_material/** read via OPENROWSET WITH clause	Columns typed as VARCHAR(20/100/50/20), FLOAT for numeric fields; no cast error	PASS
WB-G-02	gold.dim_supplierSchema Cast	reliability_rating returned as VARCHAR(20), not numeric	Parquet contains string reliability ratings (e.g. 'High', 'Medium')	reliability_rating = VARCHAR(20); no truncation or cast error	PASS
WB-G-03	gold.dim_sales_orderDate Columns	order_date and required_delivery_date cast to DATE type	Parquet stores dates as int96/timestamp; WITH clause specifies DATE	Both columns returned as DATE; NULL for any unparseable value	PASS
WB-G-04	gold.dim_production FLOAT cast	quantity_consumed column cast to FLOAT in view definition	Parquet column is double precision float	quantity_consumed readable as FLOAT; no overflow error	PASS
WB-G-05	gold.dim_dateINT cast	date_id and year/quarter columns cast to INT	Parquet stores date_id as int32, year/quarter as int32	All INT columns return correct integer values; month_name = VARCHAR(20)	PASS
WB-G-06	gold.fact_inventory_statusFlag Columns	All binary flag columns (is_*) defined as INT in view schema	Fact parquet stores flags as int32 (0/1)	is_recoverable_asset, is_mc_dead_stock, is_mc_expired, is_magna_carta, is_below_reorder, is_stockout_risk	PASS

				all read as INT	
WB-G-07	gold.fact_inventory_statusNullable FK columns	supplier_id, production_order_id, sales_order_id may be NULL (left join origin)	Fact rows where no PO or sales order was matched in Silver → null FK values in parquet	NULL values returned without error; view does not reject nulls in FK columns	PASS
WB-G-08	gold.impact_orderSilver path read	impact_order view reads from silver/container (not gold/)	BULK path = 'silver/impact_sales_orders/**'	View correctly queries silver container; returns sales_order_id, material_id, quantity_on_hand columns	PASS
WB-G-09	Wildcard Path(**) Resolution	Wildcard ** in BULK path matches multiple parquet part-files	Each Gold/Silver folder contains multiple .parquet part files written by Spark	OPENROWSET reads all part files and unions them; full row count matches Silver output	PASS
WB-G-10	Schema Mismatch(Negative test)	Column name in WITH clause does not exist in parquet file → error raised	Deliberately queried with incorrect column alias in WITH clause	Synapse raises column-not-found error; view creation blocked — confirms strict schema binding	PASS

The ten test cases (WB-G-01 to WB-G-10) validated the Gold layer Synapse SQL views across schema casting, path resolution, foreign key integrity, and schema binding. WB-G-01 confirmed that the view enforces correct VARCHAR and FLOAT types via OPENROWSET with a WITH clause, returning columns typed as VARCHAR(20) or FLOAT(50) for numeric fields without cast errors. WB-G-02 verified that the reliability_rating column is returned as VARCHAR(20) rather than numeric, correctly preserving string ratings such as "High" and "Medium". WB-G-03 confirmed that order_date and required_delivery_date are correctly cast to DATE type, with Parquet storing dates as int96/timestamp and the WITH clause handling the conversion for any unparseable values returning NULL. WB-G-04 validated that the quantity_consumed

column is correctly cast to FLOAT as defined in the view definition, ensuring no overflow errors. WB-G-05 confirmed that data_id and year/quarter columns are correctly cast to INT, with Parquet storing these as int32 and Synapse returning the correct integer values while column names remain as VARCHAR(20). WB-G-06 verified that all binary flag columns such as is_recoverable_asset, is_mc_dead_stock, is_mc_expired, is_magna_carta, is_below_reorder, and is_stockout_risk are all correctly read as INT as defined in the view schema. WB-G-07 confirmed that supplier_id, production_order_id, and sales_order_id fact rows where no purchase order or sales order was matched in Silver correctly return NULL FK values rather than rejecting rows, preserving left join origin integrity. WB-G-08 validated that the impacted orders Silver path correctly reads from the Silver container rather than Gold, returning the expected columns including sales_order_id, material_id, and quantity_on_hand. WB-G-09 confirmed that the wildcard path in BULK resolves correctly across multiple parquet part-files written by Spark, with OPENROWSET reading all parts and the row count matching the Silver output. Finally, WB-G-10 verified that Synapse raises a column-not-found error when a view is created with an incorrect column name in the WITH clause, confirming that strict schema binding is enforced and view creation is blocked on mismatch. All ten Gold layer test cases returned a PASS result.

White Box Testing Summary

The white box testing for the PPG Inventory Management Analytics System covered a total of 54 test cases across three pipeline layers Bronze, Silver, and Gold systematically validating the internal logic, transformation rules, data quality checks, and schema enforcement embedded within the Medallion Architecture. At the Bronze layer, 12 test cases (WB-S-B-01 to WB-S-B-12) verified raw data ingestion quality checks including null detection, duplicate identification, data type casting, and file write operations, all of which passed successfully. At the Silver layer, 25 test cases (WBT-S-01 to WBT-S-25) validated the core transformation logic across eight functional modules covering recoverable asset flagging, dead stock and expiry classification, Magna Carta combined flag derivation, provision amount calculation, stockout risk flagging, sales order impact join chains, and window function snapshot filtering, with all cases returning a pass result. At the Gold layer, 10 test cases (WB-G-01 to WB-G-10) verified the Synapse SQL view schema enforcement, column type casting, wildcard path resolution, foreign key null handling, and schema binding validation, all of which passed without error. Across all three layers, every test case

returned a PASS result, confirming that the pipeline correctly enforces data quality at ingestion, applies accurate business transformation logic at the Silver layer, and exposes well-structured and schema-compliant views at the Gold layer for downstream Power BI consumption.

5.5.3 User testing

Table 5.23 User Testing

Test ID	Task	Expected Result	Actual Result	Status
UT-01	View total Recoverable Assets (RA)	User can identify total RA materials from dashboard	User identified RA count correctly	Pass
UT-02	Identify MC Dead Stock materials	User can locate MC Dead Stock section	User found information successfully	Pass
UT-03	View total MC Provision Amount	User can see financial write-down value	User viewed provision amount correctly	Pass
UT-04	Filter dashboard by Material Category	Dashboard updates visualizations correctly	Filter worked as expected	Pass
UT-05	Identify materials with Stockout Risk	User can locate stockout risk materials	User identified risky materials	Pass
UT-06	Find affected Sales Orders	User can determine impacted customer orders	User found affected orders successfully	Pass
UT-07	Navigate between dashboard pages	User can move between pages easily	Navigation completed without issue	Pass

The seven user testing test cases (UT-01 to UT-07) validated the usability and functionality of the Power BI dashboard from an end-user perspective. UT-01 confirmed that users could

correctly identify the total Recoverable Asset count directly from the dashboard. UT-02 verified that users were able to successfully locate the MC Dead Stock materials section without difficulty. UT-03 confirmed that users could view the total MC Provision Amount financial write-down value as expected. UT-04 validated that filtering the dashboard by Material Category correctly updates all visualizations, with the filter working as expected. UT-05 confirmed that users could successfully identify materials flagged with Stockout Risk from the dashboard. UT-06 verified that users were able to determine affected customer orders through the impacted sales orders section. UT-07 confirmed that navigation between dashboard pages was smooth and completed without issue. All seven user testing cases returned a Pass result, demonstrating that the Power BI dashboard is functionally accessible, visually informative, and navigable for end users interacting with the inventory analytics system.

5.6 Chapter summary

Chapter 5 presented the implementation and testing of the Recoverable Assets (RA) and Inventory Risk Management (IRM) system. The implementation phase involved developing the complete end-to-end data pipeline using Microsoft Azure services and Power BI. Data from multiple operational sources were ingested into the Bronze layer, transformed and validated in the Silver layer, and structured into an analytics-ready Galaxy Schema within the Gold layer. The final processed data was visualized through interactive Power BI dashboards designed to answer the project's predefined business questions.

The chapter also described the testing activities conducted to verify system functionality and reliability. White-box testing was performed to validate internal processing logic, transformation rules, and business calculations such as Recoverable Assets identification, Magna Carta Dead Stock detection, expired material classification, and financial provision calculations. Black-box testing was conducted to evaluate system outputs, dashboard functionality, data accuracy, and user requirements fulfillment without examining internal code structures.

The testing results demonstrated that the system successfully met its functional and non-functional requirements by producing accurate inventory risk analytics, reliable reporting

outputs, and efficient dashboard visualizations. Overall, the implementation and testing phase confirmed that the proposed solution is capable of supporting inventory monitoring, risk identification, and data-driven decision-making within PPG Industries.

REFERENCES

- Chen, M.-C. and Huang, S.-H. (2003) ‘Credit scoring and rejected instances reassigning through evolutionary computation techniques’, *Expert Systems with Applications*, 24(4), pp. 433–441.
- Clerc, M. and Kennedy, J. (2002) ‘The particle swarm - explosion, stability, and convergence in a multidimensional complex space’, *IEEE Transactions on Evolutionary Computation*, 6(1), pp. 58–73.
- Databricks. (2022). Medallion Architecture. Databricks Documentation. Retrieved from <https://www.databricks.com/glossary/medallion-architecture>
- der Maaten, L. J. P., Postma, E. O., den Herik, H. J., van der Maaten, L., Postma, E. O., van den Herik, J., der Maaten, L. J. P., Postma, E. O. and den Herik, H. J. (2009) ‘Dimensionality Reduction: A Comparative Review’, *Technical Report TiCC TR 2009-005*, 10(January), pp. 1–41.
- Gosnell, M., Woodley, R., Hicks, J. and Cudney, E. (2014) ‘Exploring the Mahalanobis-Taguchi Approach to Extract Vehicle Prognostics and Diagnostics’, in *Computational Intelligence in Vehicles and Transportation Systems (CIVTS), 2014 IEEE Symposium on*, pp. 84–91.
- Gupta, A. (2015) ‘Classification of Complex UCI Datasets Using Machine Learning Algorithms Using Hadoop’, *International Journal of Scientific & Technology Research*, 4(5), pp. 85–94.
- Hu, J., Zhang, L., Liang, W. and Wang, Z. (2009) ‘Incipient mechanical fault detection based on multifractal and MTS methods’, *Petroleum Science*, 6(2), pp. 208–216.
- Huang, C.-L., Chen, Y. H. and Wan, T.-L. J. (2012) ‘The mahalanobis taguchi system—adaptive resonance theory neural network algorithm for dynamic product designs’, *Journal of Information and Optimization Sciences*, 33(6), pp. 623–635.
- Jain, A. K. A. K., Duin, R. P. W. and Mao, J. (2000) ‘Statistical pattern recognition: a

review', *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(1), pp. 4–37.

Jawad, Z. N., & Balázs, V. (2024). Machine learning-driven optimization of enterprise resource planning (ERP) systems: A comprehensive review. *Beni-Suef University Journal of Basic and Applied Sciences*, 13(1), 4. doi:<https://doi.org/10.1186/s43088-023-00460-y>

Jayaraman, K. D., & Jain, S. (2024). Leveraging Power BI for advanced business intelligence and reporting. *International Journal for Research in Management and Pharmacy*, 13(11), 21-36.

Khalid, S., Khalil, T. and Nasreen, S. (2014) 'A survey of feature selection and feature extraction techniques in machine learning', *2014 Science and Information Conference*, pp. 372–378.

Li, C., Yuan, J. and Qi, Z. (2015) 'Risky group decision-making method for distribution grid planning', *International Journal of Emerging Electric Power Systems*, 16(6), pp. 591–602.

Lv, Y. and Gao, J. (2011) 'Condition prediction of chemical complex systems based on Multifractal and Mahalanobis-Taguchi system', in *ICQR2MSE 2011 - Proceedings of 2011 International Conference on Quality, Reliability, Risk, Maintenance, and Safety Engineering*, pp. 536–539.

Mathur, Utkarsh, et al. "Evaluating Automation Using Supply Chain Cloud Solutions." *International Journal of Computer Trends and Technology*, vol. 72, no. 7, 31 July 2024, pp. 53–62, <https://doi.org/10.14445/22312803/ijctt-v72i7p106>. Accessed 31 Mar. 2026.

Microsoft. (2023). What is Azure Data Factory? Microsoft Azure Documentation. Retrieved from <https://learn.microsoft.com/en-us/azure/data-factory/introduction>

Motwani, R. and Raghavan, P. (1996) 'Randomized algorithms', *ACM Computing Surveys*, 28(1), pp. 33–37.

Qinbao Song, Jingjie Ni and Guangtao Wang (2013) 'A Fast Clustering-Based Feature

- Subset Selection Algorithm for High-Dimensional Data', *IEEE Transactions on Knowledge and Data Engineering*, 25(1), pp. 1–14.
- Rao, V. M. and Singh, Y. P. (2013) 'Decision Tree Induction for Financial Fraud Detection', in *Proceeding of the International Conference on Artificial Intelligence in Computer Science and ICT (AICS 2013)*, pp. 321–328.
- SAP SE. (2023). SAP Materials Management (MM) — Inventory Management and Physical Inventory. SAP Help Portal. Retrieved from <https://help.sap.com>
- Shi, Y. and Eberhart, R. (1998) 'A modified particle swarm optimizer', *1998 IEEE International Conference on Evolutionary Computation Proceedings. IEEE World Congress on Computational Intelligence*, pp. 69–73.
- Silver, E. A., Pyke, D. F., & Thomas, D. J. (2017). *Inventory and Production Management in Supply Chains* (4th ed.). CRC Press.
- Soylemezoglu, A., Jagannathan, S. and Saygin, C. (2011) 'Mahalanobis-Taguchi system as a multi-sensor based decision making prognostics tool for centrifugal pump failures', *IEEE Transactions on Reliability*, 60(4), pp. 864–878.
- Syntetos, A. A., Babai, M. Z., & Altay, N. (2012). On the demand distributions of spare parts. *International Journal of Production Research*, 50(8), 2101–2117.
- Theodoridis, S., Koutroumbas, K., Holmstrom, L. and Koistinen, P. (2009) *Pattern Recognition*, Wiley Interdisciplinary Reviews Computational Statistics.
- Wahyuni, I. T., Anwar, C., & Pratiwi, W. (2024). The important role of microsoft power business intelligence tool for analyzing unit cost. In *Proceeding of the International Conference on Multidisciplinary Research for Sustainable Innovation* (Vol. 1, No. 1, pp. 585-590).
- Ware, C. (2019). *Information Visualization : Perception For Design*. Morgan Kaufmann.
- Wishdesk. (2024). What is iterative design approach & how can it benefit your project? Wishdesk. Retrieved from <https://wishdesk.com/blog/what-is-iterative-design-approach>

- Zaki, M. J., Wong, L., Berry, M. J. A., Linoff, G. S., Hegland, M., Zaki, M. J. and Wong, L. (2003) 'Data Mining Techniques', *WSPC/Lecture Notes Series: 9in x 6in*, 10(1–2), p. 545.
- Dey, S. (2023). Surviving major disruptions: Building supply chain resilience and visibility through rapid information flow and real-time insights at the “edge”. *Sustainable Manufacturing and Service Economics*, 2, 100008.
- Eboigbe, E. O., Farayola, O. A., Olatoye, F. O., Nnabugwu, O. C., & Daraojimba, C. (2023). Business intelligence transformation through AI and data analytics. *Engineering Science & Technology Journal*, 4(5), 285-307.
- Ntabe, E., LeBel, L., Munson, A., & Santa-Eulalia, L. (2015). A systematic literature review of the supply chain operations reference (SCOR) model application with special attention to environmental issues. *International Journal of Production Economics*, 169, 310–332. <https://doi.org/10.1016/j.ijpe.2015.08.008>
- Supply Chain Council. (2010). Supply Chain Operations Reference Model Version 10.0. Supply Chain Council, Inc. 856p.